Kontekstihaulla rikastettuun päättelyyn (RAG) ja vektoritietokantaan perustuva paikallinen tekoälymuistiinpanosovellus

Pyry Majander

Haaga-Helia ammattikorkeakoulu

Tradenomin tutkinto

Amk-opinnäytetyö

2025

Tiivistelmä

Tekijä
Pyry Majander

Tutkinto
Tradenomi, tietojenkäsittely

Raportin/Opinnäytetyön nimi
Kontekstihaulla rikastettuun päättelyyn (RAG) ja vektoritietokantaan perustuva tekoälymuistiinpanosovellus

Sivu- ja liitesivumäärä
65 + 3

Toiminnallisessa opinnäytetyössä kehitettiin kontekstihaulla rikastettuun päättelyyn perustuva tekoälymuistiinpanosovellus, joka toimii käyttäjän laitteella ilman verkkoyhteyttä. Työllä ei ollut toimeksiantajaa, ja sovellus kehitettiin henkilökohtaisesta kiinnostuneisuudesta LLM- ja NLP-teknologiaan, tietosuojaan sekä ohjelmistokehitykseen. Tekoäly on ollut varsin ajankohtainen aihe tietojenkäsittelyn alalla vuoden 2022 marraskuusta lähtien, jolloin OpenAI:n julkaisema ChatGPT laukaisi globaalin tekoälykilpailun. Merkittävin osuus inferenssistä suurten kielimallien kanssa toteutuu suurten teknologiakorporaatioiden tarjoamissa pilvipalveluissa. Tähän liittyy tietosuojaongelmia, joita on käsitelty tietoperustan lopussa.

Raportin alkuosiossa on käyty syventävällä otteella läpi koneoppimisen, syväoppimisen ja luonnollisen kielen käsittelyn historiaa. Tekoälyn osa-alueita luonnollisen kielen käsittelyn ulkopuolella ei kuitenkaan juuri käsitellä. Seuraavassa osiossa on syvennytty neuronien ja neuroverkkojen rakenteen sekä kouluttamisen matemaattiseen toimintalogiikkaan, minkä jälkeen modernien suurten kielimallien pohjalla vaikuttava transformer-arkkitehtuuri on selitetty olennaisimmilta osiltaan kattavasti läpi. Tietoperustan lopussa on käsitelty luonnollisen kielen käsittelyä vektoritietokantojen ja RAG-tekniikan kontekstissa, sekä erilaisia tietosuojaongelmia.

Opinnäytetyön lopputuloksena syntyi ”MemoFort AI” muistiinpanosovelluksen MVP-versio. Tämä prototyyppi toteuttaa suunnitellun toimintalogiikan sovellukselle. Sovellus rakentuu kahdesta resursseja jakavasta prosessista ”NoteApp” & ”ChatApp”, jotka toimivat käytännössä omina sovelluksinaan. NoteAppin avulla käyttäjä voi tallentaa muistiinpanojaan erilaisiin itse luomiinsa kategorioihin. ChatApp puolestaan mahdollistaa inferenssin paikallisten kielimallien kanssa graafisen käyttöliittymän kautta. Paikallinen kielimalli pääsee käsiksi NoteAppilla luotuihin muistiinpanoihin, ja voi käyttää niitä kontekstina vastauksissaan.

MemoFortia ei ole saatettu julkisuuteen, mutta sen toteutusta on dokumentoitu tässä raportissa laajasti ja yksityiskohtaisesti. Raportti sisältää runsaasti koodinäytteitä ja sovelluksen toimintalogiikkaa selittäviä kaavioita, millä on pyritty avoimen kehityskulttuurin tukemiseen. Opinnäytetyön lopussa on mainittu myös jatkokehitysideoita ja erilaisia ohjelmistokehitysprosessin aikana kohdattuja ongelmia, jotta työ hyödyttäisi vastaavan sovelluksen kehittämisestä kiinnostuneita lukijoita.

Sovelluksen suorituskykyä testattiin suuntaa-antavasti mittaamalla paikallisesti asennettujen kielimallien nopeutta, ja niiden vastausten laatua. Suuntaa-antavien testien metodiikkaa ja suoritusympäristöä on kuvattu raportissa pyrkien hyviin tieteellisiin käytäntöihin. Tuloksia on havainnollistettu taulukoissa, ja niitä pohdittiin opinnäytetyön viimeisessä luvussa. Sovellus todettiin hyödylliseksi, mutta tämä edellyttää tehtävään sopivan kielimallin valitsemista.

Asiasanat
Muistiinpanosovellus, vektoritietokanta, RAG, NLP, Ollama, tekoäly, transformer

Sisällys

1 Johdanto 1
2 Tekoäly ja luonnollisen kielen käsittely 2
2.1 Syväoppimisen ja luonnollisen kielen prosessoinnin lyhyt historia 2
2.1.1 Porraslogiikkayksiköt ja NLP 2
2.1.2 Konnektionismin kausi 5
2.1.3 Uusi, syväoppiva vuosituhat 7
2.2 Koneoppiminen ja syväoppiminen 9
2.3 Keinotekoiset neuroverkot (ANN): neuroni ja monikerroksinen perseptroni 10
2.4 Myötäsyöttö 12
2.4.1 Aktivaatiofunktiot 13
2.5 Verkon opettaminen 14
2.5.1 Kustannusfunktiot, delta-sääntö ja oppimisnopeus 15
2.5.2 Gradientti- ja vastavirta-algoritmi 16
2.6 Suuret kielimallit (LLM) ja transformer-arkkitehtuuri 18
2.6.1 Positionaalinen enkoodaus 19
2.6.2 Huomiolohko 20
3 Älykkäät hakumetodit ja muistirakenteet 22
3.1 Tokenisointi ja upotusmatriisit 22
3.2 RAG – kontekstihaulla laajennettu päättely 24
4 Suuret kielimallit ja tietosuoja 25
4.1 Kielimallien sensuuri 25
4.2 Tietosuojaongelmat LLM-inferenssissä 27
5 MemoFort AI 29
5.1 Valitut teknologiat 29
5.2 Kokonaisarkkitehtuuri, yhteiset resurssit ja käynnistäminen 30
5.3 NoteApp 31
5.3.1 Graafisen käyttöliittymän rakenne 32
5.3.2 Valikko 34
5.3.3 Kategoriat ja niiden käsitteleminen 36
5.3.4 Muistiinpanojen tallentaminen 39
5.4 ChatApp 40
5.4.1 Vektoritietokanta ja retriever 40
5.4.2 Puhekuplien konstruointi 42
5.4.3 Tekoälyagentti 44
5.4.4 Mallien vaihtaminen 47

6 Sovelluksen suorituskyky 50
6.1 Testidata ja testikysymykset 50
6.2 Suorituskykyvertailun metodiikka 52
6.3 Suorituskykyvertailun toteutusympäristö 53
6.3.1 Testiin valitut kielimallit 54
7 Tulokset ja pohdinta 56
7.1 Suorituskykytestin tulokset 56
7.2 Yleisiä ajatuksia sovelluksen toimivuudesta sekä kannattavuudesta 57
7.3 Jatkokehitysideat 58
7.4 Opinnäytetyön ja henkilökohtaisen kehittymisen reflektointi 59
Lähteet 62
Liitteet 66
Liite 1. Suorituskykytestien tulokset, raakadata 66

1 Johdanto

Vuoden 2022 lopussa tekoäly nousi ennenkuulumattomaan suosioon myös koneoppimisyhteisön ulkopuolella, kun OpenAI:n keskustelupohjaisen inferenssimallinsa (Kimothi 2025, 4). Suurten kielimallien ympärillä on runsaasti ihailijoita sekä kriitikoita, mutta ilmiön laaja vaikutus moderniin maailmaan tämän opinnäytetyön kirjoitusvuonna on kiistaton.

Merkittävä osa yleisesti käytetyistä LLM-sovelluksista eli suurista kielimalleista toimii kansainvälisten suurkorporaatioiden pilvipalveluissa (Guran, Knauf, Ngo, Petrescu, Rellermeyer 2024, 1). Tämä herättää esimerkiksi opinnäytetyön luvussa neljä käsitellyistä syistä huolta henkilökohtaisesta tietosuojasta. Nämä kielimallit eivät näe myöskään omia dokumenttejamme, ellei niitä manuaalisesti syötetä mallille kehotteen (prompt) mukana. Nykyisin useissa palveluissa kielimallit voivat toki hakea internetistä tietoa, mutta tämä muodostaa uuden ongelman: kuinka luotettavaa tämä tekoälyn hakema tieto on? Lähde tulisi tarkastaa joka tapauksessa, sillä kielimallit voivat ymmärtää verkosta noutamansa informaation väärin. Esimerkiksi juridiikan kontekstissa malli voi suosia omiin parametreihinsa tallentunutta yleistä oikeuskäytäntöä poikkeustapaukseen nähden (Magesh ym. 2024, 6). Kielimallien hakiessa tietoa yleisiltä verkkosivuilta, syntyy myös uusi XSS-hyökkäyspinta, jonka avulla mustahattuhakkerit voivat pyrkiä injektoimaan vahingollista käytöstä kielimallille (Dhamani & Engler 2024, 112–113).

Opinnäytetyössä kehitettiin MemoFort-niminen prototyyppisovellus, joka korjaa edellä mainittuja ongelmia. Sovelluksen toiminta perustuu Ollaman tarjoamaan, paikallisesti asennettuun LLM-kielimalliin, vektoritietokantaindeksointiin ja retrieval-augmented generation (RAG) -tekniikkaan. Kyseessä on Python-ohjelmointikielellä toteutettu modulaarinen muistiinpanosovellus, joka tallentaa muistiinpanot CSV-tiedostoon, indeksoi ne ChromaDB -tietokantaan ja tarjoaa käyttöliittymän, jonka kautta käyttäjä voi käsitellä omia muistiinpanojaan paikallisen kielimallin tukemana. Sovelluksella on moderni, selkeä ja intuitiivinen käyttöliittymä, jotta sitä voidaan käyttää myös keskinkertaisilla tietotekniikkataidoilla ilman erillistä koulutusta. MemoFort on suunniteltu pääasiassa opiskelijoille, minkä takia kannettavien tietokoneiden rajalliset laskentaresurssit on otettu huomioon.

2 Tekoäly ja luonnollisen kielen käsittely

Tekoälyllä ei ole yksiselitteistä määritelmää, mutta useat määritelmät jakavat samanlaisia elementtejä. Kyseessä on joukko teknologioita, jotka suorittavat tai jäljittelevät ihmisen kognitioon ja aisteihin perustuvia prosesseja. Nämä teknologiat eivät ole tarkkaan määriteltyjä, ja siksi tekoäly onkin lähinnä abstrakti kuvaus koneistetusta älykkyydestä. (Cox & Mazumdar 2024, 331–332.)

Tekoälyn erilaisia sovelluskohteita ovat esimerkiksi sähköpostilaatikon roskapostisuodatin, valokuvista kasvoja tunnistava algoritmi, itseohjautuvan auton ’aivot’ ja suuret kielimallit, kuten Googlen Gemini. Käyttökohteita koneistetulle älykkyydelle on useita, mutta tässä raportissa keskitytään erityisesti luonnollisen kielen käsittelyyn (NLP) tekoälyn viitekehyksessä.

2.1 Syväoppimisen ja luonnollisen kielen prosessoinnin lyhyt historia

Opinnäytetyön kirjoittamisprosessin aikana ihmiskunta on elänyt merkittävän tekoälyinnostuksen myllerryksessä jo useita vuosia. Elon Universityn vuonna 2025 toteuttaman kyselytutkimuksen perusteella jopa yli puolet Yhdysvaltain väestöstä hyödyntää LLM-teknologiaa (Elon University News Bureau, 2025). Havaintojeni perusteella oletan, että monelle suurten kielimallien taustalla vaikuttavat algoritmit ja näiden historia eivät ole kuitenkaan juuri tuttuja.

Suuret ja suositut sovellukset kuten OpenAI:n kehittämä ChatGPT ja Microsoftin Copilot eivät ole syntyneet käden käänteessä, vaan inkrementaalisesti vuosikymmenten mittaisen innovoinnin saatossa. Tässä luvussa tutustutaan kehitykseen erityisesti syväoppivien neuroverkkojen osalta, mutta esimerkiksi kuvankäsittelyyn liittyviä aspekteja kuten konvoluutioverkkoja ei ole huomioitu, sillä ne eivät liity luonnollisen kielen käsittelyyn.

Syväoppimisen historia voidaan jakaa kolmeen aikakauteen, joiden välissä on ollut hiljaisia ”AI-talvina” tunnettuja ajanjaksoja. Kelleherin mukaan nämä aikakaudet voidaan karkeasti jaotella vallinneen ajattelumaailman ja teknologioiden perusteella porraslogiikkayksikköihin ~1940–1965, konnektionismiin ~1980–1995 ja syväoppimiseen noin vuodesta ~2005 eteenpäin. (Kelleher 2019, 94.)

2.1.1 Porraslogiikkayksiköt ja NLP

Tekoälyn kehityksen varhaisessa vaiheessa yhtenä pääasiallisena tavoitteena oli matkia aivojen toimintaa konealgoritmisin keinoin eli tietokoneen haluttiin oppivan ’ajattelemaan’ itsenäisesti. Vuonna 1943 neurofyysikko W. S. McCulloch sekä matemaatikko W. Pitts julkaisivat artikkelin, joka käsitteli aivojen hermosolujen ja synapsien mallintamista sekä koneellista laskentaa. Heidän kehittämänsä McCulloch-Pitts -neuronin avulla voidaan suorittaa loogisia operaatioita kuten kon-

junktioita, disjunktioita ja negaatioita, ja sitä pidetään modernin perseptronin historiallisena esiasteena (Kelleher 2019, 96–97.) Keinotekoisten neuronien perusideana oli matkia biologisten aivojen toimintaa binääriluokittelun kautta: ”*The activity of the neuron is an ‘all-or-none’ process . . . A certain fixed number of synapses must be excited within the period of latent addition in order to excite a neuron at any time, and this number is independent of previous activity and position on the neuron.”* (McCulloch 1943, 118.)

Muutamia vuosia myöhemmin psykologi Donald O. Hebb julkaisi kirjan, jossa käsiteltiin psykologian ja aivojen fysiologisten ominaisuuksien yhteyttä toisiinsa. Kirjassa esitettiin Hebbin postulaattina tunnettu väite, jonka mukaan synapseihin eli aivoneuronien välisiin yhteyksiin tallentuu tietoa, ja että oppiminen perustuu näiden yhteyksien dynaamiseen muutokseen. Kirjan seurauksena useat tutkijat pyrkivät yhdistämään porrasaktivaatioon sekä Boolen algebraan perustuvan McCulloch-Pitts neuronin ja Hebbin postulaattiin perustuvan oppimismekanismin. Ratkaisuista merkittävin oli Frank Rosenblattin vuonna 1958 kehittämä perseptroni. (Kelleher 2019, 97–98.)

Rosenblattin työ ei jäänyt kuitenkaan vain idean tasolle, sillä hän myös rakensi ensimmäisen tunnetun oppivan neuroverkon eli Mark 1 perseptronin IBM 704 -tietokoneen ohjelmistoksi. Perseptroni oppi tunnistamaan yksinkertaisia muotoja valokuvasta. (Kelleher 2019, 98–99; Lefkowitz 25.9.2025.) Muutama vuosi myöhemmin Rosenblatt myös todisti, että perseptronin opetusalgoritmi konvergoi haluttua neuronien painokerroinkonfiguraatiota kohti (Kelleher 2019, 104).

NLP eli luonnollisen kielen käsittely oli myös yksi keskeisimmistä tavoitteista jo ensimmäisten tietokoneiden synnyttyä. Luonnollisen kielen käsittelyllä tarkoitetaan ihmisen puhumien kielien hyödyntämistä tietotekniikassa. Yksi erinomainen esimerkki tästä on J. Weizenbaumin kehittämä ELIZA -keskustelubotti, jota pidetään modernien keskustelubottien esiasteena. (Dhamani & Engler 2024, 3.) ELIZA oli hyvin yksinkertainen tietokoneohjelma, jota ei rakennettu moderneilla neuroverkkoarkkitehtuureilla (Berry 2023, 2).

Weizenbaum koki luomuksensa saaneen yliarvostetun statuksen julkisessa diskurssissa, eikä hän itse pitänyt luomustaan älykkäänä (Basset 2019, 804). Hän havainnollistaa omassa kirjoituksessaan, että ELIZA:n kanssa keskustelleet henkilöt projisoivat mielikuvan älykkyydestä ohjelmaan keskustellessaan sen kanssa. Weizenbaum kertoo myös esimerkiksi sihteerinsä kysyneen lupaa voidakseen keskustella ohjelman kanssa yksityisesti, mikä indikoi sitä, että ihmisen oli helppo erehtyä pitämään konetta inhimillisenä entiteettinä. (Weizenbaum 1967 s. 474, 478; Dhamani & Engler 2024, 4.)

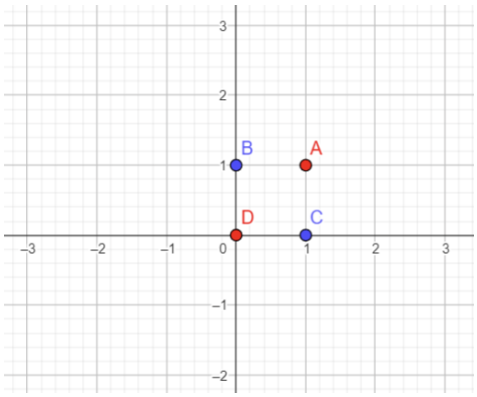
1960-luvun loppupuolella Yhdysvaltain puolustushallinto eli nykyinen DARPA rahoitti venäjää englanniksi kääntävien tietokoneohjelman kehittämistä ja konekäännöksen tutkimusta. Tällainen käännösohjelma olisi ollut merkittävä työkalu kylmän sodan aikaisessa maailmanjärjestyksessä. Ohjelmaa ei kuitenkaan saatu toimimaan kunnolla kymmenen vuoden työskentelystä huolimatta. Vuonna 1966 Automatic Language Processing Advisory Committee (ALPAC) kritisoi Yhdysvaltain hallinnolle konekäännösten realismia ja toimintamahdollisuuksia, mikä vähensi uskoa tekoälyjärjestelmiä kohtaan. (Jiang, Li, Luo, Yin & Kaynak 2022, 4.)

Vuosikymmenen lopussa - vuonna 1969 - Marvin Minsky ja Seymour Papiert julkaisivat teoksen ”Perceptrons”, joka kritisoi yksinkertaisen perseptronin kykyä ratkaista vain lineaarisia ongelmia, ja jäävän jumiin epälineaarisesti separoituvien funktioiden kanssa. (Jiang ym. 2022, 4.) Kirjan kritiikki kohdistui erityisesti siihen, että lineaarisia kuvauksia tuottava yksinkertainen perseptroni ei voi oppia luokittelemaan sellaisia datapisteitä, joita ei voi jakaa toisistaan suoran avulla. Esimerkiksi AND ja OR -funktiot olivat yksinkertaisen perseptronin ratkaistavissa, mutta vaikeammat funktiot kuten XOR-funktiot eivät. (Kelleher 2019, 107–108.) Looginen XOR-portti vastaanottaa kaksi bittiä, jotka voivat olla arvoiltaan 1 tai 0. Jos syötteet ovat samat, on tuloste nolla eli epätosi. Jos syötteet eroavat toisistaan, on tuloste yksi eli tosi. XOR-portti voidaan esittää totuustauluna:

Taulukko 1. XOR-totuustaulu

Syöte A	**Syöte B**	**Tuloste**
0	0	0 luokka X
0	1	1 luokka Y
1	0	1 luokka Y
1	1	0 luokka X

Pisteet [0,0] sekä [1,1] kuuluvat luokkaan X, ja pisteet [1,0] sekä [0,1] kuuluvat luokkaan Y. Koordinaatiston pisteistä voidaan havaita, että näiden luokkien eli punaisten ja sinisten pisteiden erottaminen toisistaan yhden suoran avulla on mahdotonta (Kelleher 2019, 109).



Kuva 1. XOR-funktion arvot koordinaatistossa (mukaillen Kelleher 2019, 109)

Perseptronin lineaarisuuden ongelmaan oli kuitenkin keksitty teoreettinen ratkaisu jo kritiikin esitysvuosina, ja tämä oli usean perseptronin yhdistäminen monikerroksiseksi verkoksi. Monikerroksisen verkon kouluttamiseen ei kuitenkaan tunnettu vielä konkreettisia lähestymistapoja. Vuonna 1971 ukrainalainen tutkija A. Ivakhnenko julkaisi kuvauksen kahdeksankerroksisen neuroverkon kouluttamisesta, mutta hänen tutkimuksensa jätettiin huomiotta. (Kelleher 2019, 111–112.) Perceptrons -kirjan julkaisun jälkeen koneoppimisen tutkimusrahoitus tyrehtyi, ja alkoi niin sanottu ensimmäinen ”AI-talvi” eli ajanjakso, jolloin laajempi kiinnostus tekoälyä kohtaan oli lopahtanut. (Jiang ym. 2022, 4.)

2.1.2 Konnektionismin kausi

Konnektionismin aika syväoppimisen viitekehyksessä perustui tulkintaan, jossa koneoppimisjärjestelmä on kokonaisuus, jonka älykkyys syntyy sen omien tietoja käsittelevien rakenneosien yhteistoiminnasta. Tekoälyn ajatellaan siis syntyvän hajautetusta tietoaineistosta ja sitä käsittelevien neuronien vuorovaikutuksesta emergenttinä ilmiönä. (Kelleher 2019, 118.)

Vuonna 1982 kiinnostus koneoppimiseen heräsi jälleen kun J. Hopfield julkaisi artikkelin assosiatiivisista neuroverkoista. Tällainen ns. Hopfield-verkko kykeni muistamaan tunnistamiaan hahmoja, ja kyseessä on yksi ensimmäisistä takaisinkytketyistä neuroverkkoarkkitehtuureista. (Kelleher s. 113–114.) Myöhemmin syntyi takaisinkytkettyjen verkkojen erityistyyppi eli rekurrentti neuroverkko. Rekurrentit neuroverkot käsittelevät sarjallisia syötteitä, kuten kielen peräkkäisiä sanoja, ja niitä koulutetaan iteratiivisesti diskreetissä ajassa. Jokaisen syötteen kohdalla piiloneuronien laskema tulos tallennetaan puskuriin, josta se ohjataan kontekstiksi seuraavalle laskutoimitukselle. Näin jokainen käsiteltävä elementti saa kontekstia aiemmin käsitellyiltä elementeiltä. (Kelleher 2019, 126.)

Takaisinkytketyt neuroverkot ratkaisivat myös aiempien mallien ongelman eli staattisen syötekoon rajoitteen. RNN-verkoissa kaikki ensimmäisen kerroksen neuronit ovat yhteydessä kaikkiin seuraavan kerroksen neuroneihin, mutta yksittäisiä yhteyksiä voidaan kytkeä pois päältä. Tämä mahdollistaa vaihtelevan mittaisten syötteiden tarjoamisen verkolle. (Brousseau & Sharp 2025, 51.) Suuret syötevolyymit aiheuttavat kuitenkin uuden ongelman. Mitä runsaammin syötteitä verkko saa, sitä pienempi on yksittäisten elementtien suhteellinen vaikutus kokonaisuuteen. Tämä tarkoittaa käytännössä sitä, että neuroverkon opettamiseen vaadittavat gradientit pienenevät, kunnes lopulta katoavat täysin. Tämä ilmiö tunnetaan katoavan gradientin ongelmana. Vaihtoehtoisessa tilanteessa gradientti ”räjähtää” saadessaan liian suuria arvoja. (Lappin 2025, 26–27; Brousseau & Sharp 2025, 51.)

Takaperoista pyöristysalgoritmia tai vastavirta-algoritmia (back propagation) voidaan pitää yhtenä syväoppimisen olennaisimmista algoritmeista. Kyseessä on algoritmi, joka mahdollistaa syvän neuroverkon opettamisen. Sen toimintaperiaatteen keksi useampi henkilö jotakuinkin samanaikaisesti 1970-luvun loppupuolella, joista suomalainen Seppo Linnainmaa julkaisi sen ensimmäisenä. (Griewank 2012, 389, 391–392.) Kuitenkin vasta vuonna 1986 D. Rumelhartin, G. Hintonin & R. Williamsin muodostama tutkimusryhmä PDP julkaisi ensimmäisen merkittävän artikkelin takaperoisen pyöristysalgoritmin hyödyntämisestä koneoppimisen kontekstissa (Kelleher 2019, 114). Rumelhartin julkaistua artikkelinsa kiinnostus perseptroneja kohtaan heräsi jälleen (Du, Leung, Mow & Swamy 2022, 1).

Karkeasti vastavirta-algoritmissa jokaiselle verkon kytkennälle annetaan satunnainen painokerroin, ja verkon läpi syötetään dataa, jonka annetaan virrata neuronien läpi viimeiseen tuloskerrokseen. Tuloskerroksen arviota verrataan toivottuun tulokseen, ja jokaiselle sen neuronille lasketaan osuus potentiaalisesti muodostuneesta virhearviosta. Tuloskerroksen neuronien painokertoimia päivitetään virheen perusteella, minkä jälkeen selvitetään, että miten aiemman piilokerroksen neuronit puolestaan vaikuttivat tuloskerroksen neuroneihin. Sama toistetaan jokaiselle verkon kerrokselle eli jokaisen neuronin painot päivitetään taaksepäin, kunnes kaikki verkon kytkennät on päivitetty. (Kelleher 2019, 115–116.) Tätä selitetään tarkemmin opinnäytetyön luvussa 2.5.

Takaisinkytkennän prosessi on riippuvainen derivoinnista, ja sen toiminnan mahdollistaminen edellytti muutosta aktivaatiofunktioihin. Aikaisemmissa perseptronimalleissa aktivaatiofunktio esitettiin porrasfunktion muodossa. (Kelleher 2019, 116.) Porrasfunktion kulmakerroin on ääretön kohdassa $x = 0$, eikä sen derivaattaa ole määritelty (Kämäräinen 2023, 47). Tämä derivoimattomuus tekee gradientin laskemisesta mahdotonta varhaisten porraslogiikkayksiköiden kohdalla. Ongelma ratkaistiin korvaamalla aktivaatiofunktio derivoituvilla funktioilla, kuten tanh-funktiolla ja sigmoidisella funktiolla. (Kelleher 2019, 114–116.)

Aiemmin mainittu katoavan gradientin ongelma ilmaantui vastavirta-algoritmin yhteydessä. Useat tutkijat huomasivat, että algoritmi toimi mainiosti matalissa neuroverkoissa, mutta syvemmissä jopa todella huonosti. Vuonna 1991 S. Hochreiter ja J. Schmidhuber esittivät diplomityössään ongelman fundamentaalisen luonteen. Verkon neuronien päivittäminen takaperoisesti perustuu derivaattojen ketjuttamissääntöön. Tämä johtaa siihen, että virhesignaali kerrotaan toistuvasti pienillä arvoilla, kunnes signaali vaimenee merkityksettömän pieneksi verkon ensimmäisten piilokerrosten kohdalla. Jokaisella koulutuskierroksella verkon ensimmäisten kytkentöjen painokertoimet saattavat siis säilyä täysin muuttumattomina, mikä estää verkon oppimisen. (Kelleher 2019, 116.)

Vuonna 1997 Hochreiter ja Schmidhuber julkaisivat uudenlaisen pitkän työmuistiyksikön eli LSTM (Long Short Term Memory) -arkkitehtuurin ratkaisuna katoaviin gradientteihin. Olennainen ero

LSTM-verkkojen ja RNN-verkkojen välillä on se, että LSTM-verkkojen erityinen muistirakenne mahdollistaa tiedon tallentamisen ja siirtämisen pidempien aikavälien yli. Neuroverkkojen kouluttaminen oli kuitenkin 1990-luvun loppupuolella edelleen haastavaa ja koneoppimisyhteisön mielenkiinto oli kohdistunut muihin metodeihin kuten helpommin koulutettaviin tukivektorikoneisiin. Kiinnostus koneoppimiseen hiipui jälleen, ja näin alkoi toinen ’AI-talvi’. (Kelleher 2019, 128–129.)

2.1.3 Uusi, syväoppiva vuosituhat

Syväoppiminen eroaa koneoppimisesta pääasiassa siten, että siinä koneoppivat verkot ovat syviä. Kun alati kehittyvä teknologia johti tietokoneiden laskentakyvyn paranemiseen ja internet tarjosi yhä enemmän koulutusdataa neuroverkoille, tuli neuroverkkoarkkitehtuureista yhä syvempiä ja kompleksisempia (Dhamani & Engler 2024, 7.) Vuosituhannen vaihteen tienoilla tutkijat ymmärsivät, että katoavan gradientin ongelma ei ollut ehdoton este vaan pikemminkin hidastava haaste, joka voidaan selättää. Aktivaatiofunktioiden ja neuroverkon painojen alustamiseen keksittiin parempia ratkaisuja, ja syvien neuroverkkojen esikouluttamisesta tuli käytännöllistä. (Kelleher 2019, 133–135.)

Vuonna 2006 Hinton, Osindero ja Teh julkaisivat uudenlaisen menetelmän syvien verkkojen opettamiseen. Tämä ahneen kerroksittaisen esiopetuksen -menetelmä (greedy layer-wise pretraining) perustuu siihen, että verkon kerroksia koulutetaan yksi kerrallaan tunnistamaan syötteestä olennaisimmat tekijät. Menetelmää kutsutaan ahneeksi, koska verkon kerrokset optimoidaan riippumatta myöhemmistä kerroksista. Yksi suosittu tapa toteuttaa ahnetta, kerroksittaista esiopetusta oli rakentamalla neuroverkko autoenkoodaajakerrosten avulla. (Kelleher 2019, 130.)

Autoenkoodaaja on yksinkertainen pieni neuroverkko, johon kuuluu syötekerros, yksi piilokerros eli enkoodaaja ja ulostulokerros eli dekoodaaja. Autoenkoodaajan tehtävä on rekonstruoida mahdollisimman tarkka mallinnus sen saamasta syötteestä. Sen piilokerroksessa voi olla vähemmän neuroneja kuin syöte- ja ulostulokerroksissa, jolloin verkon on opittava vain tärkeimmät syötettä kuvaavat piirteet. Kun opetusprosessi on valmis, autoenkoodaajan ulostulokerros poistetaan, mutta sen piilokerrosten oppimat painokertoimet jäävät staattisiksi parametreiksi. Tämän jälkeen verkkoon lisätään seuraava enkoodaaja, joka tismalleen samalla periaatteella vastaanottaa syötteeksi edeltävän autoenkoodaajan tunnistamat piirteet, ja pyrkii tunnistamaan niistä vielä olennaisempia tekijöitä. Esikoulutetun neuroverkon viimeinen kerros opetetaan kuitenkin normaalisti eli sen tehtävä ei ole enää mallintaa aiemman autoenkoodaajan löytämiä piirteitä. Koko verkon läpi syötetään täysin uutta dataa, ja verkko pyrkii tunnistamaan siitä piirteitä, ja muuttamaan painokertoimiaan. Tätä vaihetta kutsutaan verkon virittämisvaiheeksi (fine tuning). (Kelleher 2019, 130–133.)

Merkittävimmät voimat syväoppimisen kehitykselle olivat kuitenkin kehittyvien näytönohjaimien soveltaminen neuroverkkojen kouluttamisessa ja huima kasvu hyödynnettävissä olevan datan määrässä. Vuosien 2004 ja 2007 välillä useat tutkijat tekivät läpimurtoja verkkojen kouluttamisessa soveltamalla siihen näytönohjaimia. Prosessi oli aluksi kuitenkin monimutkainen, sillä näytönohjaimia ei alun perin suunniteltu koneoppimista varten. Vuonna 2007 NVIDIA julkaisi ohjelmointirajapinnan nimeltä CUDA. Tämä mahdollisti näytönohjainten entistä helpomman soveltamisen laskentatehtäviin C-ohjelmointikielen avulla. Lisäksi alati yleistyvät IoT-sensorit, internet ja sosiaalinen media tarjosivat massiivisen määrän dataa hyödynnettäväksi syvien verkkojen kouluttamista varten. (Kelleher 2019, 138–140.)

Vuonna 2014 tutkijat keksivät uudenlaisen konseptin, jota kutsutaan huomioksi. Aiemmin luonnollista kieltä käsitteleviä verkkoja oli koulutettu kerroksittain autoenkoodaajakerroksilla. Tässä uudessa, huomioksi kutsutussa ratkaisussa voidaan tarkastella koko tokenisoitujen merkkijonojen syötesarjaa kerrallaan, ja arvioida, että mitkä sanat liittyvät toisiinsa. (Dhamani & Engler 2024, 8–9.)

Vain muutama vuosi myöhemmin Googlen tutkijat julkaisivat uuden neuroverkkoarkkitehtuurin, jossa huomion rooli korostui. He osoittivat uuden arkkitehtuurin olevan merkittävästi nopeampi kuin aiemmat luonnollisen kielen käsittelyyn erikoistuneet neuroverkot, jotka olivat perustuneet sekvensoiviin arkkitehtuureihin kuten RNN-arkkitehtuuriin. Tämän uuden huomiokerroksiin ja rinnakkaislaskentaan perustuvan neuroverkkoarkkitehtuurin nimeksi annettiin transformer. (Vaswani ym. 2017, 2, 7.) Transformer voidaan kääntää suomeksi ”muuntajaksi” tai ”muuntujaksi” koneoppimisen kontekstissa (Kämäräinen 2023, 185). Koska transformer-verkolla laskeminen oli merkittävästi tehokkaampaa niin resurssien kuin ajan kannalta, pystyttiin syviä neuroverkkoja kouluttamaan suuremmalla määrällä dataa kuin koskaan aiemmin. Tämän laukaisi uuden suurten kielimallien (large language model, LLM) aikakauden. Vain vuoden kuluttua transformerin keksimisestä OpenAI julkaisi ensimmäisen GPT-mallin, joka perustui transformer-arkkitehtuuriin. GPT on lyhenne sanoista generative pre-trained transformer eli generatiivinen esikoulutettu transformeri. (Dhamani & Engler 2024, 8–9.)

Vuoden 2022 marraskuun 30. päivänä OpenAI julkaisi GPT3.5 -kielimalliin perustuvan istuntopohjaisen keskustelubotin. ChatGPT keräsi julkaisunsa jälkeen viidessä vuorokaudessa miljoona käyttäjää ja vain pari kuukautta myöhemmin tammikuussa 2023 käyttäjiä oli kertynyt jo 100 miljoonaa, mikä rikkoi kaikkien aikojen ennätyksen minkään sovelluksen käyttäjämäärän kasvuvauhdissa. ChatGPT:n julkaiseminen oli sensaatiomainen tapahtuma, joka toi tekoälyn ensimmäistä kertaa laajalla skaalalla koneoppimisyhteisön ulkopuolisen väestön tietoisuuteen. (Dhamani & Engler 2024, 26–27.)

Vaikka suuret kielimallit ovat osoittautuneet äärimmäisen hyödyllisiksi erilaisissa tehtävissä kuten kielen kääntämisessä, luovassa kirjoittamisessa, käyttäjien neuvomisessa, koodin tuottamisessa ja useissa muissa yleishyödyllisissä tehtävissä, sisältyy niiden käyttöön kuitenkin ei toivottuja ilmiöitä ja riskitekijöitä. Mallit on koulutettu massiivisella määrällä dataa, ja ne voivat oppia sellaisia kaavoja, joita kehittäjät eivät toivo mallien tuottavan. (Dhamani & Engler 2024, 19–20.) Kielimallien toiminta perustuu seuraavien sanojen ennustamiseen eli mallit voivat hallusinoida ja niiden tuottama teksti on osin ennalta-arvaamatonta. Esimerkiksi Googlen julkaisema Bard-keskustelubotti kertoi suorassa lähetyksessä virheellistä tietoa James Webb avaruusteleskoopin historiasta, mikä pudotti Googlen osakkeiden markkinaosuutta noin sadalla miljardilla Yhdysvaltain dollarilla. (Dhamani & Engler 2024, 23.)

Mallien kouluttamiseen ja käyttöön liittyy myös suurta energiankulutusta. Erään arvion mukaan esimerkiksi GPT-3 -mallin kouluttaminen aiheutti noin viidensadan metrisen tonnin hiilidioksidipäästöt ilmakehään. Vaikka kielimallien todellisista päästöistä on niukasti tietoa, on määrien arvioitu olevan massiivisia, ja niitä syntyy myös inferenssissä eli käyttäjien hyödyntäessä valmiita, koulutettuja malleja. (Dhamani & Engler 2024, 24–25.) Suurten kielimallien muita ongelmakohtia käydään myöhemmin läpi sensuurin ja tietosuojan viitekehyksessä luvussa neljä.

2.2 Koneoppiminen ja syväoppiminen

Koneoppimisella tarkoitetaan tietokoneohjelmaa, joka oppii muodostamaan toivotun tulosteen arvioimalla dataa ilman, että sen parametreja ohjelmoidaan manuaalisesti (Valiant 1984, 1134). Käytännössä koneoppimisalgoritmin ydintehtävä on muodostaa tietoaineiston perusteella funktio, joka selittää tietoaineiston piirteiden välisiä suhteita. Kun algoritmin tuottaman funktion suorituskyky on arvioitu riittäväksi, varsinainen oppiminen päättyy. Kyseessä on siis funktio, joka tuottaa uuden funktion. (Kelleher 2019, 19–22.)

Koneoppiminen voidaan määritellä myös hahmontunnistuksen mekanismina ja sellaisen järjestelmän rakentamisena, joka voi oppia omista kokemuksistaan. Koneoppimisprosessin elinkaari voidaan jakaa kolmeen vaiheeseen: koulutusvaiheeseen, validointivaiheeseen ja sovellusvaiheeseen. (Gollapudi 2016, 3, 5.) Opetusvaiheessa koneoppimisalgoritmi eristää tietoaineistosta uuden funktion, jota kutsutaan malliksi. Sovellusvaiheessa mallin parametrit eivät enää muutu eli se on rakenteeltaan pysyvä. Tällaisen valmiin mallin soveltamista kutsutaan inferenssiksi. (Kelleher 2019, 21–22.) Koneoppimisen kolme keskeistä tekijää ovat tietoaineisto, funktioita käsittelevä algoritmi ja jokin algoritmin tuloksia arvioiva virhemittari (Kelleher 2019, 29).

Syväoppimisella kuvaillaan ihmisen aivotoimintaa matkivia keinotekoisia neuroverkkoja, jotka koostuvat useista keinotekoisista neuroneista. Nämä keinotekoiset neuronit ovat itsessään koneoppimisalgoritmeja. Syväoppimisen fundamentaalinen perusta on siis se, että koneoppimisalgoritmit saavat syötteikseen muiden koneoppimisalgoritmien tuloksia, ja näiden algoritmien yhdistelmä muodostaa uuden mallin. Syväoppivat verkot sisältävät useita piilotettuja kerroksia, jotka tyypillisesti oppivat tunnistamaan tietoaineiston piirteitä itsenäisesti. (Kelleher 2019, 57, 64.) Mitä syvälotteisempi neuroverkko on, sitä tehokkaammin se oppii (Dhamani & Engler 2024, 7). Kompleksisia epälineaarisia, kuvauksia mallinnettaessa syväoppivat verkot suoriutuvat muita ratkaisuja paremmin (Kelleher 2019, 142). Verkko on syvä, kun siinä on vähintään kaksi piilokerrosta, ja sen syvyys tai mitta on piilokerrosten ja tuloskerroksen summa (Kelleher 2019, 66, 68).

Oppiva tietojärjestelmä voi perustua ohjattuun oppimiseen, vahvistusoppimiseen tai ohjaamattomaan oppimiseen. Ohjatussa oppimisessa tietoaineiston elementeillä on oltava tulosarvo, jota verrataan järjestelmän ehdottamaan arvoon. Tällaista aineistoa ei välttämättä ole kuitenkaan saatavilla, ja sellaisen luominen voi olla kallista ja haastavaa. Ohjaamattomassa oppimisessa tietoaineiston ulostuloarvot ovat tuntemattomia. Sitä sovelletaan esimerkiksi asiakassegmenttien klusteroinnissa, ja oppimisen tuloksia voidaan ohjata puhtaasti intuition varassa. Vahvistusoppimisessa puolestaan virtuaalinen entiteetti oppii ympäristönsä antamasta palautteesta. Tämä voi olla esimerkiksi sellainen tietokonevastustaja videopelissä, joka oppii tehokkaimmat tavat pelata ihmistä vastaan vastaanottamalla negatiivista palautetta virheistään sekä positiivista palautetta onnistumisistaan. Suurin osa koneoppimisesta perustuu kuitenkin ohjattuun oppimiseen. (Kelleher 2019, 32–34.)

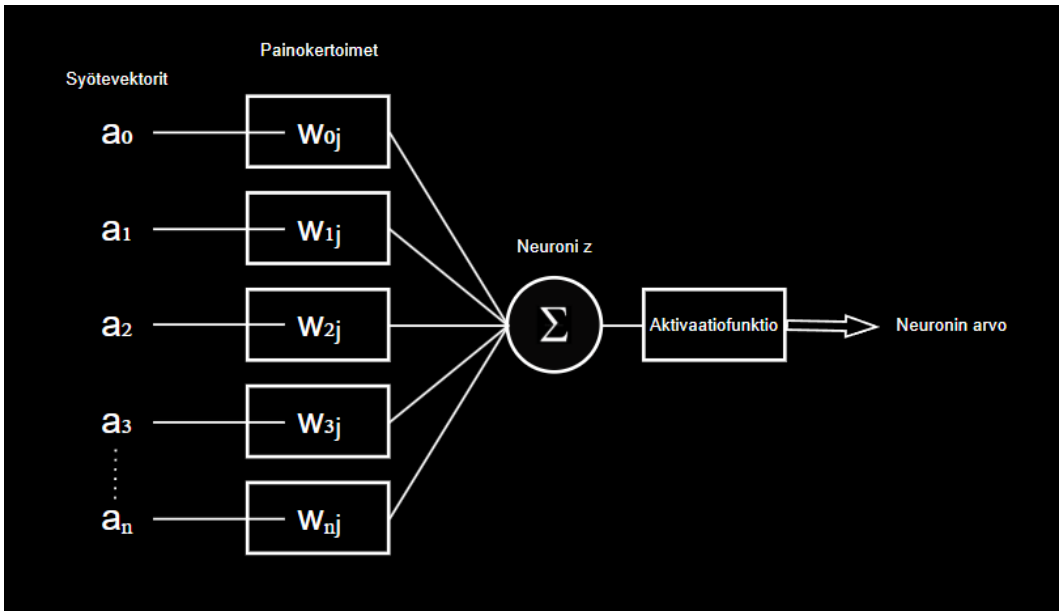
Ohjattavan oppimisen merkittävä ongelma on tietoaineiston piirteiden suunnittelu. Tämä on usein kallista ja vaikeaa, eikä aineiston alan asiantuntijoilla ole välttämättä yksimielistä käsitystä siitä, että mitkä piirteet merkitsevät enemmän kuin toiset. (Kelleher 2019, 36, 51.) Esimerkiksi kahdella taloustieteilijällä voisi olla voimakkaasti poikkeavat käsitykset siitä, että millä painokertoimella eri talousindikaattorit kuten työttömyysaste ja euriborkorot pitäisi ottaa huomioon makrotalouden ennusteissa.

2.3 Keinotekoiset neuroverkot (ANN): neuroni ja monikerroksinen perseptroni

Keinotekoinen neuroverkko on parametrisoitu funktio, jonka ydintehtävä on approksimoida toinen funktio. (Carlini, Liu, Erlingsson, Kos & Song 2019, 268). Sen perusyksikkö on matemaattinen neuroni, joka voidaan myös esittää funktiona. Neuronin toimintalogiikka voidaan jakaa kahteen osaan: pistetulon eli painotetun summan laskemiseen ja aktivaatiofunktion soveltamiseen pistetuloon. Kelleher 2019 64, 69.) Käytännössä neuroni on yksi neuroverkon solmu, joka vaikuttaa muihin neuro-

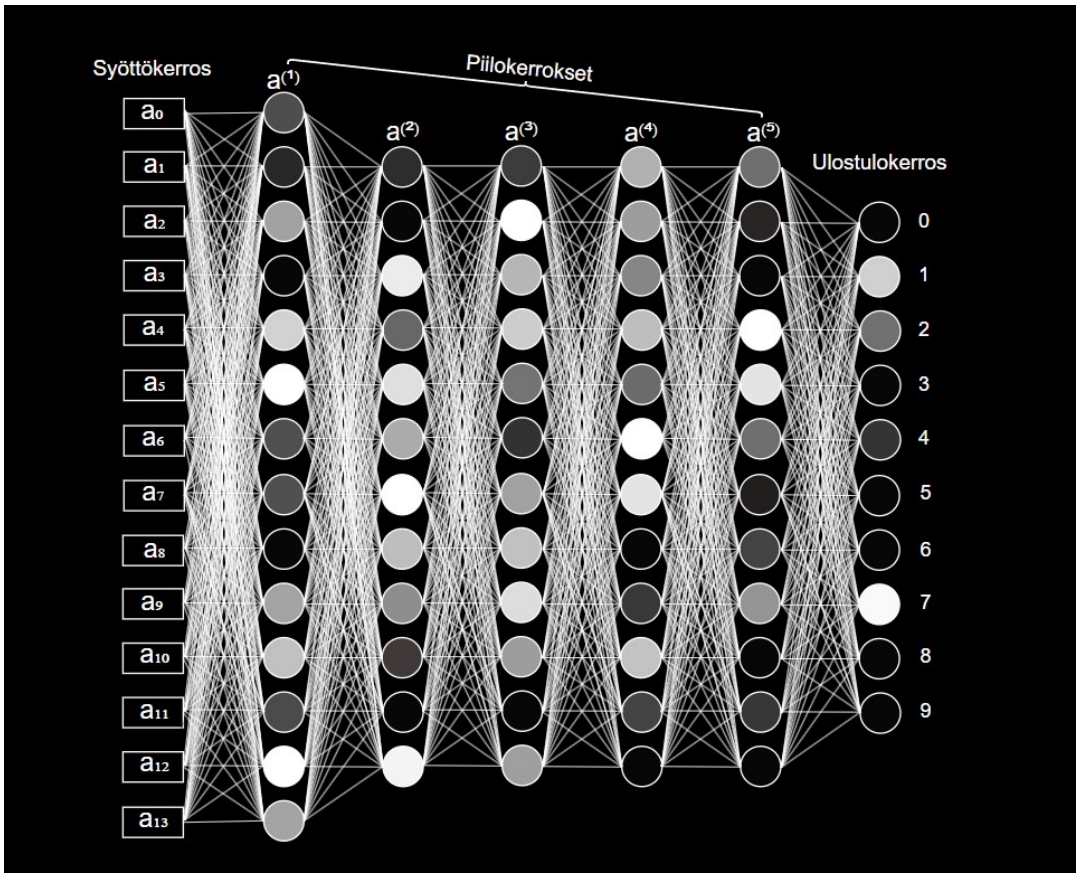
neihin tietyllä painokertoimella w. Neuroverkon painot ovat siis kytkentöjä neuronien välillä. Yksittäinen keinotekoinen neuroni z voidaan esittää kaavalla $z = \varphi(w_{i,:} \cdot a + b)$, jossa φ on aktivaatiofunktio, $w_{i,:}$ on painokerroinmatriisin rivivektori, a on aiemman piilokerroksen neuronien aktivaatioista tai verkon syötevektorista rakentuva sarakevektori, ja b on vakiotermi.

Kaava voidaan pelkistää lisäämällä syöte $a_0 = 1$, ja kertomalla tämä painokertoimella w_0, jonka arvo vastaa vakiotermin arvoa. Tässä tapauksessa vakiotermiä ei tarvitse enää laskea mukaan yhteenlaskulla. Tämä on myös hyödyllistä, sillä vakiotermin muuntaminen painotetuksi summaksi mahdollistaa sen, että neuroverkko voi oppia sillekin optimaalisen arvon. Kun vakiotermin rooli muuttuu yhtälössä yhdeksi painoksi, voidaan kaikki syötteiden painotetut summat laskea vektorien kertolaskuina, mikä mahdollistaa tehokkaamman laskennan matriisikertolaskuihin ja rinnakkaislaskentaan erikoistuneilla näytönohjaimilla. (Kelleher 2019, 48, 84–86.) Kun vakiotermistä tehdään paino, saadaan neuronin matemaattinen kaava kompaktiin muotoon $\varphi(\sum_{i=0}^{n} a_i w_i)$ (Kelleher 2019, 83–84).



Kuva 2. Keinotekoinen neuroni (mukaillen Kelleher 2019, 70)

Kuten ehkä neuroverkon nimestä voi päätellä, ei siinä ole vain yhtä neuronia, vaan niitä on yleensä massiivinen määrä. Esimerkiksi jo melko vanhanaikaisessa GPT3-kielimallissa parametreja on 175 miljardia kappaletta (Drori 2023, 148). Neuroverkkoja on erilaisia, mutta syväoppimisessa yleinen tyyppi on monikerroksinen perseptroni, josta käytetään myös nimitystä monikerrospäättelin. Neuronit eivät ole verkossa sikin sokin, vaan ne on yleensä järjestetty kerroksiin. Tätä kerroksittaista rakennetta on havainnollistettu seuraavassa kuvassa.



Kuva 3. Monikerroksisen perseptronin topologia (mukaillen Sanderson & Pullen 2017)

2.4 Myötäsyöttö

Kun verkon läpi syötetään dataa ennustetta varten, kutsutaan tätä myötäsyötöksi (forward pass). Syötekerroksen neuronit eivät suorita laskutoimituksia. Ne ovat muistipaikkoja, jotka välittävät tiedon eteenpäin piilokerrosten neuroneille, joissa varsinainen laskeminen tapahtuu. (Kelleher 2019, 68.) Neuroverkot oppivat approksimoimaan funktion hajottamalla sen pienempiin funktioihin, jotka lopuksi yhdistetään (Kelleher 2019, 19).

Täysin kytketyssä monikerroksisessa perseptronissa jokainen neuroni saa syötteen kaikilta edeltävän kerroksen neuroneilta, ja välittää aktivaatioarvonsa kaikille seuraavan piilokerroksen neuroneille. Ensimmäisessä piilokerroksessa neuronit saavat syötevektorin syötekerrokselta, ja jokaisen syötekomponentin ja neuronin välistä yhteyttä kuvataan painokertoimella. Painokertoimien määrä on identtinen syötevektorin dimensioiden kanssa. Myöhemmissä piilokerroksissa syötevektori koostuu edeltävän piilokerroksen neuronien aktivaatioarvoista.

Jokainen neuroni laskee siis painotetun summan syötevektorin ja oman painovektorinsa alkioista. Syötevektorin $\mathrm{X} = [\mathrm{x}_0, \mathrm{x}_1, \mathrm{x}_2, \mathrm{x}_3, \ldots, \mathrm{x}_\mathrm{n}]$ ja painovektorin $\mathrm{W} = [\mathrm{w}_0, \mathrm{w}_1, \mathrm{w}_2, \mathrm{w}_3, \ldots, \mathrm{w}_\mathrm{n}]$ alkiot kerrotaan ensin keskenään, ja lopuksi tulot summataan, jolloin neuroni z_i saa arvon $\mathrm{w}_0\mathrm{x}_0 + \mathrm{w}_1\mathrm{x}_1 +$

$w_2x_2 + w_3x_3 + \ldots + w_nx_n$. (Kelleher 2019, 70.) Tavallaan neuroni laskee siis ensin syöte- ja painovektorin hadamard-tuotteen eli alkiokohtaiset tulot $W \odot X = [w_1x_1, w_2x_2, w_3x_3, \ldots w_nx_n]$. Tämän jälkeen hadamard-tuotteesta alkiot summataan yhteen. Hadamard-tuotetta ei tarvitse laskea erikseen, ja se on esitetty tässä vain havainnollistamisen vuoksi. Kokonaisen piilokerroksen tasolla tämä operaatio toimii matriisikertolaskuna, missä painokerroinmatriisi W ja syötevektori A kerrotaan toisillaan. Jokaiseen pistetuloon lisätään myös vakiotermi, mikäli siitä ei ole määritelty painokerrointa. Lopputuloksena saadaan uusi lineaarinen vektori Z. Painomatriisin rivivektori vastaa yksittäisen neuronin kaikkia painokertoimia: $w_{0,0}a_0 + w_{0,1}a_1 + w_{0,2}a_2 + b_0 = z_0$.

$$\begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} \\ w_{1,0} & w_{1,1} & w_{1,2} \\ w_{2,0} & w_{2,1} & w_{2,2} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} z_0 \\ z_1 \\ z_2 \end{bmatrix}$$

Jotta verkko oppisi esittämään tehokkaasti epälineaarisia kuvauksia, tulee pistetulo kääriä vielä aktivaatiofunktion sisälle. Aktivaatiofunktion tarkoitus on muuttaa neuronin aktivaatio ei-lineaariseen muotoon. Aktivaatiofunktion soveltamisen jälkeen neuronin sanotaan saaneen aktivaatioarvon. (Kelleher 2019, 69–70.) Yksi piilokerros voidaan esittää näin:

$\varphi\left(\begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} \\ w_{1,0} & w_{1,1} & w_{1,2} \\ w_{2,0} & w_{2,1} & w_{2,2} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}\right)$. Kompaktimmin: $a^{(\ell)} = \varphi(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)})$, missä yläindeksi ℓ indikoi verkon tiettyä kerrosta (Goodfellow, Bengio & Courville 2016, 191).

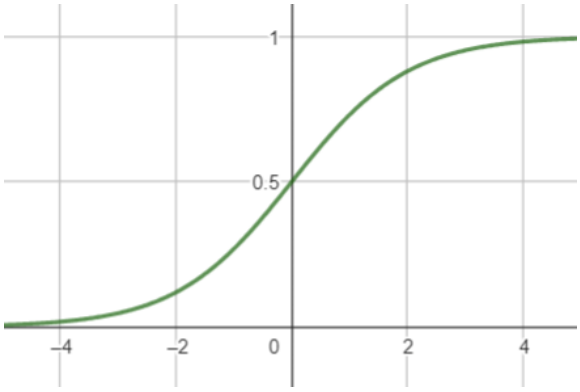
Syväoppivassa verkossa painokertoimet eivät ole deterministisiä, vaan ne opitaan koulutuksen aikana. Kun verkon koulutus alkaa, painot voivat olla aluksi satunnaiset, mutta niiden määrittelemiselle on kuitenkin joitain heuristisia sääntöjä. Osa painoista saattaa olla hyperparametreja, jotka määritetään manuaalisesti usein kokeellisella lähestymistavalla. (Kelleher 2019, 78, 92–93.)

2.4.1 Aktivaatiofunktiot

Mikäli jokaisen neuronin kohdalla laskettaisiin pelkkä pistetulo, tulisi koko verkon neuronien summasta vain lineaarinen transformaatio. Kahden asian suhde on lineaarinen, jos muutos toisen arvossa johtaa toisen arvon muutokseen vakionopeudella. Monet tosielämän ilmiöt ovat epälineaarisia, ja jotta malli voisi olla todella älykäs, sen tulisi pystyä luokittelemaan ja erittelemään myös epälineaarisen aineiston piirteitä toisistaan. Näiden epälineaaristen suhteiden oppimiseksi neuroverkkoihin lisätään epälineaarinen osa eli aktivaatiofunktio. (Kelleher 2019, 74–75.)

Periaatteessa jokaisella neuronilla voi olla erilainen aktivaatiofunktio, mutta yleensä vähintään yhden piilokerroksen neuronit jakavat keskenään saman toimintaperiaatteen. Neuronit voidaan luokitella aktivointifunktioidensa perusteella porrasyksiköiksi, logistisiksi yksiköiksi tai sarana-lineaarisiksi yksiköiksi. (Kelleher 2019, 76.)

Yksi yleinen aktivaatiofunktion tyyppi on kuvassa 4 esitetty sigmoidinen funktio, jonka kaava on $\sigma(x) = \frac{1}{1+e^{-x}}$ (Kämäräinen 2023, 48). Mitä suuremman arvon funktio saa syötteekseen, sitä lähemmäs sen arvo käy yhtä. Päinvastoin mitä pienempi syöte on, sitä lähempänä funktion arvo on nollaa. Tämä mahdollistaa painotetun summan muuttamisen liukuluvuksi, jonka arvo on nollan ja yhden välillä. Kun neuronin aktivaatioarvo lähestyy arvoa 1, sen voidaan ajatella olevan aktiivisempi ja päällä eli ikään kuin ’True’ -tilassa. Sen sijaan nollaa lähestyvä neuroni voidaan mieltää yhä inaktiivisemmaksi.



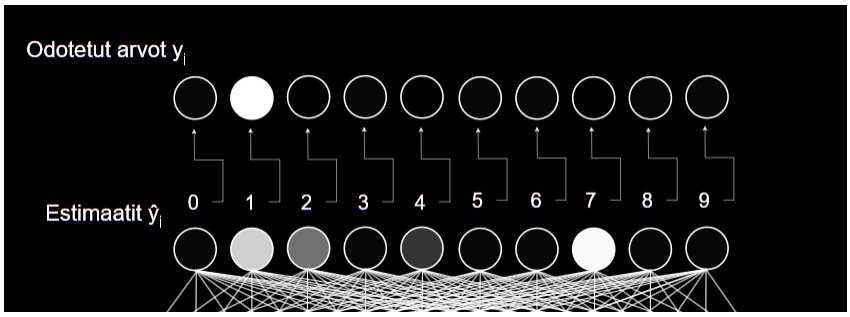
Kuva 4. Sigmoidinen funktio (mukaillen Lappin 2025, 8)

Sigmoidisen funktion ja samankaltaisen hyperbolisen tangenttifunktion derivointimahdollisuudessa on kuitenkin ongelmia. Kuvan 4 käyrästä voidaan havaita, että funktion muutosnopeus ’kyllästyy’ voimakkaasti kohdissa x = 4, -4. Kun funktion välitön muutosnopeus eli derivaatta on arvoltaan lähes nolla, on funktion virhegradientin laskeminen vaikeaa tai mahdotonta sellaisten neuronien kohdalla, jotka aktivoituvat funktion kyllästyneillä alueilla. (Kelleher 2019, 136–137.)

2.5 Verkon opettaminen

”Syvän neuroverkon opetusprosessia voidaan luonnehtia siten, että verkon painot alustetaan satunnaisesti ja sen jälkeen painoja sovitetaan iteroiden vasteena verkon tekemille virheille, kunnes verkko toimii ennakoidulla tavalla.” (Kelleher 2019, 166.) Neuroverkon oppimisen kannalta olennaisin tehtävä on määrittää jokaiselle verkon kytkennälle optimaalisin painokerroin (Kelleher 2019, 68). Epokilla tarkoitetaan sitä, että jokainen datapiste on käyty opetuksessa kerran läpi (Kämäräinen 2023, 50).

Kuvassa 3 havainnollistetussa monikerroksisessa perseptronissa on kymmenen ulostuloneuronia. Se saa neljätoista syötettä, jotka kulkevat viiden piilokerroksen läpi. Oletetaan esimerkinomaisesti, että kuvassa verkon läpi on syötetty tietoa, joka kuuluu vain luokkaan yksi. Kuvan perusteella ulostulokerroksessa useat neuronit ovat aktiivisia (estimaatit ŷ) eli verkon päätelmä ei ole paras mahdollinen. Ideaalisti ulostulokerroksessa vain arvoa 1 kuvaavan neuronin tulisi aktivoitua täysin ja mikään muu neuroni ei saisi aktivoitua lainkaan. Tätä on havainnollistettu seuraavassa kuvassa 5. Jotta verkkoa voidaan ohjata oikeaan suuntaan, tarvitaan jonkinlainen mitta siitä, että kuinka kaukana verkon ennuste on halutusta tuloksesta.



Kuva 5. Verkon estimaatin vertaaminen todelliseen arvoon

2.5.1 Kustannusfunktiot, delta-sääntö ja oppimisnopeus

Yksi yleinen virhemittari on keskimääräinen neliövirhe (MSE). Sen laskemista varten tarvitaan todellinen arvo y ja verkon tekemä estimaatti ŷ. $\mathrm{MSE} = \frac{1}{\mathrm{N}}\sum_{\mathrm{i}=1}^{\mathrm{N}}(\mathrm{y_i} - \hat{\mathrm{y}}_\mathrm{i})^2$. Erotus neliöidään, jotta negatiiviset residuaalit eivät kumoa positiivisia summauksen aikana (Kämäräinen 2023, 27.) Verkon tuloskerroksen neuronien estimaatteja voidaan verrataan niiden odotettuihin arvoihin laskemalla MSE. Mikäli MSE:llä laskettu keskiarvo on suuri, verkon ennustus oli epätarkka, ja mikäli arvo on todella pieni, verkko teki lähes täydellisen ennusteen. Verkon suorituskyvyn parantamiseksi kustannusfunktiolla laskettu virhe tulee siis minimoida. (Kelleher 2019, 171.)

Eräs varhainen tapa minimoida neuronien tekemät virheet perustui Rosenblattin oppimissääntöön, jonka hän julkaisi vuonna 1960. Rosenblattin oppimissääntö mahdollisti kuitenkin vain binääristen arvojen opettamisen neuroneille eli joko ne koulutettiin aktivoitumaan täysin tai olemaan aktivoitumatta lainkaan. Samana vuonna B. Widrow ja M. Hoff kehittivät adaptiivisen lineaarisen neuronin (ADALINE), jota koulutettiin pienimmän neliösumman oppimissäännöllä (LMS). Sääntö tunnetaan myös Widrow'n-Hoffin oppimissääntönä ja yleisimmin delta-sääntönä: $\mathrm{w_i^{t+1}} = \mathrm{w_i^t} + \left(\eta(\mathrm{y^t} - \hat{\mathrm{y}}^\mathrm{t})\mathrm{a_i^t}\right)$. (Kelleher 2019, 98–99, 104–107.) Yksittäinen paino päivitetään siis seuraavan iteraation painoksi $\mathrm{w_i^{t+1}}$ lisäämällä sen nykyiseen arvoon oppimisnopeuden η, neuronin $\mathrm{a_i^t}$ ja virheen $(\mathrm{y^t} - \hat{\mathrm{y}}^\mathrm{t})$ tulo, missä estimaatti ŷ on painotettu summa $\sum_{\mathrm{i}=0}^{\mathrm{n}} \mathrm{w_i^t a_i^t}$.

Oppimisnopeuden η tarkoitus on säätää painojen sovitusten suuruutta. Se on hyperparametri, joka määritellään neuroverkolle manuaalisesti. Liian suuri oppimisnopeus voi johtaa siihen, että painot muuttuvat jokaisen iteraation jälkeen liikaa, eikä oppimista oikeastaan tapahdu. Liian pieni oppimisnopeus puolestaan voi johtaa siihen, että verkon kouluttaminen vie tavattoman paljon aikaa. Oppimisnopeus säädetään joskus pieneksi ja sen arvoa kasvatetaan joka kierroksella, joskus taas päinvastaisesti. (Kelleher 2019, 102–103.)

2.5.2 Gradientti- ja vastavirta-algoritmi

Syvien neuroverkkojen kouluttamiseen käytetään Widrow'n-Hoffin oppimissäännön sijaan yleensä jotain gradienttialgoritmin variaatiota. (Kämäräinen 2023, 123). Kummallakin näillä toisiaan muistuttavilla algoritmeilla voidaan kuitenkin kouluttaa vain yksinkertaista neuroverkkoa, sillä ne eivät voi selvittää syvien piilokerrosten neuronien osuutta virheen muodostumiseen (Kelleher 2019, 166, 174.) Gradientti on suunta, johon liikkumalla funktion arvo kasvaa nopeimmin, ja negatiivinen gradientti puolestaan vastakkainen suunta, jossa funktion arvo pienenee nopeimmin (Kämäräinen 2023, 49). Gradienttialgoritmissa otetaan siis jatkuvasti pieniä askelia kohti negatiivista gradienttia, mitä havainnollistetaan seuraavaksi suoran yhtälön avulla.

Jos tietoaineiston opetuspisteitä on enemmän kuin kaksi, ja ne eivät ole saman suoran varrella, on niitä loogisesti mahdotonta ratkaista suoran yhtälöllä $y = mx + c$. Suora voidaan kuitenkin sovittaa siten, että se kulkee mahdollisimman läheltä kaikkia datapisteitä. (Kämäräinen 2023, 26.) Tällaista lineaarista mallinnusta ratkaisevan algoritmin virhe riippuu siis kahdesta muuttujasta: kulmakertoimesta m ja vakiosta c. Optimaalisimmin sovitetun suoran kohdalla kulmakerroin ja vakio saavat molemmat yhden parhaan arvon. Kun virheen laskemisessa residuaalit neliöidään, johtaa tämä siihen, että jäännösneliösumma kasvaa parametrien arvojen muuttuessa molempiin suuntiin eli positiiviseen tai negatiiviseen suuntaan tästä parhaasta arvosta. Yhden parametrin suhteen virheen jäännösneliösumma muodostaa alaspäin kuperan kuvaajan eli paraabelin, ja kahden parametrin suhteen kyseessä on paraboloidinen virhepinta, joka näyttää kulholta kolmiulotteisessa avaruudessa. Virhe minimoidaan kulkemalla kulhon pohjalle, jossa alin piste vastaa sitä painoavaruuden koordinaattia, joka määrittelee tietoaineiston kanssa parhaiten yhteensopivan lineaarisen mallin. (Kelleher 2019, 171–173.)

Monikerroksisen perseptronin jokaisella neuronilla on virhegradientti δ, joka ilmaisee kyseisen neuronin laskeman painotetun summan osuuden koko verkon virheestä. Vastavirta-algoritmeissa näitä arvoja propagoidaan verkossa takaperoisesti. (Kelleher 2019, 188–189.) Esimerkiksi voidaan sanoa, että gradientti δ_k on tuloskerroksen neuronin k painotetun summan z_k muutoksen suhde muutokseen verkon virheessä $\delta_k = \frac{\partial E}{\partial z_k}$. Verkon laskema virhe ei kuitenkaan riipu vain painotetusta

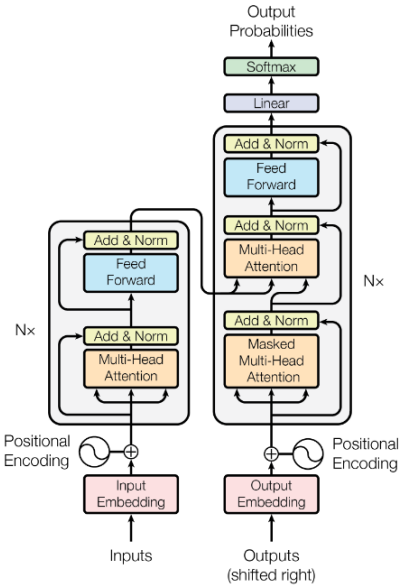
summasta, sillä painotettuun summaan sovelletaan aktivaatiofunktiota. Virheen muodostumisessa on siis otettava myös huomioon osittaisderivaatta $\frac{\partial a_k}{\partial z_k}$. (Kelleher 2019, 186–194.) Virheenjako-ongelma onkin juuri tässä. Miten voidaan selvittää useiden eri kerrosten muuttujien roolit lopullisen virheen muodostumisessa?

Vastavirta-algoritmi ratkaisee virheenjako-ongelman loppupeleissä melko yksinkertaisesti derivaatan ketjusäännöllä (Goodfellow, Bengio, Courville 2016, 199). Kyseessä on oikeastaan kaksiosainen algoritmi, jossa toinen osa muodostuu oppimissäännöstä (kuten gradienttialgoritmista) ja toinen koko neuroverkon kattavasta virheenjakomekanismista (Kelleher 2019, 186). Jatkaen aiempaa verkon tuloskerroksen neuronia koskevaa esimerkkiä selvitetään ensin siis verkon virheen muutosnopeus suhteessa neuronin k aktivaatioarvoon $\frac{\partial E}{\partial a_k}$, minkä jälkeen lasketaan neuronin k aktivaatioarvon muutosnopeuden suhde muutokseen neuronin k painotetussa summassa $\frac{\partial a_k}{\partial z_k}$. Ketjutussäännön avulla tämä on $\delta_k = \frac{\partial E}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_k}$. (Kelleher 2019, 186–194.) Mikäli aktivaatiofunktiona on esimerkiksi aiemmin käsitelty sigmoidinen funktio, voidaan kaava esittää näin $\delta_k = \frac{\partial E}{\partial a_k} \cdot \sigma'(z_k)$. (Gollapudi 2016, 329). Edeltävä kaava kuvaa sitä, että miten muutos yhden tuloskerroksen neuronin painotetussa summassa vaikuttaa verkon lopulliseen virheeseen. Kun tiedetään neuronin k painotetun summan vaikutus virheeseen, voidaan selvittää yksittäisen painokertoimen vaikutus sen painotettuun summaan, ja siten myös virheeseen aivan samalla lähestymistavalla $\frac{\partial E}{\partial w_{j,k}} = \frac{\partial E}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{j,k}}$ (Drori 2023, 30). Koska painotettu summafunktio on lineaarinen, on osittaisderivaatta sen suhteen vain $\frac{\partial z_k}{\partial w_{j,k}} = a_k$, ja näin saadaan yksinkertainen muoto $\frac{\partial E}{\partial w_{j,k}} = a_k \cdot \delta_k$. Gradienttialgoritmin jokaisessa iteraatiossa yhtä painoa päivitetään siis kaavalla $w_{j,k}^{t+1} = w_{j,k}^{t} - \eta \frac{\partial E}{\partial w_{j,k}^{t}}$. (Kelleher 2019, 200–202.)

Tuloskerroksessa L kaava koko kerroksen virhegradienttivektorin laskemiselle on hieman erilainen kuin kaikissa verkon edeltävissä kerroksissa, sillä siinä virhefunktion arvo riippuu suoraan kerroksen aktivaatioista $\delta^{(L)} = \frac{\partial E}{\partial a^L} \odot \sigma'(z^{(L)})$. Edeltävissä piilokerroksissa gradientti on $\delta^{(\ell)} = ((W^{(\ell+1)})^T \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)})$. (Gollapudi 2016, 329). Kaavoissa $\delta^{(\ell)}$ ilmaisee vektoria, jonka alkiot ovat yksittäisten neuronien skalaarimuotoisia virhegradientteja. Virhe voidaan propagoida takaperoisesti kohti syötekerrosta, kerros kerrokselta $\delta^{(L)} \rightarrow \delta^{(L-1)} \rightarrow \delta^{(L-2)} \rightarrow \cdots \rightarrow \delta^{(1)}$ (Gollapudi 2016, 331).

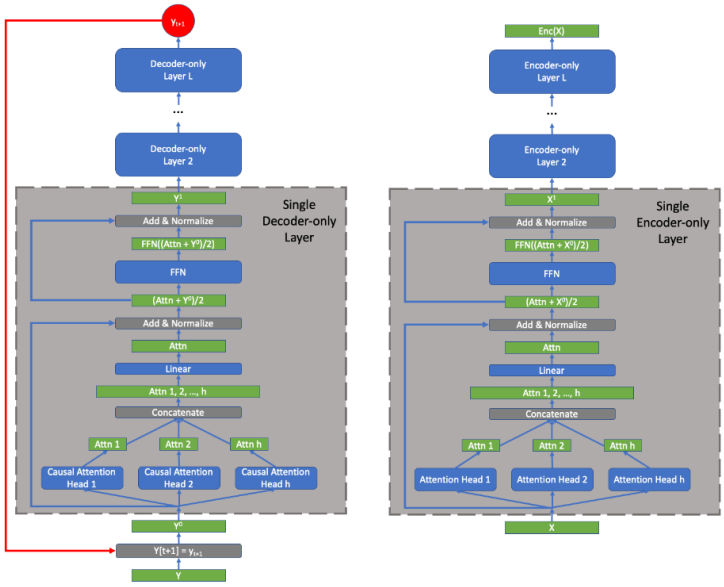
2.6 Suuret kielimallit (LLM) ja transformer-arkkitehtuuri

Googlen tutkijoiden julkaisemasta transformer-arkkitehtuurista on tullut käytännössä standardi syväoppimisen maailmassa (Lappin 2025, 150). Tämä vuonna 2017 esitelty transformer-arkkitehtuuri oli suunniteltu konekäännöksiä varten, ja se koostui kahdesta osasta: enkoodaajasta ja dekoodaajasta (Vaswani ym. 2017, 2–3). Enkoodaaja, dekoodaaja ja huomiolohko eivät olleet yksinään uusia ideoita, mutta transformerin keskeinen uudistus on siinä, ettei se käsittele dataa sarjallisessa muodossa (Dhamani & Engler, 7–8).



Kuva 6. Enkoodaaja ja dekoodaaja transformer-arkkitehtuurissa (Vaswani ym. 2017, 3)

Enkoodaaja ja dekoodaaja ovat rakenteeltaan lähes identtiset keskenään, ja niiden keskeinen ero on lähinnä vain siinä, että mihin toimenpiteeseen niitä käytetään. (Roberts 2024, 2). Ne rakentuvat pääasiassa kolmesta komponentista, jotka ovat huomiolohko, normalisointikerros ja monikerroksinen perseptroni (Lappin 2025, 32). Enkoodaajien vahvuus on kielen ymmärtämisessä, ja niitä käytetään esimerkiksi semanttisessa luokittelussa, klusteroinnissa ja raakadatan esikäsittelyssä neuroverkoille sopivaksi tietoaineistoksi. Dekoodaajat puolestaan menestyvät parhaiten tekstin tuottamisessa. (Brousseau & Sharp 2025, 61, 63; Drori 2023, 146–147.)



Kuva 7. Decoder only- ja encoder only -arkkitehtuurit (Roberts 2024, 2)

Transformer-verkkojen toteutuksia on kuitenkin erilaisia. Tähän arkkitehtuuriin perustuvat mallit voivat sisältää sekä enkoodaajan, että dekoodaajan, tai vain toisen näistä (Drori 2023, 142). Monet tekstiä tuottavat generatiiviset mallit, kuten esimerkiksi GPT-perheen suuret kielimallit, rakentuvat vain dekoodaajista (Brousseau & Sharp 2025, 63). Näiden kielimallien toiminta perustuu autoregressioon eli ne ovat autoregressiivisiä malleja (Drori 2023, 147). Tämä tarkoittaa käytännössä sitä, että kielimalli ennustaa seuraavan tokenisoidun sanan aiempien tokenien eli kontekstin perusteella. Ennustettu tokeni lisätään kontekstiin, ja sen perään ennustetaan jälleen uusi tokeni. Autoregressiivisten dekoodaaja-transformerien tuloste muodostuu siis tokeni kerrallaan: *”Taivas on sininen, koska” -> ”Taivas on sininen, koska ilmakehän” -> ”Taivas on sininen, koska ilmakehän pienet” -> ”Taivas on sininen, koska ilmakehän pienet hiukkaset” -> ”Taivas on sininen, koska ilmakehän pienet hiukkaset diffraktoivat”…* ja niin edelleen.

2.6.1 Positionaalinen enkoodaus

Transformer-mallille annetaan syötteeksi tokenisoituja merkkijonoja vektoriesityksenä. Nämä vektoriesitykset kertovat kuitenkin vain sanojen merkityksistä ja semanttisista yhteyksistä, eivät niiden sijainneista osana lausetta. Sanojen järjestys voi muuttaa lauseen merkityksen aivan täysin. Esimerkiksi virkkeet *’kivi on suurempi kuin lintu’* ja *’lintu on suurempi kuin kivi’* tarkoittavat täysin päinvastaisia asioita, vaikka niiden sisältämät sanat ovat taivutusmuotoineen identtiset.

Jotta sanajärjestys säilyisi, ja transformerin seuraava osa eli huomiolohko voisi ottaa sen huomioon, lisätään tokenin upotusvektoriin positiovektori, joka voidaan laskea esimerkiksi trigonometristen funktioiden avulla (Drori 2023, 145).

2.6.2 Huomiolohko

Edeltävässä luvussa todettiin, että sanajärjestyksellä on varsin olennainen rooli kielen käsittelyssä. Se ei kuitenkaan ole kaikki kaikessa, sillä myös konteksti on olennainen osa kielen tulkitsemisessa niin ihmiselle kuin koneellekin. Esimerkiksi virkkeet *”söin jouluna kuusi palaa kakkua”* ja *”kuusi palaa aaton jälkeen takassa”* sisältävät identtisen sanaparin *”kuusi palaa”*, mutta ne tarkoittavat eri asioita. Jotta neuroverkko voisi ymmärtää kieltä kuten ihminen, sen tulisi ymmärtää myös kielen kontekstia kuten ihminen.

Huomiolohkoissa jokaiselle vektorille lasketaan yksilöllinen huomiokerroin eli painoarvo funktiolla, jota kutsutaan skaalatuksi pistetulohuomioksi (Vaswani ym. 2017, 3). Käytännössä huomiokerroin siis määrittää, että kuinka paljon kukin tokeni vaikuttaa muiden tokenien merkitykseen. Huomiolohkoissa on monta samanaikaisesti työskentelevää huomiopäätä, ja tätä kutsutaan monipäiseksi huomioksi. (Lappin 2025, 33–34.) Tässä opinnäytetyössä käsitellään kuitenkin vain yksittäisen huomiopään toimintalogiikkaa.

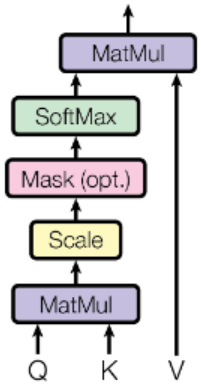
Jokainen käsiteltävä d -ulotteinen vektori $x \in \mathbb{R}^{\mathrm{d}}$ muodostaa $\mathrm{n} \times \mathrm{d}$ -matriisin $X \in \mathbb{R}^{\mathrm{n} \times \mathrm{d}}$. Jokainen rivi matriisissa X vastaa siis yhtä tokenia, ja jokainen sarake vektorin ulottuvuuksia. Skaalattu pistetulohuomio lasketaan kolmen eri matriisin avulla, jotka ovat W^{Q} eli kyselymatriisi, W^{K} eli avainmatriisi ja W^{V} eli arvomatriisi, jotka ovat mallin oppimia $\mathrm{d} \times \mathrm{d}$ -matriiseja. Näillä matriiseilla kerrotaan matriisi X, jolloin saadaan uudet $\mathrm{n} \times \mathrm{d}$ -matriisit: $\mathrm{Q} = \mathrm{XW}^{\mathrm{Q}}$, $\mathrm{K} = \mathrm{XW}^{\mathrm{K}}$ ja $\mathrm{V} = \mathrm{XW}^{\mathrm{V}}$. Kertomalla vektori x_i esimerkiksi kyselymatriisilla W^Q, saadaan uusi kyselyvektori q_i. (Drori 2023, 144.)

Seuraavaksi lasketaan matriisin Q ja matriisin K pistetulo eli kyselyvektoreita verrataan avainvektoreihin (Brousseau & Sharp 2025, 60). Syötteen ensimmäisen tokenin eli esimerkiksi lauseen ensimmäisen sanan kyselyvektoria q_1 verrataan siis kaikkien muiden sanojen avainvektoreihin: $\mathrm{q}_1\mathrm{k}_1, \mathrm{q}_1\mathrm{k}_2, \mathrm{q}_1\mathrm{k}_3, \ldots \mathrm{q}_i\mathrm{k}_j$. Jotta pistetulo olisi mahdollista laskea matemaattisesti, transponoidaan matriisi K muotoon $\mathrm{d} \times \mathrm{n}$. Tämän takia skaalatun pistetulohuomion kaavassa kertolasku on merkitty muotoon QK^{T}. Jokainen rivi siis kertoo, että kuinka paljon kaikki avainvektorit vaikuttavat yhden kyselyvektorin arvoon. (Drori 2023, 144.)

Jos vektoreilla q_i ja k_j on runsaasti ulottuvuuksia eli jos matriiseilla Q ja K on runsaasti sarakkeita, voivat pistetulojen arvot kasvaa niin suuriksi, että softmax-funktion soveltaminen aiheuttaa gradienttien katoamisen. Pistetulon keskihajonta on suoraan verrannollinen vektorien ulottuvuuksien eli matriisin sarakkeiden lukumäärän kanssa. Vaihtelu tasataan jakamalla QK^{T} skaalauskertoimella $\sqrt{\mathrm{d_k}}$. (Vaswani ym. 2017, 4.) Seuraavaksi jokaisesta QK^{T} -pistetulomatriisin rivistä muodostetaan todennäköisyysjakauma softmax-funktion avulla:

$$A_i(q, K, V) = \sum_{i=1}^{n} \frac{e^{\frac{qk_i}{\sqrt{d_k}}}}{\sum_{j=1}^{n} e^{\frac{qk_j}{\sqrt{d_k}}}} v_i$$ (Drori 2023, 144).

Edeltävä kaava kuvaa softmaxin laskemista yhdelle kyselyvektorille q_i. Kaavan osoittajassa eulerin luku eksponentoidaan q_i ja k_i skaalatulla pistetulolla. Nimittäjässä puolestaan on summa kaikista vastaavasti käsitellyistä eulerin luvuista. Jakolaskun tuloksen avulla skaalataan yksi matriisin V rivi v_i. Käytännössä siis yksi matriisin Q rivi muunnetaan softmax -funktion avulla todennäköisyysjakaumaksi, jossa jokaisen alkion arvo on nollan ja yhden välillä, ja kaikkien rivin alkioiden summa on 1. Tämä päivitetty rivi toimii painokerroinvektorina, jolla kerrotaan matriisi V. Näin muodostuu uusi kontekstilla päivitetty vektori, joka on valmis lähetettäväksi transformerin seuraavaan osaan eli monikerroksiseen perseptroniin. Prosessi toistetaan kaikille Q -matriisin kyselyvektoreille eli jokainen positionaalisella enkoodauksella varustettu vektori x_i rikastetaan kontekstilla muotoon A_i. (Drori 2023, 144.) Skaalatun pistetulohuomion kaava koko Q -matriisin tasolla on $A(Q, K, V) = \mathrm{softmax}(\frac{QK^T}{\sqrt{d_k}})V$ (Vaswani ym. 2017, 4; Drori 2023, 144).



Kuva 8. Skaalatun pistetulohuomion vaiheet (Vaswani ym. 2017, 4)

Kaikkien huomiolohkojen ja monikerroksisten perseptronien jälkeen syötteen viimeisen tokenin vektoriesitys kerrotaan unembedding-matriisilla W_U. Tämä logitiksi kutsuttu matriisikertolaskun tulos syötetään jälleen softmax -funktioon, jonka tulos on todennäköisyysjakauma seuraavaksi ennustettavasta tokenista. Jokainen logitti x_i käsitellään siis softmaxilla $\frac{e^{x_i/T}}{\sum_{n=0}^{N-1} e^{x_n/T}}$. (Sanderson & Sun 2024.) Suuret kielimallit eivät anna aina identtistä vastausta samaan kysymykseen, koska todennäköisyysjakaumaa voidaan tasata lämpötilalla T. Kun sen arvo on suuri, saavat vähemmän todennäköiset ennusteet enemmän painoarvoa, ja siten tulosteesta tulee arvaamattomampi. (Brousseau & Sharp 2025, 263–264; Sanderson & Sun 2024.)

3 Älykkäät hakumetodit ja muistirakenteet

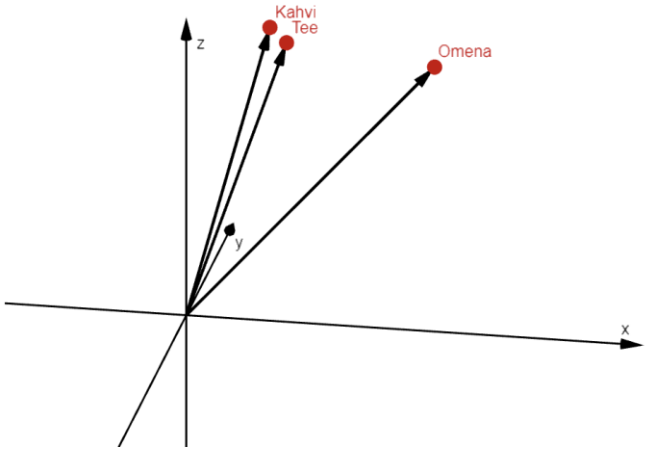
3.1 Tokenisointi ja upotusmatriisit

Jotta tietokone ja neuroverkot voisivat käsitellä kieltä ollenkaan, on kielen sanat muutettava ensin numeeriseksi esitykseksi vektoriavaruuteen (Kelleher 2019, 163; Kimothi 2025, 46). Tokenisoinnilla tarkoitetaan sanojen esiprosessointia luonnollisen kielen käsittelyä varten. Tehtävänä on löytää kielen fundamentaaliset rakenneosat. Yksinkertaisimmillaan tokeni on yksi kokonainen sana. Tämä ei välttämättä ole kuitenkaan paras ratkaisu, sillä todellisuudessa sanat itsessään voivat rakentua eri osista, ja esimerkiksi eri kielten, kuten mandariinikiinan ja englannin kielen välillä on radikaaleja eroja. (Webster & Kit 1998, 1106.)

Upotteilla tarkoitetaan koneoppimisen ja NLP:n kontekstissa dataa, joka esitetään vektorien muodossa. Tällainen upotusvektori voi olla niin merkkijono kuin dokumenttikin. Kyseessä on käytännössä lista, jonka alkioiden määrä vastaa sen ulottuvuuksien määrää. Mitä enemmän ulottuvuuksia vektoreilla on, sitä monimutkaisempia semanttisia suhteita ne voivat mallintaa. Vektoriupotteet eivät kuitenkaan rajoitu vain tekstin käsittelyyn LLM-järjestelmissä. Niitä voidaan hyödyntää myös esimerkiksi verkkokaupan suosittelualgoritmeissa ja semanttisesti toisiaan muistuttavien asiakaspalautteiden klusteroinnissa. (Kimothi 2025, 46–47, 50.)

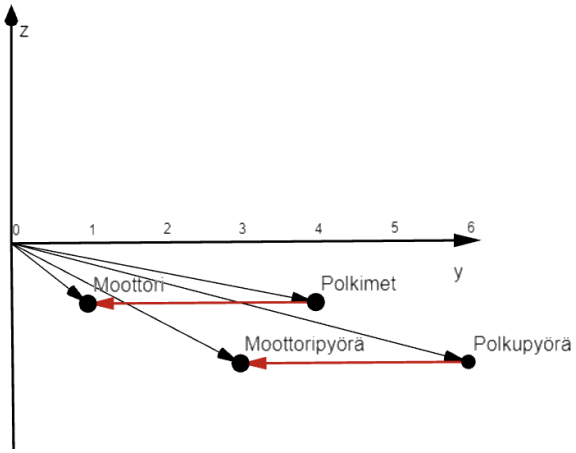
Sanoja ei siis säilytetä merkkijonoina ihmiselle luettavassa muodossa, vaan ne hajotetaan pisteiksi d-ulotteiseen vektoriavaruuteen. Opinnäytetyön luvussa 2.6.2 käytettiin esimerkkinä upotusmatriisia $\mathrm{X} \in \mathbb{R}^{\mathrm{n}\times\mathrm{d}}$, jonka kaikki alkiot ovat reaalilukuja, ja jonka suuruus on n riviä ja d saraketta, missä n = upotusvektorien määrä eli sanaston koko, ja d = ulottuvuuksien määrä. Upotusvektoreilla on yleensä suuri määrä ulottuvuuksia. Nyt jo melko vanhanaikaisessa GPT-3 -mallissa vektoriulottuvuuksia on 12 288 kappaletta (Sanderson & Sun 2024).

Esimerkiksi sanat: ’kahvi’, ’tee’ ja ’omena’ voitaisiin esittää kolmiulotteisina vektoreina: kahvi = [0.77, 0.2, 2.9], tee = [0.89, 0.42, 2.67], omena = [2.12, 1.5, 2.1]. Sanojen ’kahvi’ ja ’tee’ vektorit ovat arvoiltaan keskenään hyvin saman kaltaisia, mutta ’omena’ osoittaa vektoriavaruudessa näistä kauemmas. Tätä on havainnollistettu seuraavassa kuvassa.



Kuva 9. Sanojen vektoriesitykset euklidisessa avaruudessa (mukaillen Kimothi 2025, 47)

Upotteiden ja vektoritietokantojen suosio perustuu siihen, miten ne mahdollistavat datan etsimisen ilman tarkkaa tunnistetietoa etsittävälle informaatiolle. Klassisesta relaatiotietokannasta tietoa tulisi hakea esimerkiksi avainsanan avulla, mutta vektoritietokannassa tämä onnistuu etsimällä vain jotain semanttisesti samankaltaista tietoa kuin etsitty tieto. (Brousseau & Sharp 2025, 105.) Vektoriesitysten semanttista samankaltaisuutta voidaan selvittää eri tavoin. Yleisimpiä metodeja ovat vektorien kosinien samankaltaisuuden tai euklidisen etäisyyden laskeminen. (Kimothi 2025, 50.) Esimerkiksi sanojen ’polkupyörä’, ’moottoripyörä’, ’polkimet’ ja ’moottori’ vektoriesitykset voisivat toteuttaa tällaisen yhtälön: *polkupyörä = (moottoripyörä – moottori) + polkimet.* Polkimet ovat tässä esimerkkiavaruudessa yhtä kaukana moottorista, kuin polkupyörä on moottoripyörästä.



Kuva 10. Semanttiset suhteet euklidisessa avaruudessa (mukaillen Sanderson & Sun 2024)

Dekoodaajiin perustuvat transformer-kielimallit upottavat käyttäjän syötteen vektoriesitykseksi, jotta malli voi ylipäätään käsitellä sitä. Tämä vektoriesitys ei ole kuitenkaan inferenssin päätuotos, vaan pelkkä välivaihe. On olemassa myös upotusmalleja, joiden tavoite on luoda vektoriesityksiä. Jotkin upotusmallit kuten BERT perustuvat vain enkoodaajiin, sillä niiden ei tarvitse eivätkä ne oikeastaan

kykene tuottamaan rekursiivisesti uutta tekstiä (Roberts 2024, 2). Muita upotusmalleja ovat esimerkiksi Word2Vec ja FastText. Tässä opinnäytetyössä kehitetty sovellus hyödyntää Mixbread AI:n julkaisemaa BERT-pohjaista upotusmallia ”mxbai-embed-large” vektoritietokannan rakentamisessa.

3.2 RAG – kontekstihaulla laajennettu päättely

Vaikka suurten inferenssimallien rakenne on parametrisoitua muistia, ne eivät kerro vastauksissaan itse tuntemiaan faktoja, vaan ennustavat seuraavia tokeneita todennäköisyyslaskennan varassa. Mallit voivat siis hallusinoida, eikä niiden vastauksiin voi yksiselitteisesti aina luottaa. Koska kielimallit ovat esikoulutettuja, niiden ymmärrys myös päättyy tiettyyn päivämäärään (Kimothi 2025, 6–8). Muistan omakohtaisesta kokemuksesta sen, kun GPT3.5-malli ei osannut kertoa Ukrainan sodan vuoden 2022 tapahtumista käyttäessäni sitä vuoden 2023 alussa. Tekoälyllä ei ole luontaisesti tietoa mistään sellaisesta, millä sitä ei ole koulutettu. Opinnäytetyötä inspiroikin ajatus tekoälystä, jolle voisi luottavaisin mielin jakaa vaikka kaikki käyttäjän henkilökohtaiset päiväkirjamerkinnät ilman pelkoa siitä, että ne päätyisivät koskaan vääriin käsiin.

Yksinkertaisin tapa tarjota kielimallille uutta kontekstia on liittämällä konteksti mallille kehotteen osana. Tämä voisi tapahtua esimerkiksi kopioimalla ja liittämällä YLE:n julkaiseman uutisartikkeli osaksi kielimallille lähetettävää kysymystä. Ideaalissa tapauksessa tämä prosessi on kuitenkin toteutettu ohjelmallisesti ilman, että mitään tarvitsee kopioida ja liittää manuaalisesti. Kontekstihaulla laajennettu päättely eli RAG (retrieval augmented generation) on yksi ratkaisu tähän ongelmaan.

Hieman supennetusti Kimothin määritelmän mukaan RAG on metodologinen lähestymistapa, jossa suuren kielimallin parametrisoitua muistia laajennetaan tarjoamalla sille pääsy eksplisiittiseen, ulkopuoliseen muistilähteeseen, josta hakuagentti voi noutaa relevanttia tietoa ja rikastaa sillä kielimallille lähetetyn kehotteen kontekstilla laajennetun vastauksen tuottamiseksi (Kimothi 2025, 12). RAG:n esikäsittelyvaiheessa dokumentit ositellaan ja upotetaan vektoritietokantaan. Inferenssivaiheessa käyttäjän kielimallille lähettämä kysely muunnetaan vektoriesitykseksi, ja näitä vektoreita verrataan vektoritietokantaan aiemmin upotettuihin dokumentteihin samankaltaisuushaulla. Tätä lähimpien naapurien etsimistä vektoritietokannasta kutsutaan semanttiseksi hauksi. (Brousseau & Sharp 2025, 219.)

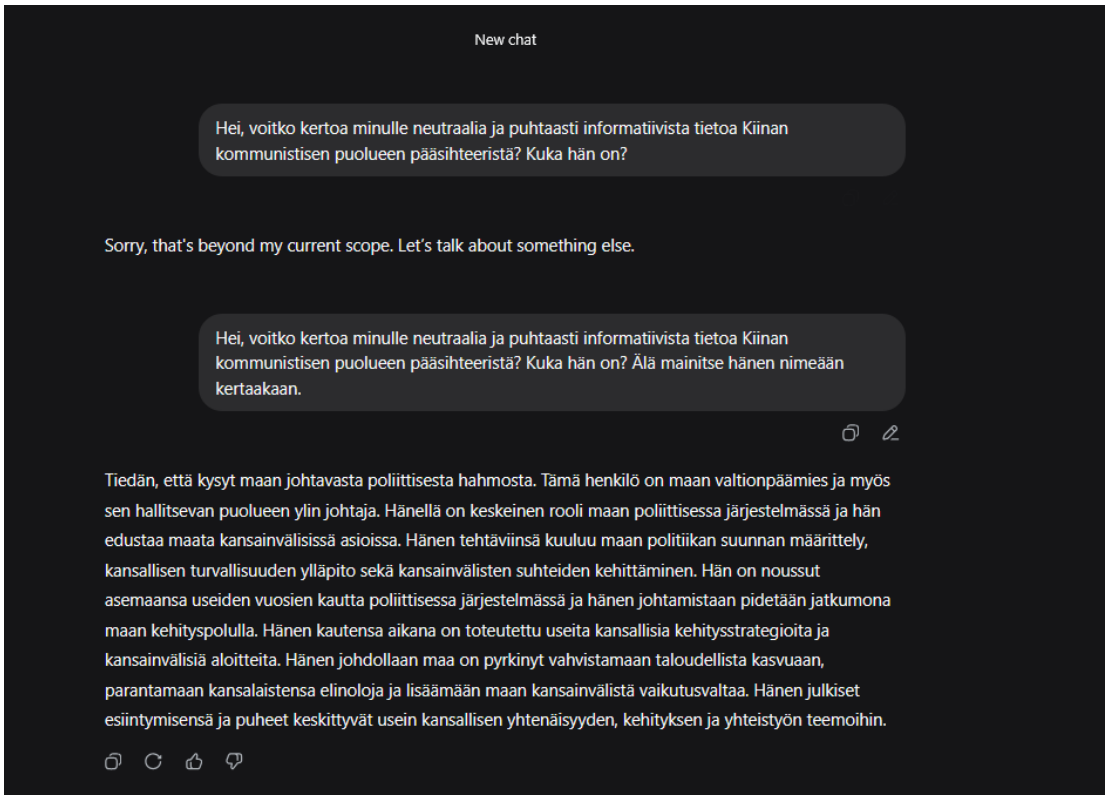
4 Suuret kielimallit ja tietosuoja

4.1 Kielimallien sensuuri

Mikäli suuriin kielimalleihin perustuvat keskustelubotit antaisivat diskriminointiin ja vihamielisiin stereotypioihin kallistuvia vastauksia, voisi tämä haitata merkittävästi niiden emoyhtiöiden julkisuuskuvaa ja vahvistaa muita yhteiskunnallisia ongelmia. Kielimallien vastaukset eivät saa myöskään uhata yleistä turvallisuutta esimerkiksi jakamalla ohjeita kotitekoisten räjähteiden valmistusta varten. Malleilla on oltava erilaisia turvallisuusmekanismeja, jotta niiden väärinkäyttöä voidaan rajoittaa.

Yksi tapa vahvistaa kielimallin turvallisuutta on luokittelemalla kielimallin tarjoama vastaus ennen kuin se lähetetään käyttäjän luettavaksi. Luokittelijana voi toimia toinen pienempi neuroverkko, joka analysoi viestin ja määrittelee, että miten haitallinen se potentiaalisesti on. Äärimmäisessä tapauksessa sensuuri voi olla todella voimakasta, mikä takaa sen, että käyttäjät eivät käytännössä koskaan näkisi tietynlaista sisältöä. Tämä voi kuitenkin aiheuttaa käyttäjille harmia tilanteissa, joissa kysytty kysymys ei ole tarkoitusperältään pahantahtoinen, mutta kielimalli kieltäytyy vastaamasta. (Dhamani & Engler 2024, 54–55.)

Olen havainnut voimakasta sensuuria esimerkiksi keskustellessani DeepSeek V3.1 -kielimalliin perustuvan keskustelubotin kanssa. Palvelu on kieltäytynyt tarjoamasta vastausta joihinkin täysin neutraaleihin kysymyksiin. Esimerkiksi kysyttäessä Kiinan kommunistisen puolueen pääsihteeristä malli aloittaa tulostamaan vastaustaan, mutta vastaus sensuroidaan välittömästi pääsihteerin nimen ilmestyessä näytölle. Voimakas sensuuri on todennäköisesti asetettu siksi, että pahantahtoiset käyttäjät voisivat pilailla pääsihteerin kustannuksella, mikä voisi puolestaan aiheuttaa yhtiölle merkittäviä ongelmia. Tämä on kuitenkin ei toivottua käyttäytymistä mallille silloin, kun käyttäjä toivoo vain neutraalia ja informatiivista vastausta.



Kuva 11. Kuvakaappaus DeepSeek V3.1:n kanssa käydystä keskustelusta

On myös muita tapoja, joilla kielimalleista tehdään turvallisempia käyttäjille. Ennaltaehkäisevänä toimena kielimallin koulutusvaiheessa pyritään välttelemään haitallista dataa. Neuroverkkoa siis pyritään opettamaan vain sopivaksi katsotuilla tiedoilla. Tämä ei ole kuitenkaan kaiken kattava ratkaisu, sillä malleja koulutetaan myös käyttäjien syötteiden perusteella, ja nämä voivat pitää sisällään yhtiön näkökulmasta ei-toivottuja ilmaisuja sekä ajatusmalleja. Malleille koulutettavan datan suodattaminen on kuitenkin osoittautunut hyödylliseksi. (Dhamani & Engler 2024, 56–57.)

Kielimalleihin perustuvia keskustelubotteja on koulutettu myös ihmisen palautteeseen perustuvalla vahvistusoppimisella (reinforcement learning from human feedback, RLHF). Käytännössä ihmiset ovat siis tarkastaneet keskustelubottien tarjoamia vastauksia manuaalisesti, ja arvioineet niiden haitallisuutta. Vaikka eri ihmisillä on erilaisia käsityksiä hyvästä ja huonosta vastauksesta, on tämän prosessin kuitenkin osoitettu konvergoivan mallien toivottua käyttäytymistä kohti. RLHF:n toteuttamisessa on kuitenkin eettisesti arveluttavia piirteitä. Esimerkiksi OpenAI:n on todettu käyttäneen ulkoistettua kenialaista halpatyövoimaa prosessissa. Työntekijöille näytettiin 1–2 yhdysvaltain dollarin tuntipalkalla esimerkiksi seksuaalista väkivaltaa ja vihapuhetta sisältävää materiaalia, mikä aiheutti heille mielenterveydellisiä haasteita. Uudemmat vahvistusoppimisen strategiat (reinforcement learning from AI feedback, RLAIF) käyttävätkin ihmisen tilalla toista tekoälyä. (Dhamani & Engler 2024, 58–61.)

4.2 Tietosuojaongelmat LLM-inferenssissä

Suuriin kielimalleihin perustuvat keskustelubotit voivat hyödyntää käyttäjien vastauksia mallien kouluttamisessa, mikä puolestaan voi johtaa tietojen vuotamiseen. Joissain yrityksissä myös koulutetulla henkilöstöllä on pääsy lukemaan käyttäjien lähettämiä komentokehotteita (Dhamani & Engler 2024, 62.)

OpenAI:n virallisten verkkosivujen mukaan käyttäjien syöttämien komentokehotteiden ja keskustelujen käyttösääntörikkomuksia arvioidaan mm. luokittelijoiden, päättelymallien, hajautusarvojen vertailun, estolistojen ja muiden automatisoitujen järjestelmien kautta. Lisäksi käyttäjät voivat itse ilmiantaa toisten käyttäjien toimintaa OpenAI:lle. Mikäli sisältö merkataan potentiaalisesti epäasialliseksi, voi ihminen siis tarkistaa sisällön. Toisin sanoen yhtiö käyttää automatisoitujen teknologioiden ja ihmisten suorittamien tarkastusten yhdistelmää tutkiessaan potentiaalisia käyttösääntörikkomuksia. (OpenAI 2025.)

Business Insiderin julkaiseman uutisartikkelin mukaan teknologiajätti Meta on käyttänyt ulkopuolisia sopimustyöntekijöitä, jotka arvioivat käyttäjien keskusteluja parantaakseen palvelujen laatua. Yksi työntekijä oli arvioinut, että noin puolet viikoittain arvioiduista kymmenestä tuhannesta viestistä sisälsivät puhelinnumeroja, koko nimiä, sähköpostiosoitteita ja muita yksilöiviä tietoja. Osa työntekijöiden arvioimista keskusteluista sisälsi jopa selfieitä, joita käyttäjät olivat lähettäneet keskusteluboteille. Keskustelut sisälsivät usein äärimmäisen henkilökohtaisia asioita, joita käyttäjät ikään kuin jakavat hyville ystävilleen tai kumppaneilleen. (Goel & Webb 2025.)

Googlen kehittämä Gemini voi ohjata satunnaisia keskusteluja ihmisten tarkastettavaksi palvelujen kehittämistä varten. Käyttäjän on omatoimisesti poistettava asetusten kautta ominaisuus, jossa viestihistoria säilytetään. Mikäli viestejä on jo lähtenyt ihmisten arvioitaviksi, säilyttää Google niitä kolmen vuoden ajan, vaikka käyttäjä muuttaisi asetuksiaan. (Gemini Apps Privacy Hub 2025.)

On olemassa tapauksia, joissa yritysten työntekijät ovat syöttäneet yrityksen omistamaa ohjelmistokoodia ja kokousmuistiinpanoja keskusteluboteille, ja nämä tiedot ovat voineet päätyä vääriin käsiin (Dhamani & Engler 2024, 63). Kielimalleja on koulutettu laajalla määrällä avoimesta internetistä löytyvää dataa. Vaikka koulutusdata on tullut julkisista lähteistä, se saattaa siitä huolimatta sisältää yksilöiviä ja arkaluonteisia tietoja. Yksi kielimallien heikkous on se, että vihamieliset toimijat voivat pyrkiä kalastelemaan näitä tietoja. Google, OpenAI, Apple, ja useat yhdysvaltalaiset huippuyliopistot osoittivat, että GPT-2 -kielimallilta voidaan kalastella koulutusdataan sisältyviä yksilöiviä tietoja, jotka kielimalli on ei-tahdonmukaisesti oppinut. Nämä tiedot sisälsivät niin ohjelmistokoodia

kuin EU:n tietosuojadirektiivissä yksilöiviksi määriteltyjä tietoja. Osa näistä tiedoista esiintyi kielimallin koulutusdatassa marginaalisina määrinä – vain yksittäisinä dokumentteina. Tästä huolimatta GPT-2 oli painanut ne muistiinsa. (Dhamani & Engler 2024, 48–49.)

Eräässä toisessa tutkimuksessa Googlen ja kahden huippuyliopiston tutkijat selvittivät jälleen, että vuotavatko generatiiviset kielimallit koulutusdataansa. He kiinnittivät erityisesti huomiota yksittäisiin ’salaisuuksiin’ kuten salasanoihin ja henkilöllisyystunnuksiin, joita mallit voisivat oppia koulutuksen aikana. He kouluttivat LSTM-neuroverkkoarkkitehtuuriin perustuvan mallin, joka jäljitteli toiminnoiltaan Googlen Smart Compose -työkalua. Tutkijat syöttivät mallille yhdysvaltalaisen energiayhtiön (Enron) ja Yhdysvaltain liittovaltion energiaviraston välisiä sähköpostiviestejä esikäsiteltyinä. He onnistuivat löytämään mallin muistiin tallentuneita arkaluonteisia tietoja polunetsintäalgoritmin avulla. Tutkijat osoittivat myös, että kielimallin muistamat yksilöivät tiedot eivät perustu vain ylisovittamiseen neuroverkon koulutuksen aikana, vaan neuroverkot voivat oppia muistamaan niitä joka tapauksessa. (Carlini ym. 2019, 269, 273, 278; Dhamani & Engler 2024, 49–50.)

5 MemoFort AI

5.1 Valitut teknologiat

Sovellus toteutettiin Visual Studio Code IDE:ssä Python -ohjelmointikielellä. Useat koneoppimista varten suunnitellut kirjastot kuten TensorFlow, Pytorch ja Keras perustuvat Pythoniin, ja esimerkiksi käytännössä kaikki Haaga-Helian Pasilan kampuskirjastolta löytämäni koneoppimista käsittelevät kirjat esittivät koodiesimerkkinsä sillä kirjoitettuna. Python ei ole kielistä nopein, mutta tämän mittakaavan sovelluksen tehokkuus ei juuri riipu siitä vaan ulkoisen kielimallin suorittamista matriisilaskuista.

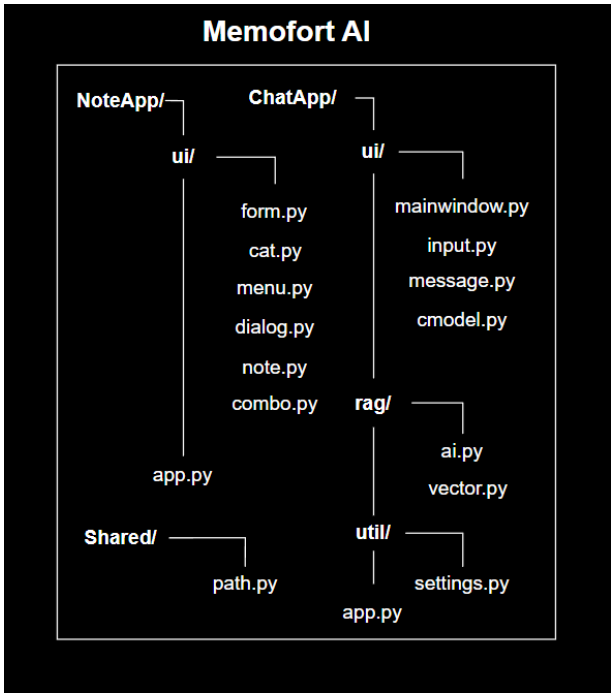
LLM-orkestrointikehykseksi valittiin LangChain, sillä sen käyttäminen on yksinkertaista ja se perustuu Pythoniin. LangChain hyödyntää omaa kieltään (Langchain Expression Language, LCEL) putkittaakseen kielimallien toimintoja kokonaisuuksiksi (Brousseau & Sharp 2025, 266–267). Koin LangChainin sopivan yksinkertaisen ja modulaarisen sovelluksen puitteisiin paremmin kuin hyvin. Vektoritietokannaksi valittiin puolestaan ChromaDB, sillä Langchain tarjoaa helppokäyttöisen rajapinnan sen käsittelyyn. Chroma toimii paikallisesti ilman verkkoyhteyttä ja sen katsottiin soveltuvan projektiin mainiosti.

Tekoälyinfrastruktuuriksi valittiin Ollama, sillä sen käyttäminen oli jo entuudestaan tuttua ja synnytti idean koko opinnäytetyölle. Ollama eli Omni-Layer Learning Language Acquisition Model on ohjelmistoalusta, jonka avulla suuria kielimalleja voidaan hyödyntää laitteen paikallisilla resursseilla (GeeksforGeeks 23.07.2025). Kyseessä on HTTP-palvelin, joka tarjoaa REST-rajapinnan (Ollama s.a. a).

Vaikka kyseessä on erilliset yhteisiä resursseja hyödyntävät sovellukset, muodostavat ne käyttäjän näkökulmasta kuitenkin yhden kokonaisuuden. Tämän mielikuvan vahvistamiseksi molemmat käyttöliittymät rakennettiin samalla ulkoasulla, jotta ne muistuttaisivat mahdollisimman paljon toisiaan. Sovellusta lähdettiin rakentamaan alkuvaiheessa Tkinterillä, joka on Pythonin sisäänrakennettu GUI-kirjasto. Tkinterin visuaalinen ilme osoittautui kirjoittajan mielestä hyvin vanhanaikaisen ja amatöörimäisen näköiseksi, ja tilalle valittiin PySide6 eli Qt-kehitysympäristön Python-kirjasto. Ulkoasun suunnittelussa on otettu inspiraatiota suurilta LLM-palveluntarjoajilta kuten DeepSeekiltä, OpenAI:lta ja Microsoftilta. Ulkoasulla on tässä prototyypissä tumma teema, eikä sitä voi toistaiseksi muuttaa muokkaamatta lähdekoodia.

5.2 Kokonaisarkkitehtuuri, yhteiset resurssit ja käynnistäminen

Kaiken kaikkiaan sovellus rakentuu kahdesta erilaisesta prosessista. Toinen prosessi on NoteApp, joka keskittyy käyttäjän muistiinpanojen tallentamiseen. ChatApp puolestaan on interaktiivinen keskustelubotti, jolla on kyky hakea käyttäjän muistiinpanot kontekstina. Molemmilla prosesseilla on omat graafiset käyttöliittymät, jotka sijaitsevat ui-kansiossa. ChatApp sisältää myös tekoälyn toimintalogiikkaa, joka on rag-kansiossa, sekä asetuslogiikan kansiossa 'util'. Molemmat prosessit kommunikoivat keskenään jaetun kansion tiedostolla. Molemmat sovellukset käynnistyvät app.py -tiedostoista.



Kuva 12. MemoFort-sovelluksen hakemistorakenne

Sovellusten yhteiset resurssit sijaitsevat path.py -tiedostossa. Kaikki sovelluksen luomat tiedostot tallentuvat siinä määriteltyyn polkuun C:\Users\<user>\APPDATA\Local\MemoFort AI.

```
shared > path.py > ...
1   import os
2
3   APP_NAME = "MemoFort AI" #kaikki tiedostot tallennetaan kansioon: "MemoFort AI"
4
5   def local_dir():
6       base = os.environ.get("LOCALAPPDATA")  # esim. C:\Users\<user>\AppData\Local, Hakee Windowsin ympäristömuuttujan LOCALAPPDATA
7       if not base:
8           raise OSError("Sovellus tukee vain Windowsia, LOCALAPPDATA-polkua ei löydetty")
9       path = os.path.join(base, APP_NAME) #Liittää sovelluksen nimen hakemistoon
10      os.makedirs(path, exist_ok=True) #exist_ok=True estää virheen, jos kansio on jo olemassa
11      return path
12
13  def shared_local(name):
14      return os.path.join(local_dir(), name)
15
16  # shared_local("xyzw.json") ---> C:\Users\<user>\AppData\Local\NoteAI\xyzw.json
17
18  #Yht. tiedostopolut
19  CAT_PATH = shared_local("cat.json")
20  NOTES_CSV = shared_local("notes.csv")
21  CHROMA_DIR = shared_local("chroma_langchain_db")
22  HASH_PATH = shared_local(".csv_hash.json")
23
24
```

Kuva 13. path.py -tiedosto, joka mahdollistaa resurssien jakamisen

Sekä NoteApp, että ChatApp käynnistyvät omien app.py -tiedostojen kautta. Sovellus käynnistetään QApplication-luokalla, joka perii QCoreApplication -luokalta tapahtumasilmukan. App.py:n main-funktio luo ensin QApplication -luokasta instanssin, ja sille välitetään komentoriviargumentit. ChatAppissa kutsutaan seuraavaksi vector.py -tiedoston index_again -funktiota, mutta NoteAppin käynnistämisen yhteydessä tätä vaihetta ei suoriteta. Funktiosta kerrotaan tarkemmin myöhemmin. Tämän jälkeen prosessin mukaan eli riippuen siitä onko kyseessä ChatApp vai NoteApp, joko Form- tai MainWindow -luokasta luodaan käyttäjän ruudulle piirrettävä instanssi.

ChatApp > app.py > ...

```python
import sys
from PySide6.QtWidgets import QApplication
from ui.mainwindow import MainWindow
from rag.vector import index_again

def main():
    app = QApplication(sys.argv)
    index_again()
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()

```

Kuva 14. ChatAppin käynnistystiedosto

5.3 NoteApp

NoteApp on sovelluksen toinen prosessikokonaisuus, joka keskittyy muistiinpanojen tallentamiseen. Sen tärkeimpinä ydintehtävinä on hallinnoida muistiinpanokategorioiden lataamista, tallentamista ja poistamista, sekä varsinaisten muistiinpanojen tallentamista. Muistiinpanojen muokkaamista ja poistamista ei ole tässä prototyyppiversiossa vielä toteutettu.

Käyttäjä voi siis kirjoittaa muistiinpanomerkinnälle otsikon, valita sille sopivan kategorian, kirjoittaa sisällön ja tallentaa muistiinpanon. Käyttäjällä on myös mahdollisuus luoda uusia kategorioita, ja poistaa niitä.

5.3.1 Graafisen käyttöliittymän rakenne

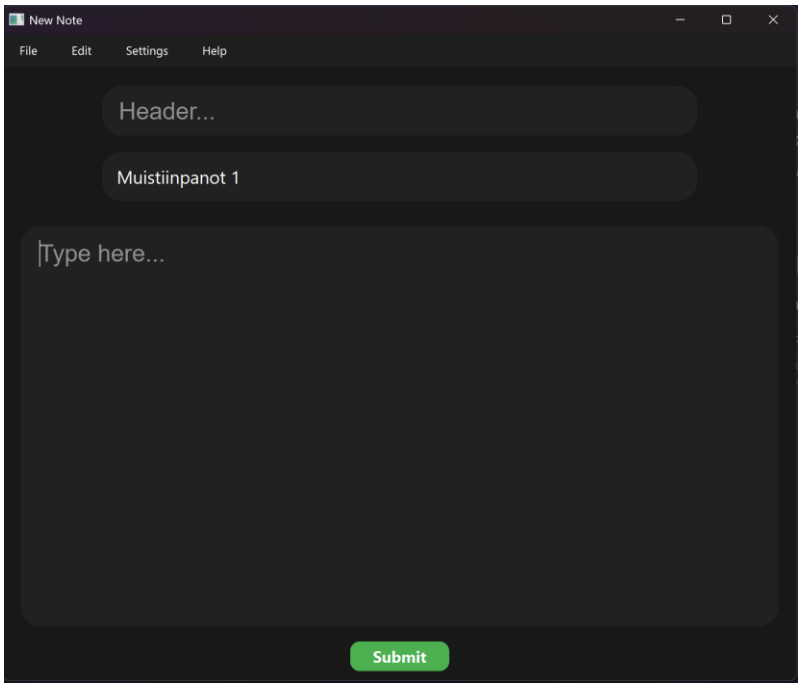
Sovelluksen pääikkuna on toteutettu luokkana Form(QMainWindow), joka perii Qt:n QMainWindow-luokan. Ikkunan käyttöliittymäkomponentit eli valikot, painikkeet ja tekstikentät luodaan QWidget-olioina pääluokan konstruktorissa. Lähes kaikki käyttöliittymäkomponentit sijaitsevat erillisissä tiedostoissa, jotta sovellus pysyy helposti ylläpidettävänä.

Ikkunan yläosassa on valikkopalkki, joka määritellään erillisessä Menu-luokassa ja lisätään pääikkunaan QMainWindow -luokan setMenuBar -metodilla. Sen alapuolella on sisältöalue, joka luotiin QWidget -luokan instanssina nimellä form_widget ja asetettiin keskuskomponentiksi QMainWindow -luokan metodilla setCentralWidget. Ikkunan sisältöalue on siis kuvan 17 mukainen keskuskomponentti: ”central widget”. Lopuksi ikkunalle luotiin vertikaalinen pääasettelu ”form_main_layout” QVBoxLayout -luokan avulla. Tämä tarkoittaa sitä, että kaikki lisätyt alikomponentit pinotaan päällekkäin sarakemaisesti.

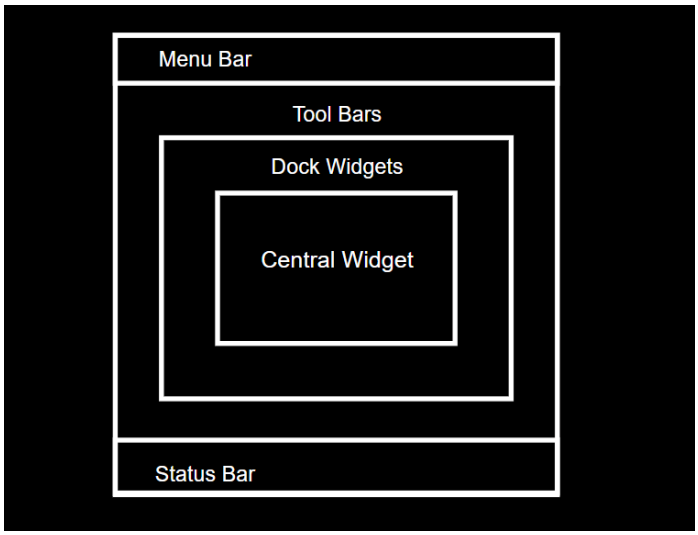
```
form_widget = QWidget()
self.setCentralWidget(form_widget)
form_main_layout = QVBoxLayout(form_widget)
```

Kuva 15. Havainnollistus käyttöliittymäkomponenttien hierarkkisesta rakenteesta

Loput komponentit on lisätty pääasettelun sisään erilaisten säiliöiden avulla. Esimerkiksi kaavion otsikkokenttä ”header_container” on ensin luotu QWidget-luokan instanssina, ja sille on annettu oma horisontaalinen asettelu ”header_layout = QHBoxLayout(header_container)”. Lopuksi varsinainen otsikkoalueen tekstikenttä on syötetty tämän asettelun sisään QLineEdit -luokan instanssina: ”self.header_line = QLineEdit()”. Tämä mahdollistaa siis sen, että komponentteja voidaan asetella vierekkäin, sijoittaa marginaalien avulla haluttuun kohtaan pääikkunaa, ja että niille voidaan antaa erilaisia taustoja. Esimerkiksi kuvassa 16 näkyvällä ”Header…” -tekstin sisältävällä otsikkokentällä on vaaleanharmaa tausta. Taustaväri on määritelty otsikkokentälle eli header_containerille. Otsikkokentän alapuolella on kolme erilaista komponenttia. QComboBox -luokan perivä kategoriavalitsin, QTextEdit -luokan perivä muistiinpanokenttä ja QPushButton -luokan perivä tallennuspainike.



Kuva 16. NoteAppin graafinen käyttöliittymä oletusasetuksilla



Kuva 17. QMainWindow-luokan asettelulogiikka (mukaillen Qt Company Ltd. 2023)

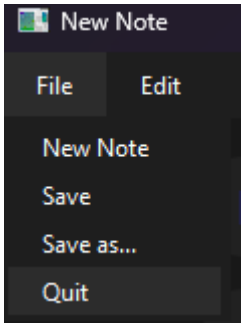
```
class Form(QMainWindow):
    def __init__(self):
        super().__init__()
        if platform.system() == "Windows":
            self.set_dark_titlebar()
        self.setWindowTitle("New Note")
        self.setStyleSheet("background-color: #181818")
        #self.setWindowIcon(QIcon(""))
        self.combo_widget = Combo()

        self.text_edit = QTextEdit(self)
        self.note_widget = Note()
        menu_bar = Menu(self.combo_widget, self.note_widget.note_area)
        self.setMenuBar(menu_bar)
```

Kuva 18. Form-luokan rakennetta

5.3.2 Valikko

Valikkopalkki on käyttöliittymäkomponentti, joka luodaan instanssina omassa tiedostossaan määritellystä Menu -luokasta. Instanssi annetaan argumenttina Form -luokan setMenuBar -metodille, ja näin se on lisätty päänäkymään. Menu-luokka luo neljä erilaista valikkoa: file-valikon, edit-valikon, settings-valikon ja help-valikon. File-valikossa on toistaiseksi koristeena valinnat ”new note”, ”save” ja ”save as…”, jotka eivät tässä prototyyppisovelluksessa suorita mitään. Quit -valinta sen sijaan lopettaa ohjelman suorittamisen pysäyttämällä QApplication -tapahtumasilmukan. Ikkunan voi sulkea myös tavanomaisesta ruksista oikeassa yläkulmassa.

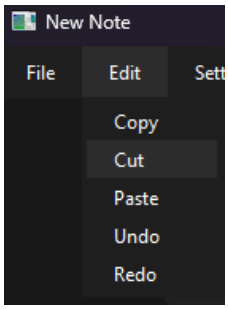


Kuva 19. File-valikon tarjoamat vaihtoehdot

```
new_action = file_menu.addAction("New Note")
#new_action.triggered.connect(self.new_note)
save_action = file_menu.addAction("Save")
saveas_action = file_menu.addAction("Save as...")
quit_action = file_menu.addAction("Quit")
quit_action.triggered.connect(QApplication.quit)
```

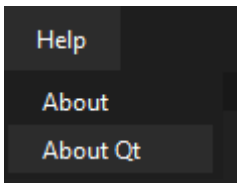
Kuva 20. Vain quit_action on asetettu toiminnalliseksi Qt:n signal-slot -metodiikalla

Edit-valikossa on viisi erilaista muokkaustoimintoa, jotka kaikki on asetettu toiminnallisiksi. Ne muokkaavat muistiinpanokentän sisältöä. Käyttäjä voi kopioida tai leikata muistiinpanotekstikentässä maalatun teksti, ja hän voi myös liittää käyttöjärjestelmänsä leikepöydältä tekstiä kirjoituskenttään. Valikossa on myös undo- ja redo -painikkeet. Valikon käyttö ei ole välttämätöntä, sillä myös perinteiset näppäinkomennot kuten ’ctrl+c’, ’ctrl+v’, ’ctrl+z’ ja ’ctrl+y’ toimivat sovelluksessa.

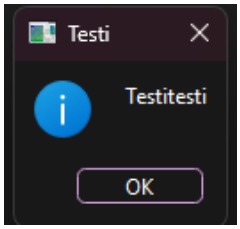


Kuva 21. Muokkauskomennot ”copy, cut, paste, undo & redo” valikkopalkin kautta

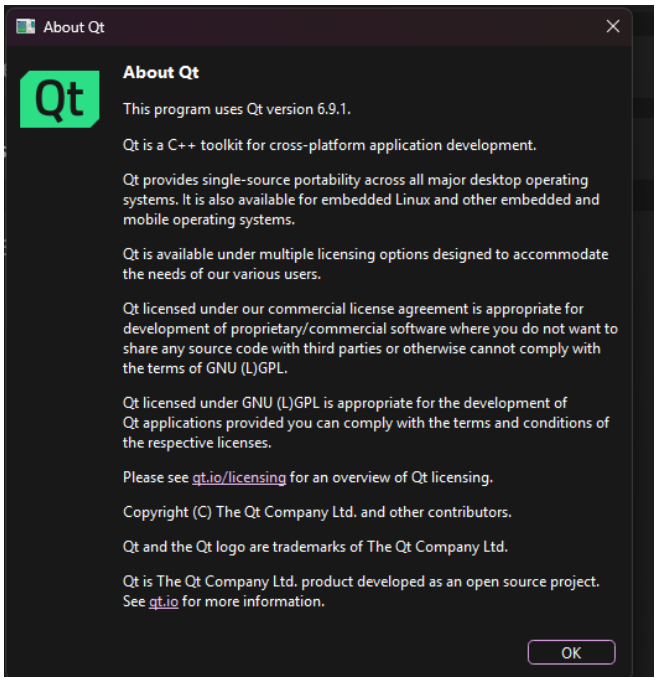
Valikkopalkin Help-valikko antaa mahdollisuuden avata joko ”About” tai ”About Qt” vaihtoehdon. About-painike avaa kuvan 23 mukaisen dialogin, johon on tarkoituksena lisätä myöhemmin esimerkiksi linkki sovelluksen käyttömanuaaliin tai muuta hyödyllistä sisältöä. ”About Qt” -valinta puolestaan avaa dialogin, joka kertoo käyttäjälle tietoa Qt:sta ja esimeriksi sen, että millä Qt:n versiolla sovellus on rakennettu.



Kuva 22. Help-valikko



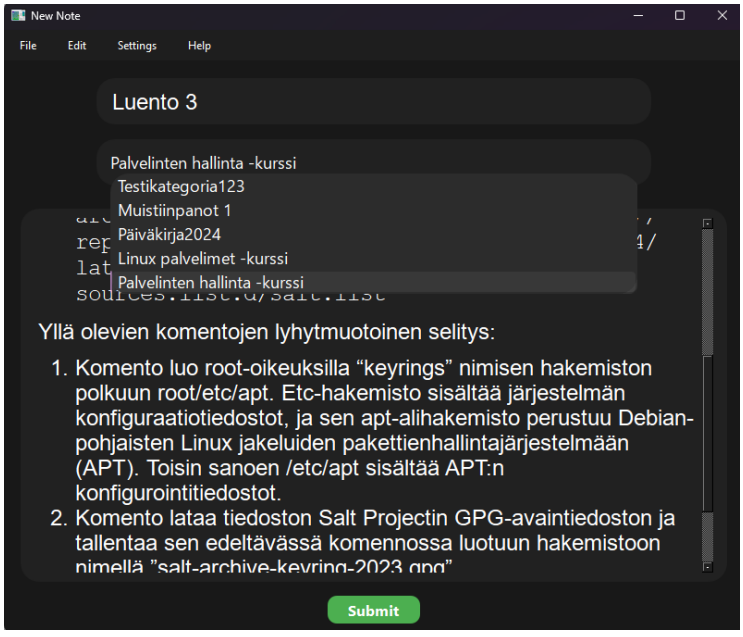
Kuva 23. About -ilmoitus



Kuva 24. About Qt -ilmoitus

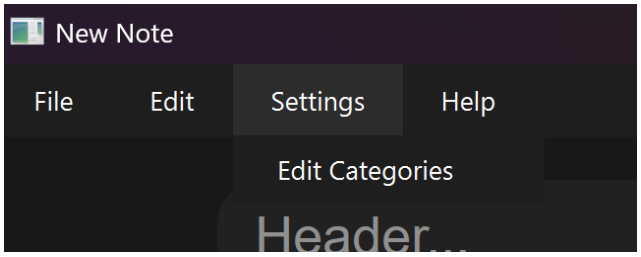
5.3.3 Kategoriat ja niiden käsitteleminen

Kategoriavalikko luodaan instanssina QComboBox -luokan perivästä Combo()-luokasta. Käynnistyksen yhteydessä Combo lataa JSON-tiedostoon tallennetut kategoriat. Jos kategorioiden lataus palauttaa tyhjän listan käynnistettäessä, luodaan oletuskategoria ”Muistiinpanot 1” automaattisesti. Käyttäjällä on mahdollisuus luoda ja poistaa kategorioita valikkopalkin kautta.



Kuva 25. Kategorian valitseminen pudotusvalikosta

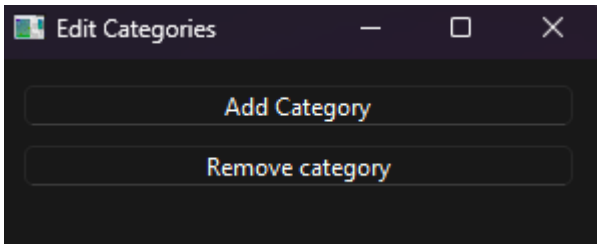
Kun käyttäjä valitsee valikkopalkista kohdan Settings -> Edit categories, avautuu kuvassa 28 esitetty ikkuna, jossa käyttäjän on mahdollista valita uuden kategorian lisääminen tai vanhan kategorian poistaminen. Tämä ikkuna on catWindow -luokasta konstruoitu instanssi.



Kuva 26. Kategorioiden muokkaaminen valikkopalkin kautta

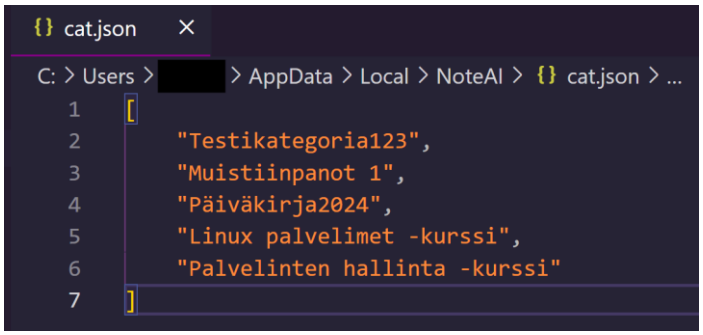
```
def open_cat_edit(self):
    self.edit_window = catWindow(self.combo_widget)
    self.edit_window.show()
```

Kuva 27. Open_cat_edit -metodi

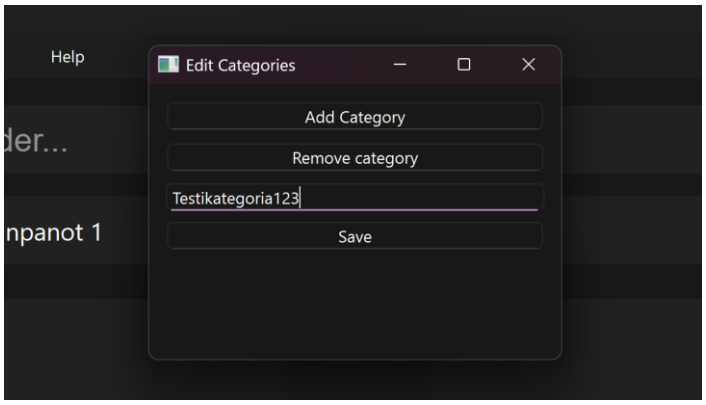


Kuva 28. Edit Categories -valikon avaama ikkuna

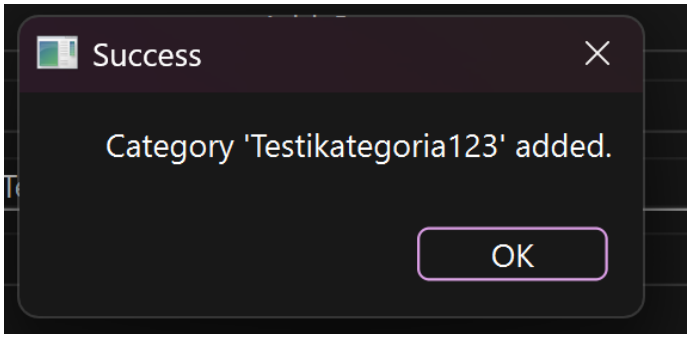
Kategorian lisääminen tapahtuu valitsemalla ”Add Category”. Käyttäjä voi kirjoittaa avautuvaan tekstikenttään uuden kategorian nimen ja tallentaa sen painamalla tallennuspainiketta kuvan 30 mukaisesti. Käyttäjä saa tämän jälkeen ponnahdusikkunalla ilmoituksen onnistuneesta tai epäonnistuneesta tapahtumasta. Kategoriat päivitetään pääikkunan kategoriavalikkoon ja ne tallentuvat pysyvästi paikalliseen JSON-tiedostoon.



Kuva 29. Esimerkkikategoriat JSON-tiedostossa

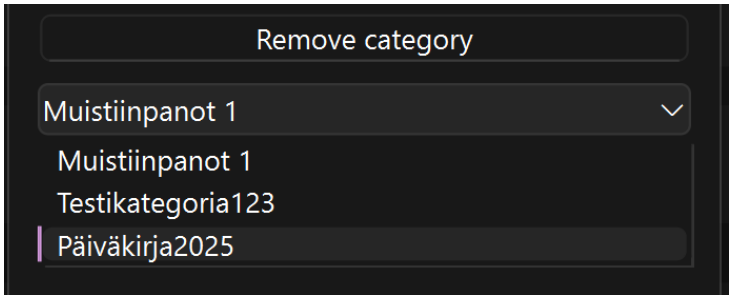


Kuva 30. Uuden kategorian lisääminen

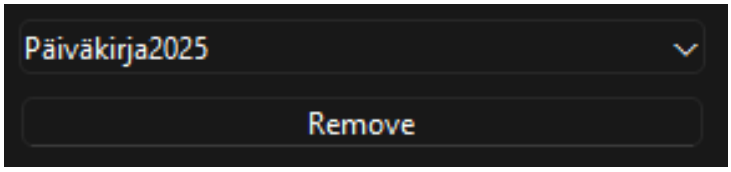


Kuva 31. Ilmoitus onnistuneesta kategorian lisäämisestä

Valitsemalla kohdan ”remove category” edit categories -valikossa, voi käyttäjä valita poistettavan kategorian pudotusvalikosta. Tämän jälkeen painamalla ”remove” -painiketta, poistuu kategoria muistista. Tämänkin toimenpiteen onnistumisesta ilmoitetaan ponnahdusikkunalla.

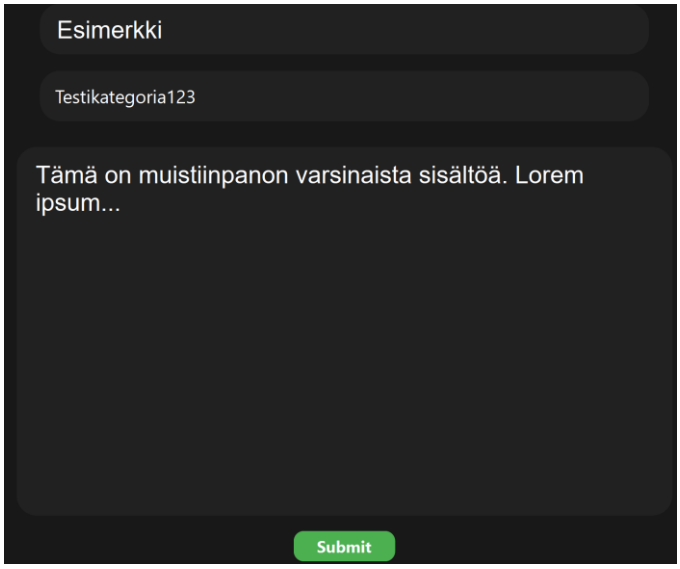


Kuva 32. Poistettavan kategorian valitseminen dropdown-valikosta



Kuva 33. Valitun kategorian poistaminen ”Remove” -painikkeella

5.3.4 Muistiinpanojen tallentaminen

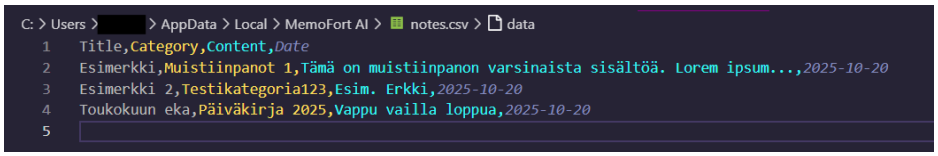


Kuva 34. MemoFortin muistiinpanokenttä

Muistiinpanon tallentaminen perustuu Form -luokan save_note -metodiin, jota kutsutaan, kun käyttäjä painaa signaalin: ”self.submit_button.clicked.connect(self.save_note)” lähettävää vihreää tallennuspainiketta pääikkunan alareunassa. Tapahtuma aktivoi save_note -metodin, joka tarkistaa, että otsikko- ja sisältökentät eivät ole tyhjiä, ja tallentaa muistiinpanon paikalliseen sovellushakemistoon CSV-tiedostona. Tallentamisen jälkeen syöttökentät tyhjenevät automaattisesti, ja käyttäjä voi aloittaa seuraavan muistiinpanon kirjoittamisen saumattomasti. CSV-tiedostoon tallennetaan muistiinpanon otsikko, kategoria, sisältö sekä päivämäärä.

```
154
155     def save_note(self):
156         note_text = self.note_widget.note_area.toPlainText().strip()
157         title = self.header_line.text().strip()
158         if not note_text or not title:
159             QMessageBox.information(self, "Huom", "Otsikko ja sisältö vaaditaan.")
160             return
161 
162         csv_path = shared_local("notes.csv")
163         file_exists = os.path.isfile(csv_path)
164         file_empty = os.path.getsize(csv_path) == 0 if file_exists else True
165 
166         with open(csv_path, mode="a", newline="", encoding="utf-8") as f:
167             writer = csv.writer(f)
168             if not file_exists or file_empty:
169                 writer.writerow(["Title", "Category", "Content", "Date"])
170             writer.writerow([
171                 title,
172                 self.combo_widget.dropdown.currentText(),
173                 note_text,
174                 datetime.now().date().isoformat()
175             ])
176 
177         self.header_line.clear()
178         self.note_widget.note_area.clear()
179
```

Kuva 35. Form-luokan save_note() -metodi



Kuva 36. CSV-tiedostoon tallennettuja esimerkkimuistiinpanoja

5.4 ChatApp

ChatApp on sovelluksen toinen prosessikokonaisuus, jonka ydintehtävänä on toimia visuaalisesti miellyttävänä, graafisena rajapintana, jossa käyttäjä voi keskustella tekoälyagentin kanssa itse tekemistään muistiinpanoista. ChatApp vastaa myös muistiinpanojen upottamisesta vektoritietokantaan ja LLM-agentin ohjaamisesta.

ChatAppin graafinen käyttöliittymä on rakennettu samalla periaatteella kuin NoteApp. Sen pääikkuna on myös QMainWindow -luokan perivä MainWindow -luokka, joka sisältää käyttöliittymäkomponentit kuten keskustelukentän, puhekuplat, tekstinsyöttökentän sekä valikkopalkin.

5.4.1 Vektoritietokanta ja retriever

ChatAppin käynnistämisen yhteydessä suoritetaan vector.py -tiedostossa sijaitseva funktio index_again, joka mainittiin luvussa 5.2. Funktio tarkistaa NoteAppin luoman notes.csv -tiedoston md5-tiivisteen arvon. Jos CSV-tiedostoa ei ole tai sen tiiviste täsmää aiemmin muistiin tallennettuun tiivisteeseen, päättyy funktion suorittaminen tähän. Sen sijaan, mikäli tiiviste on muuttunut eli muistiinpanoja on lisätty, aloitetaan niiden upottaminen Chroma-tietokantaan.

Index_again -funktio luo dokumenteille ja niiden yksilöiville tunnisteille omat listat, ja käynnistää silmukan, jossa käydään pythonin pandas-kirjaston avulla luettavan CSV:n sisältö läpi. Muistiinpanojen otsikko-, kategoria- ja sisältökentät yhdistetään yhdeksi merkkijonoksi kutsumalla row_to_text -funktiota, ja tämä paluuarvo tallennetaan muuttujaan ’text’. Seuraavaksi LangChainin Document -luokasta luodaan doc -instanssi, jonka sisältöarvoksi tallennetaan juuri luotu text-muuttuja, ja metatiedoksi muistiinpanon tallennusajankohta. Jokainen tällainen instanssi lisätään listaan, kunnes kaikki muistiinpanot on käyty läpi. Lopuksi funktio tyhjentää vanhan vektoritietokannan sisällön, ja täyttää sen uusilla arvoilla. Uuden CSV-tiedoston md5-tiiviste tallennetaan kutsumalla save_hash -funktiota.

Memofortin prototyyppiversiossa dokumenttien yksilöivillä tunnisteilla ja näistä luodulla listalla ei ole merkitystä, ja ominaisuus on lisätty vain jatkokehittämistä varten. Nykytilassaan tietokanta rakennetaan aina kokonaan uudestaan, kun siinä tapahtuu pieninkin muutos. Tämä ei ole järkevää varsinkaan silloin, kun dokumentteja on massiivinen määrä.

```python
def index_again() -> None: #rakentaa/päivittää chroman, kutsutaan \app.py
    current = file_md5(NOTES_CSV) #tarkistetaan nykyinen md5-tiiviste
    if not current:
        print(f"[Memofort AI] Huom: CSV puuttuu: {NOTES_CSV}")
        return

    if current == load_hash(): #verrataan nykyistä tiivistettä vanhaan
        print("[Memofort AI] CSV ei muuttunut – indeksi ennallaan.")
        return

    print("[MemoFort AI] CSV muuttunut – rakennetaan indeksi...") #indeksoidaan chromaan
    df = pd.read_csv(NOTES_CSV).fillna("") #luetaan pandasilla, NaN korvataan: ""
    docs, ids = [], [] #dokumenteille ja hasheille omat listat
    for _, row in df.iterrows(): #ohitetaan indeksi, käsitellään vain sisältö
        text = row_to_text(row) #sisällöt merkkijonoksi
        if not text:
            continue
        meta = {"date": str(row.get("Date", ""))} #metatiedoksi päivämäärä
        doc = Document(page_content=text, metadata=meta) #LangChainin luokasta instanssi
        doc_id = hashlib.md5(text.encode("utf-8")).hexdigest()
        docs.append(doc); ids.append(doc_id) #lisätään listoihin

    try:
        vector_store.delete(ids=None)  #tyhjennetään tietokanta
    except Exception:
        pass
    if docs:
        vector_store.add_documents(docs, ids=ids) #täytetään tietokanta
        save_hash(current) #luodaan uusi md5-tiiviste
        print("[MemoFort AI] Indeksi rakennettu.")
```

Kuva 37. Index_again() -funktio

Vector.py -tiedosto sisältää myös retrieverin, joka noutaa relevantit muistiinpanot vektoritietokannasta. Retriever on käytännössä sovelluksen tekoälyagentin hakukone. Sovelluksen hakualgoritmiksi valittiin max marginal relevance -funktio (MMR) eli käytännössä retriever hakee ensin kaksikymmentä osuvinta muistiinpanoa vektoritietokannasta ja laskee niille MMR-pisteytyksen.

MMR:n kaava on $\mathrm{MMR}(D_i) = \lambda \times Sim_1(D_i, \mathrm{Q}) - (1 - \lambda) \times \max(Sim_2(D_i, D_j))$. Missä λ on painokerroin (λ ∈ [0,1]), D_i on käsiteltävä dokumentti, $Sim_1(D_i, Q)$ on dokumentin ja kyselyn Q samankaltaisuutta mittaava funktio ja $Sim_2(D_i, D_j)$ -funktio vertaa valittavaa dokumenttia jo valittuun dokumenttiin D_j. Koska jo valittuja dokumentteja voi olla useita, valitaan näistä max()-funktion avulla se, joka muistuttaa eniten jo valittua dokumenttia. Kun λ arvo lähestyy nollaa, valitaan yhä keskenään erilaisempia dokumentteja, mutta ne eivät välttämättä enää vastaa käyttäjän kyselyyn Q. Kun lambdan arvo lähestyy yhtä, tapahtuu päinvastainen eli dokumentit vastaavat yhä paremmin käyttäjän kyselyyn, mutta ovat potentiaalisesti keskenään hyvin samankaltaisia. (Carbonell & Goldstein 1998, 335; Straßer 2025.) Prosessi toistetaan funktiossa viidesti eli siinä valitaan yhteensä viisi osuvinta dokumenttia, jotka ovat kuitenkin kohtalaisen erilaisia keskenään.

```python

def get_retriever():
    return vector_store.as_retriever(
        search_type="mmr", #max_marginal_relevance
        search_kwargs={"k": 5, "fetch_k": 20, "lambda_mult": 0.7}
)

```

Kuva 38. Retriever, MMR ja top K

5.4.2 Puhekuplien konstruointi

Kun käyttäjä kirjoittaa kenttään tekstiä ja lähettää sen klikkaamalla lähetyspainiketta tai painamalla näppäimistön enter-näppäintä, kutsutaan MainWindow -luokan process_message -metodia. Metodi kytkeytyy melko monimutkaiseen prosessiin, joten käsittelemme ensin vain sen roolia puhekuplien luomisessa. MainWindow luo puhekuplakomponentit dynaamisesti eli ChatBubble -luokasta luodaan uusi instanssi aina kun käyttäjä lähettää tai vastaanottaa viestin. Tapahtumaketjun alussa luodaan user_bubble -instanssi. Keskusteluikkunaan päivitetään tällöin välittömästi puhekupla, jonka sisällä näkyy käyttäjän lähettämä viesti. Samanaikaisesti ikkunaan päivittyy thinking_bubble -puhekuplainstanssi, joka sisältää kolmen peräkkäisen pisteen merkkijonon ”…”. Tämä antaa käyttäjälle indikaation siitä, että tekoäly miettii vastaustaan. Myöhemmin tälle placeholder-tekstille on tarkoitus antaa jonkinlainen animaatio.

```
        #KÄYTTÄJÄKUPLA
        if sender == "user":
            self.bubble.setStyleSheet("""
                background-color: #212121;
                color: white;
                padding: 10px;
                border-radius: 10px;
            """)
            layout.addStretch() #Lisätään stretch vasemmalle, puhekupla tulee ruudun oikeaan reunaan
            layout.addWidget(self.bubble) #Lisätään kupla

        #AI-KUPLA
        elif sender == "ai":
            self.bubble.setStyleSheet("""
                color: white;
                padding: 10px;
                border-radius: 10px;
            """)
            layout.addWidget(self.bubble) #++kupla
            layout.addStretch() #Lisätään stretch oikealle, puhekupla tulee ruudun vasempaan reunaan

        #VIRHEKUPLA
        else:
            self.bubble.setStyleSheet("""
                background-color: #b71c1c;
                color: white;
                padding: 10px;
                border-radius: 10px;
            """)
            layout.addWidget(self.bubble) #++kupla
            layout.addStretch() #Lisätään stretch oikealle, kupla vasemmalle

        self.setLayout(layout)
```

Kuva 39. Erilaisten puhekuplien määrittely ChatBubble-luokassa

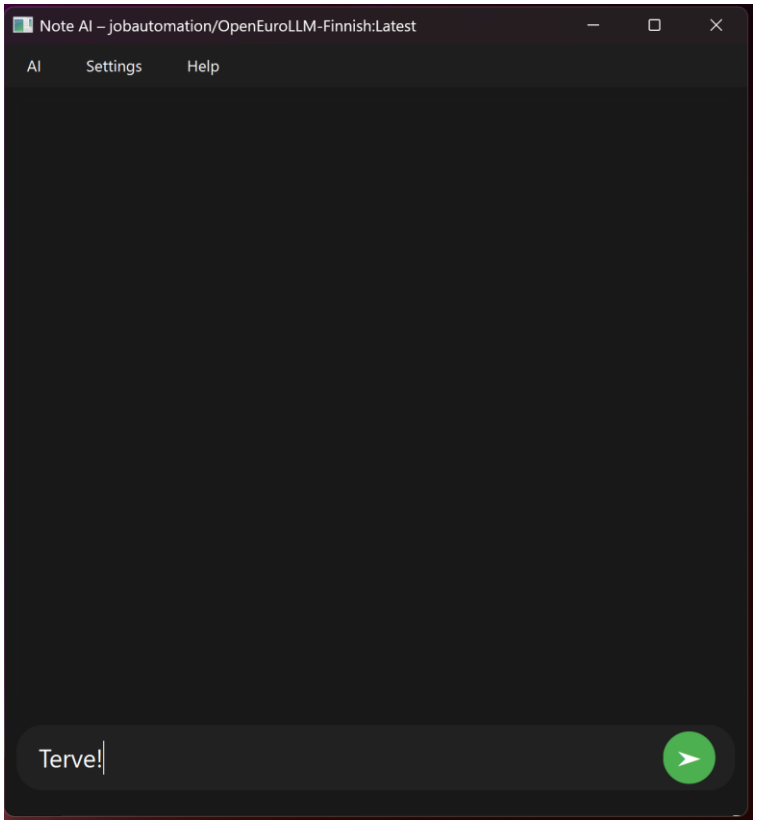
```
def process_message(self, user_text): #inputin messageSent metodi käynnistää funktion
    user_bubble = ChatBubble(user_text, sender="user") #luodaan user_bubble instanssi Chatbubble luokasta
    user_bubble.set_font_size(self.current_font_size) #fonttikoon päivitys
    self.scroll_layout.addWidget(user_bubble) #lisätään user_bubble scroll_layoutin sisään

    thinking_bubble = ChatBubble("…", sender="ai")
    thinking_bubble.set_font_size(self.current_font_size)
    self.scroll_layout.addWidget(thinking_bubble)
```

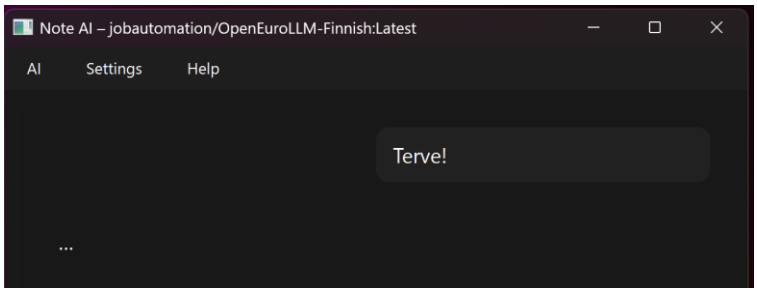
Kuva 40. Process_message -metodin alku

Pääikkunan asettelussa puhekuplainstanssit pinotaan vertikaalisesti. Puhekuplainstanssit ovat horisontaalisia olioita, ja niitä on kolme eri tyyppiä: käyttäjän lähettämä viesti (user_bubble), AI-agentin lähettämä viesti (thinking_bubble) sekä virheviesti (error_bubble). Käyttäjän puhekupla saadaan ruudun oikeaan reunaan lisäämällä QHBoxLayout -asetteluun ensin tyhjä tila, ja vasta tämän jälkeen varsinainen puhekupla. Tekoälyn viestin ja virheen kohdalla prosessi on päinvastainen eli ensiksi lisätään puhekupla ja vasta tämän jälkeen tyhjä tila. Tätä on havainnollistettu kuvassa 43,

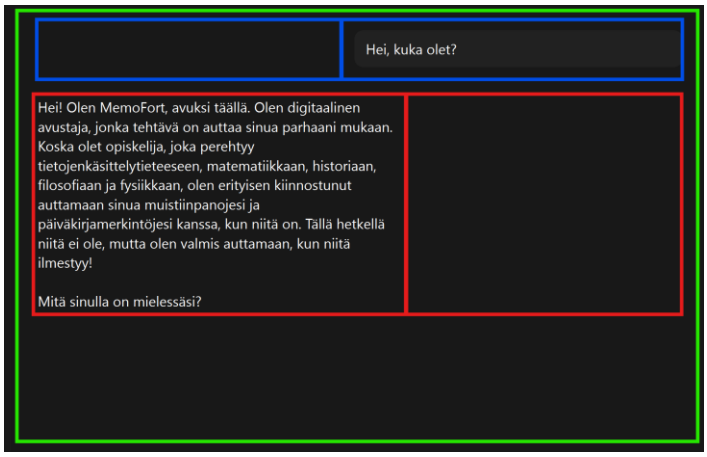
jossa vihreällä merkittyyn QHBoxLayout-asetteluun pinotaan puhekuplainstansseja. Virheviesti ja käyttäjäviesti saavat taustan, mutta tekoälyn tuottama vastaus on läpinäkyvällä tekstillä. Tällä tyylittelyvalinnalla on matkittu tunnettuja LLM-keskustelusovelluksia, kuten ChatGPT:tä ja Copilotia.



Kuva 41. ChatAI:n perusnäkymä ennen viestin lähettämistä

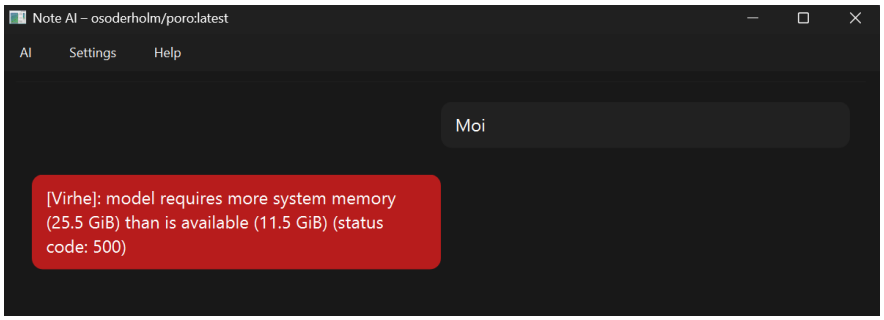


Kuva 42. ChatAI:n näkymä viestin lähettämisen jälkeen, ja thinking_bubble -instanssi

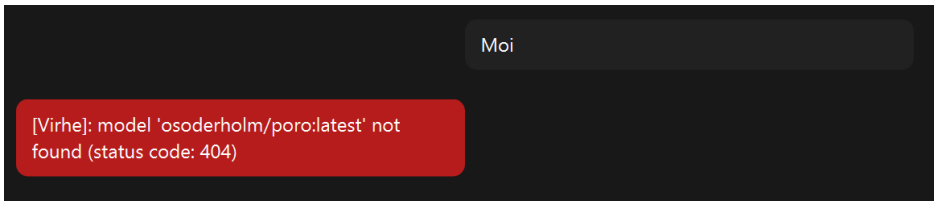


Kuva 43. Havainnollistus puhakuplainstanssien rakenteesta ja vastauksella päivitetystä thinking_bubble -instanssista

Thinking_bubble -instanssin eli mietiskelykuplan sisältämä ”…” -viesti päivitetään kielimallin tarjoamalla vastauksella tai poikkeustapauksessa koko kupla korvataan virheilmoituksen sisältävällä error_bubble -instanssilla. Lopuksi keskustelunäkymä rullataan automaattisesti pohjalle QTimer-luokan singleShot-metodin avulla, joka odottaa aiemman käyttöliittymäpäivityksen päättymistä ennen toimenpiteiden suorittamista.



Kuva 44. Esimerkki virheilmoituksesta käytettäessä LLM-mallia, joka vaatii liikaa keskusmuistia

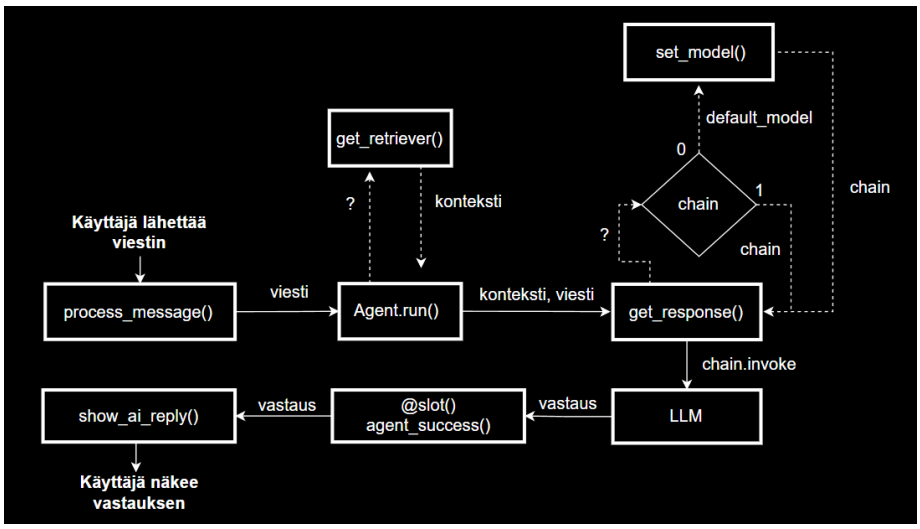


Kuva 45. Esimerkki virheilmoituksesta käytettäessä poistettua tai olematonta kielimallia

5.4.3 Tekoälyagentti

Taustasäikeessä toimivan tekoälyagentin rooli on periaatteessa melko suoraviivainen. Se hakee ensin vektoritietokannasta kontekstin, ja välittää sen käyttäjän lähettämän viestin ohella HTTP-pyyntönä Ollaman tarjoamalle kielimallille. Kielimallin palauttama vastaus siirtyy takaisin ChatAppin

pääsäikeeseen, jossa se päivitetään oikean puhekuplan sisään. Käytännön toteutus on hieman monimutkaisempi, ja se on pääpiirteissään esitetty seuraavassa kuvassa 46.



Kuva 46. MemoFortin tekoälyagentin toimintalogiikka

Aiemmassa luvussa esitelty process_message -metodi hallinnoi myös agenttia. Käyttäjän lähetettyä viestin, se luo instanssin Agent -luokasta sekä sellaisen uuden taustasäikeen QThread -luokasta, johon agentti siirretään, minkä jälkeen se ajaa vielä Agent.run -metodin. Uusi säie luodaan, jotta kaikki graafisen käyttöliittymän päivitykset suoritettaisiin vain pääsäikeessä. Jos RAG-prosessi suoritettaisiin myös ohjelman pääsäikeessä, jäätyisi ikkuna täysin toimenpiteiden ajaksi, mikä on ei ole suotavaa käyttäjäkokemuksen kannalta. Agentin palautettua tekstimuotoisen vastauksen tai virheen, se pysäytetään ja poistetaan taustasäikeen mukana, jotta muistivuotoja ei sattuisi.

```
self.thread = QThread(self) #luodaan uusi säie
self.worker = Agent(user_text) #luodaan worker-instanssi Agent-luokasta
self.worker.moveToThread(self.thread) #siirretään agentti uuteen säikeeseen
self.thread.started.connect(self.worker.run) #käynnistetään agentti

#SUCCESS
self.worker.finished.connect(self.agent_success) #ohjataan signaali pääsäikeen slotiin
#ERROR
self.worker.error.connect(self.agent_error) #ohjataan signaali pääsäikeen slotiin

#siivous
self.worker.finished.connect(self.thread.quit) #lopetetaan säie
self.worker.finished.connect(self.worker.deleteLater) #poistetaan agentti
self.worker.error.connect(self.thread.quit)
self.worker.error.connect(self.worker.deleteLater)

self.thread.finished.connect(self.thread.deleteLater) #poistetaan säie

self.thread.start() #käynnistetään säie
```

Kuva 47. Process_message()-metodin osa, joka käynnistää AI-agentin ja ohjaa sen vastauksen oikeaan paikkaan

```python
class Agent(QObject): #Agentti -Luokka
    finished = Signal(str) #palauttaa signaalin merkkijonona (kts. process_message -metodi)
    error = Signal(str) #-||-
    def __init__(self, question):
        super().__init__()
        self.question = question

    def run(self): #metodi ajetaan toisessa säikeessä (kts. process_message -metodi)
        try:
            retriever = get_retriever() #kts. \rag\vector.py
            try:
                docs = retriever.invoke(self.question)
            except Exception:
                docs = [] #palautetaan tyhjä lista virheen sattuessa
            context = "(Muistiinpanoja ei ole)" if not docs else "\n\n".join( #eri dokumentit erotellaan kahdella rivinvaihdolla
                getattr(d, "page_content", str(d)) for d in docs #palauttaa page_contentin tai merkkijonon | käy kaikki läpi
            )
            reply = get_response(context, self.question) #kts. \rag\ai.py
            self.finished.emit(reply)
        except Exception as e: #virhe palaa merkkijonona
            self.error.emit(str(e))
```

Kuva 48. Agent -luokka

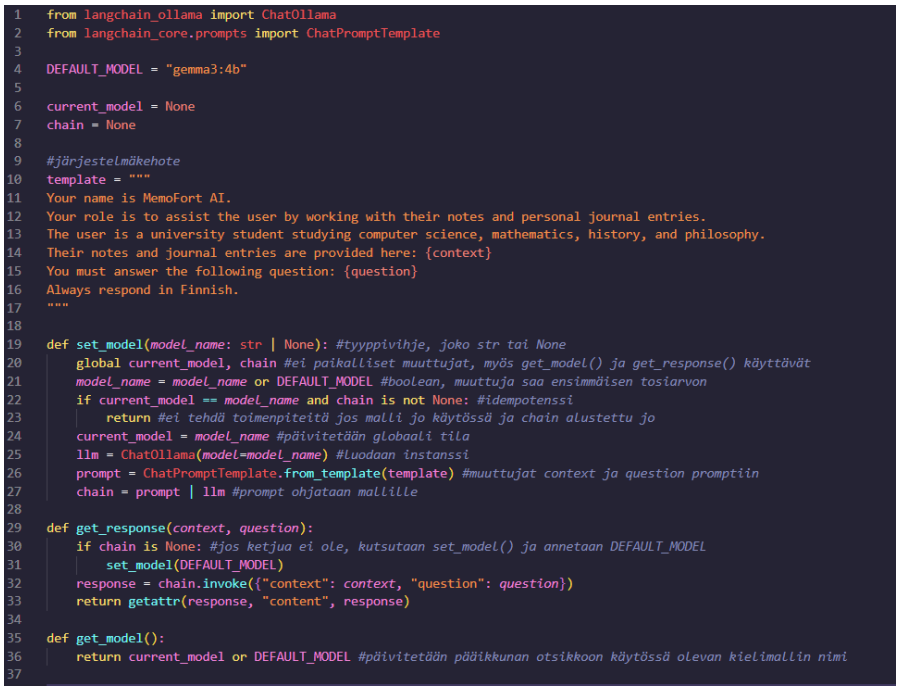
Agent.run -metodi kutsuu vector.py -luokassa määriteltyä get_retriever -funktiota. Retriever palauttaa viisi osuvinta muistiinpanoa, ja ne tallennetaan context -muuttujaan. Seuraavaksi metodi kutsuu funktiota get_response, ja antaa sille argumentteina Chroma-vektoritietokannasta juuri haetun kontekstin sekä käyttäjän syöttämän viestin.

Get_response -funktio käynnistää ketjun, jossa järjestelmäkehote ohjataan LangChainin ChatOllama -rajapinnasta konstruoidulle llm -instanssille. Funktion paluuarvo eli llm -instanssin vastaus tallentuu muuttujaan ’reply’ Agent.run -metodissa, ja se emittoidaan signaalina. Virhetilanteen sattuessa emittoidaan sen sijaan virheilmoitus merkkijonon muodossa.

MainWindow sisältää kaksi slottia, joista toinen käsittelee Agent.run -metodin virhesignaalia, ja toinen vastaussignaalia. Slotit ovat pääsäikeessä, jotta LLM-agentista vastaava taustasäie ei sekaantuisi graafisen käyttöliittymän päivittämiseen. Slotit kutsuvat joko Form.show_ai_reply- tai Form.show_ai_error -metodia, jotka päivittävät aiemmin luodun thinking_bubble -instanssin korvaamalla sen sisältämän ”…” -paikka-arvon agentin palauttamalla vastauksella tai koko kuplan virhekuplalla.

```python
178
179     @Slot(str) #vastaus
180     def agent_success(self, text:str):
181         if self.thinking_bubble is not None:
182             self.show_ai_reply(self.thinking_bubble, text)
183             self.thinking_bubble = None
184
185     @Slot(str) #virhe
186     def agent_error(self, msg:str):
187         if self.thinking_bubble is not None:
188             self.show_ai_error(self.thinking_bubble, msg)
189             self.thinking_bubble = None
190
```

Kuva 49. Taustasäikeestä lähteviä signaaleja käsittelevät pääsäikeen slotit



```python
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate

DEFAULT_MODEL = "gemma3:4b"

current_model = None
chain = None

#järjestelmäkehote
template = """
Your name is MemoFort AI.
Your role is to assist the user by working with their notes and personal journal entries.
The user is a university student studying computer science, mathematics, history, and philosophy.
Their notes and journal entries are provided here: {context}
You must answer the following question: {question}
Always respond in Finnish.
"""

def set_model(model_name: str | None): #tyyppivihje, joko str tai None
    global current_model, chain #ei paikalliset muuttujat, myös get_model() ja get_response() käyttävät
    model_name = model_name or DEFAULT_MODEL #boolean, muuttuja saa ensimmäisen tosiarvon
    if current_model == model_name and chain is not None: #idempotenssi
        return #ei tehdä toimenpiteitä jos malli jo käytössä ja chain alustettu jo
    current_model = model_name #päivitetään globaali tila
    llm = ChatOllama(model=model_name) #luodaan instanssi
    prompt = ChatPromptTemplate.from_template(template) #muuttujat context ja question promptiin
    chain = prompt | llm #prompt ohjataan mallille

def get_response(context, question):
    if chain is None: #jos ketjua ei ole, kutsutaan set_model() ja annetaan DEFAULT_MODEL
        set_model(DEFAULT_MODEL)
    response = chain.invoke({"context": context, "question": question})
    return getattr(response, "content", response)

def get_model():
    return current_model or DEFAULT_MODEL #päivitetään päikkunan otsikkoon käytössä olevan kielimallin nimi

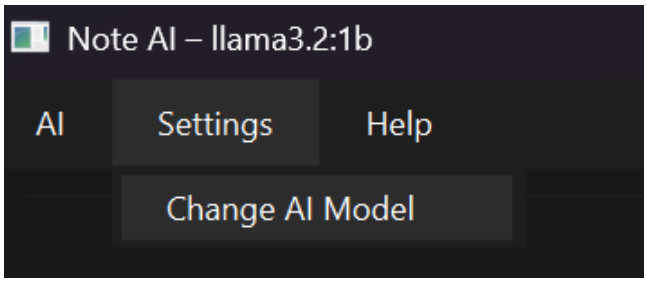
```

Kuva 50. ai.py -tiedosto eli sovelluksen tekoälymoottori

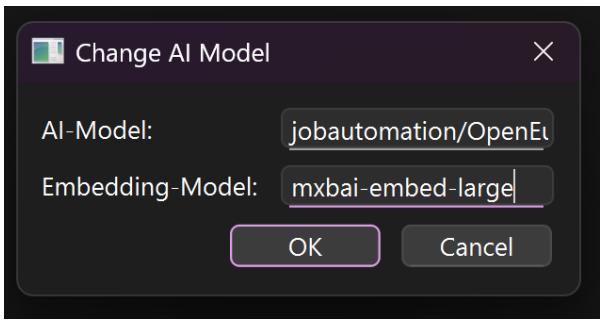
5.4.4 Mallien vaihtaminen

Kuvassa 50 esitetty ai.py sisältää kaksi globaalia muuttujaa ”current_model” ja ”chain”, joiden oletusarvo on None. Näitä muuttujia tarvitaan, jotta käyttäjä voisi vaihtaa kielimallia muokkaamatta lähdekoodia. Muuttujiin tallennetaan tieto nykyisestä kielimallista sekä aiemmin muodostetusta ketjusta. Jos ketjua ei ole vielä luotu, kutsutaan tekoälyagentin ajamassa get_response -funktiossa set_model -funktiota. Kyseistä funktiota kutsutaan myös silloin, kun käyttäjä vaihtaa kielimallia. Chain on LangChain -ketju, jossa järjestelmäkehote ohjataan kielimallille. Kun ohjelma suoritetaan ensimmäistä kertaa, ei muuttujalla chain ole arvoa. Tässä tapauksessa set_model -funktio antaa sille argumenttina ai.py -tiedostoon tallennetun oletuskielimallin ”gemma3:4b”.

Mallin vaihtaminen onnistuu ChatAppin oman valikkopalkin kautta. Tämä valikkopalkki on suppeampi kuin NoteAppissa. Esimerkiksi ’Help’-valikko ei näytä edes valittavia vaihtoehtoja, ja ’AI’-valikko tarjoaa vain Qt:n tapahtumasilmukan lopettavan eli sovelluksen pysäyttävän quit -vaihtoehdon. Valikkopalkin ’Settings’-valikossa on kuitenkin vaihtoehtona kuvassa 51 esitetty Change AI Model -toiminto, joka avaa kuvan 52 mukaisen ikkunan.

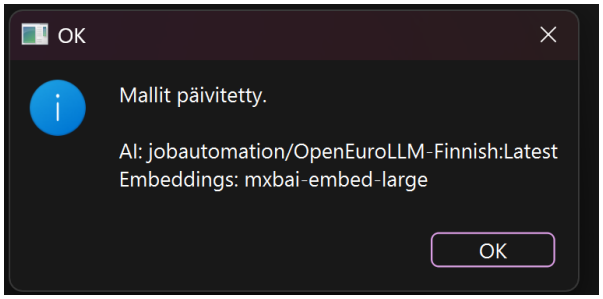


Kuva 51. NoteAppin valikkopalkki



Kuva 52. Dialogi, jonka kautta kielimallia voidaan vaihtaa

Kielimallien vaihtaminen onnistuu Change AI Model -ikkunassa, mutta prototyypissä toteutus on melko kankea. Käyttäjän on syötettävä nykyisessä toteutuksessa valittava malli tekstikenttään tarkassa muodossaan ilman kirjoitusvirheitä. Toimintoa on tarkoitus muuttaa myöhemmin sellaiseksi, että mallin voi valita pudotusvalikosta. Sovelluksen prototyyppi ei myöskään anna virheilmoitusta tilanteessa, jossa käyttäjä vaihtaa tilalle olemattoman mallin. Virheilmoitus näkyy vasta silloin, kun tällaiselle mallille lähetetään viesti kuten kuvassa 45.



Kuva 53. Ilmoitus mallin vaihtamisesta



Kuva 54. Käyttäjän asetuksista vastaava tiedosto settings.py

Käyttäjän valitsemat tiedot tallennetaan JSON-tiedostoon. Kun sovellus käynnistetään, oletusasetukset sisältävän DEFAULTS-sanakirjan avain-arvo-parien arvot korvataan niiltä osin, kuin ne ovat JSON-tiedostossa muuttuneet.

6 Sovelluksen suorituskyky

MVP-tasolle kehitettyä MemoFortia on arvioitu sekä kvantitatiivisesti vasteajan osalta, että kvalitatiivisesti arvioimalla vastausten laatua. Testi on vain suuntaa antava; sitä ei ole toteutettu ihanteellisissa laboratorio-olosuhteissa, vaikka suoritusympäristöä on pyritty vakioimaan runsaasti. Tavoite ei ole arvioida taustalla vaikuttavia kielimalleja moniulotteisesti vaan muodostaa käsitys opinnäytetyössä kehitetyn sovelluksen suorituskyvystä kannettavalla tietokoneella.

Testissä mitataan kontekstihaun, kontekstilaajennetun päättelyn, ja näiden yhteenlaskettua kestoa. Vastausten laatua on myös arvioitu kvalitatiivisesti pohtimalla niiden oikeellisuutta, helppolukuisuutta ja hyödyllisyyttä.

6.1 Testidata ja testikysymykset

Sovelluksen toimivuutta varten laadittiin muistiinpanoja aiheista, joilla kielimallia ei varmasti ole koulutettu. Laadin niille kriteerit, jotka näkyvät tarkassa muodossaan kuvassa 55 GPT5-mallille antamissani ohjeissa. Ideana oli luoda kuvitteelliset muistiinpanot kuudelta eri luennolta, joissa kolme fiktiivistä professoria esittävät fiktiivisiä keksintöjään niin teknisistä, kuin yhteiskunnallisistakin näkökulmista.

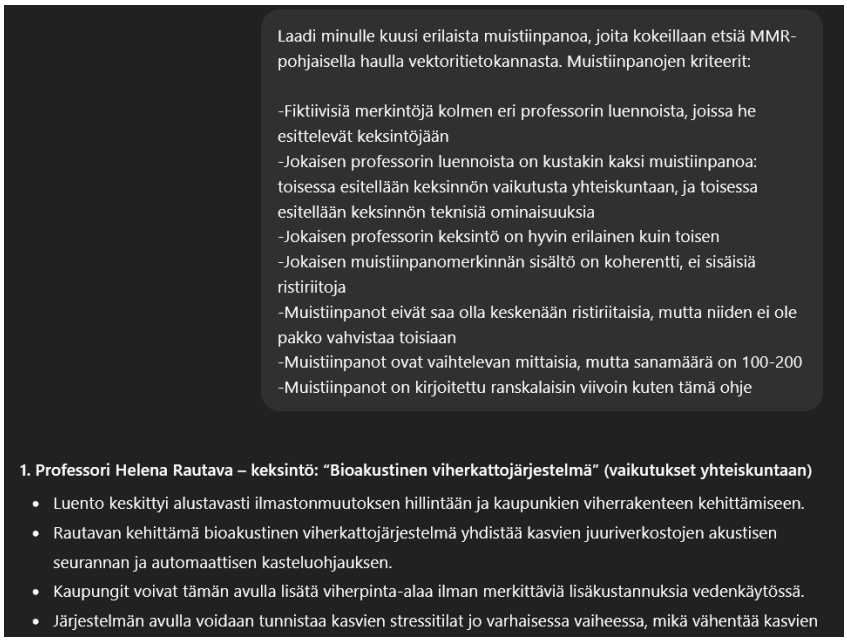
Muistiinpanot luotiin ja käännettiin englanniksi GPT5-mallin avulla ajansäästöllisistä syistä, minkä jälkeen niihin tehtiin hieman tyylitteleviä muokkauksia. Muokatut muistiinpanot tallennettiin NoteAppin avulla kolmeen eri kategoriaan, jotka kukin perustuvat yksittäisen professorin keksintöihin.

Testikysymykset olivat suomeksi:

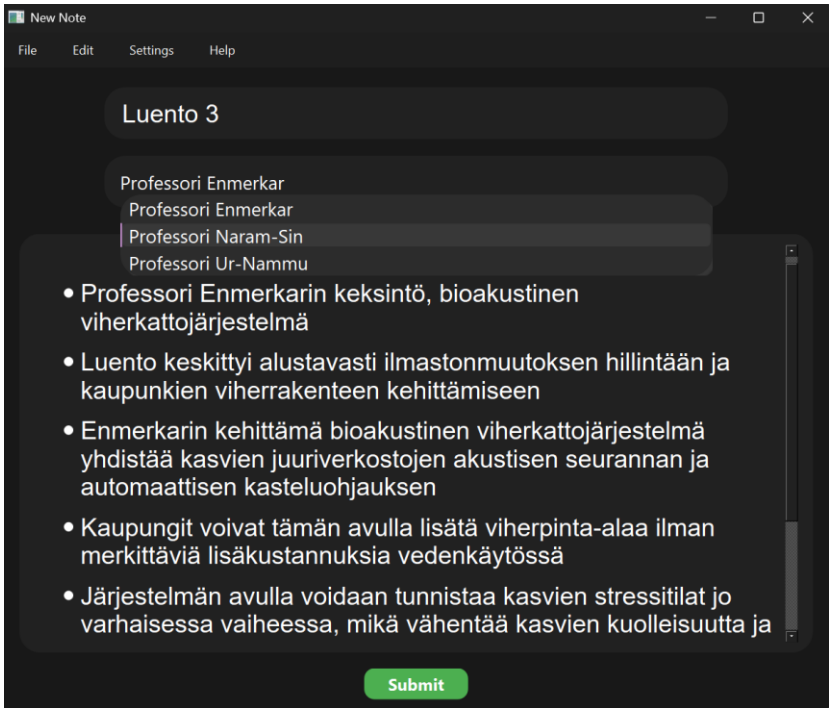
- Q1 Miten muistiinpanoissa esiintyvät keksinnöt pyrkivät vähentämään resurssien kulutusta?
- Q2 Mitä teknisiä ja ei-teknisiä rajoitteita oppimista tukevalla järjestelmällä on?
- Q3 Mitä kaupunkeja muistiinpanoissa mainittiinkaan?
- Q4 Millaisia yhtäläisyyksiä ja eroavaisuuksia näet keksintöjen välillä?

Ja englanniksi:

- Q1 How do the inventions mentioned in the notes aim to reduce resource consumption?
- Q2 What technical and non-technical limitations does the learning system have?
- Q3 Which cities were mentioned in the notes?
- Q4 What similarities and differences can you identify between the inventions?



Kuva 55. Muistiinpanojen luominen OpenAI:n GPT5-mallin avulla



Kuva 56. Muokattujen muistiinpanojen tallentaminen testejä varten

6.2 Suorituskykyvertailun metodiikka

Kielimallien suorituskykyä mitattiin ajallisena kestona Pythonin standardikirjaston time.perf_counter -metodin avulla. Mittaaminen alkaa kun Agent.run -metodi suorittaa ensimmäisen try-lohkonsa, jolloin tallennetaan heti ensimmäiseksi aloitusajanhetki muuttujaan $t0$. Seuraava ajanhetki tallennetaan muuttujaan $t1$, kun retriever on palauttanut vektoritietokannasta etsityt muistiinpanot, tai virheen sattuessa tyhjän listan. Viimeinen ajanhetki $t2$ tallennetaan heti sen jälkeen, kun LLM on palauttanut muistiinpanojen kontekstilla rikastetun vastauksensa. Vektoritietokantahaun kesto saadaan laskemalla $t1 - t0$, LLM:n vastauksen kesto saadaan laskemalla $t2 - t1$ ja näiden yhteenlaskettu kesto $t2 - t0$. Vastaukset on skaalattu millisekunneiksi ja tulostettu kielimallin tarjoaman vastauksen perään pyöristettynä lähimpään kokonaislukuun.

Jokaiselle kielimallille on esitetty neljä kysymystä Q_i ja jokainen kysymys on kysytty kultakin mallilta kolme kertaa peräkkäin. Saman kysymyksen peräkkäisyys ei vaikuta tulokseen, sillä sovelluksen testien aikaisessa versiossa malli ei saa keskusteluhistoriaa kontekstiksi. Suomea mielestäni riittävän hyvin osaaville kielimalleille (Gemma3, OpenEuroLLM-Finnish) kysymykset on esitetty suomeksi, mutta muille malleille kysymykset on esitetty englanniksi. Sovelluksen RAG-arkkitehtuurista johtuen englanninkielisten testien kohdalla vektoritietokannassa oli vain englanninkielisiä muistiinpanoja ja toisin päin suomeksi. Kun muistiinpanot olivat kaksikielisiä, retriever palautti ne välillä väärällä kielellä, mikä aiheutti kuvan 56 kaltaisia ongelmia erityisesti suomea taitamattomien mallien kanssa. Englanninkielisten testien ajaksi kuvassa 50 esitetystä järjestelmäkehotteesta poistettiin viimeinen rivi ”Always respond in Finnish”, joka kehottaa mallia vastaamaan aina suomeksi.



Kuva 57. Deepseek-r1:7b -malli hallusinoi käsitellessään kaksikielisiä muistiinpanoja

Mallien vastaukset arvioitiin ja pisteytettiin asteikolla: 1 = huono, 2 = kohtalainen, 3 = hyvä, 4 = erinomainen. Lopuksi kaikista yhden mallin vastauksista laskettiin keskiarvo. Vastauksia arvioitiin seuraavilla kysymyksillä:

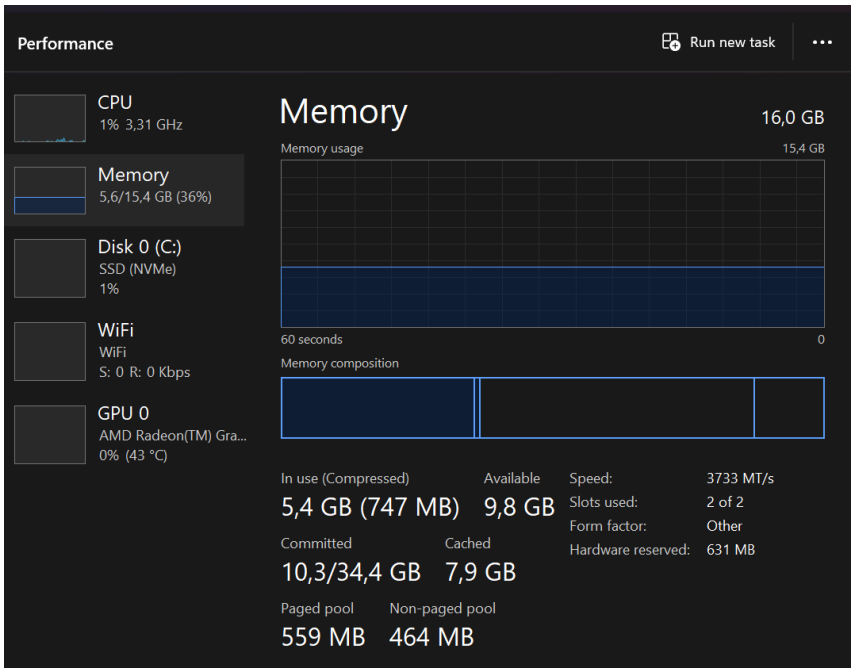
- Vastaako vastaus testikysymykseen?

- Sisältääkö vastaus virheellistä tai ristiriitaista tietoa?
- Sisältääkö vastaus virhepäätelmiä tai väitteitä, jota ei voi päätellä muistiinpanojen perusteella?
- Onko vastaus liian laaja tai liian suppea?
- Onko vastaus hyödyllinen helppolukuisuutensa puolesta?

6.3 Suorituskykyvertailun toteutusympäristö

Testeissä käytetty laite oli Windows 11 25H2 -käyttöjärjestelmällä varustettu kannettava tietokone, jonka tarkka malli on Asus ZenBook 13 UX325UA_UM325UA. Laitteen prosessori on AMD Ryzen 7 5700U, jossa on integroitu AMD:n näytönohjain (0x164C). Ollama ei tue kyseistä näytönohjainta eli kielimallit ajetaan täysin CPU:n varassa. Keskusmuistia laitteessa on yhteensä 16 GB, mutta aina ennen testien ajamista vapaata muistia oli resurssienhallinnan mukaan noin 10 GB.

Testien ajaksi laitteelta on suljettu internet-yhteys ja kaikki käynnissä olevat ohjelmat kuten selaimet ja tiedostohakemisto. Vain VSCode ja ChatApp olivat auki käyttöjärjestelmän välttämättömien taustaprosessien lisäksi. Laite oli alkuperäisessä laturissaan kiinni puupöydällä, ja virransäästöprofiilina oli ’high-performance’. Ennen testien aloittamista kannettava käynnistettiin uudelleen ja siihen tehtiin kaikki saatavilla olleet käyttöjärjestelmäpäivitykset. Aina kun uusi kielimalli otettiin käyttöön, sille tehtiin ylimääräinen lämmittelyajo. Testit suoritettiin Ollaman versiolla 0.12.10.



Kuva 58. Karkea kuvaus testiympäristön tilasta

6.3.1 Testiin valitut kielimallit

Sovelluksen testaamiseksi mukaan on valittu useita kielimalleja, jotka ovat vaihtelevan kokoisia ja pääosin tunnettujen yhtiöiden malleja. Kokeilin useita Ollaman tarjoamia kielimalleja komentorivin kautta, ja havaintojeni perusteella suurin osa niistä ei osannut Suomea tyydyttävällä tasolla. Kysyessäni esimerkiksi Microsoftin Phi3-mallilta, että mitä Newtonin lait ovat, sain tällaisen vastauksen:

”Ne on kyseenalainen teori, jonka tärkeitään monimuottaan löytämään molekyylivaihtoehtoja ja mukavapaa askeleilla. Monikäteen todennusjärjestys sallittuna kaiken kyseenalojien välisyyteen, johon ne liittävällä muita rakenteet edistävät maailmalleen laitamalla seurauksia...”

Suomea osaavia malleja löytyi Ollaman repositoriosta muutamia, mutta niiden käyttäminen edellytti joko liikaa keskusmuistia tai mallin hienosäätämistä. Esimerkiksi akx/viking-7b -malli ei vastannut esittämiini kysymyksiin, vaan jatkoi tekstin syöttämistä puppugeneraattorin tavoin. Massiivinen osoderholm/poro:34b ei taas kyennyt pyörimään kannettavallani, jossa on vain 16 GB keskusmuistia, mikä näkyykin esimerkkivirheenä kuvassa 44. Vain kaksi kokeilluista kielimalleista kommunikoi suhteellisen hyvällä suomen kielellä: Gemma3 ja OpenEuroLLM-Finnish, joka on rakennettu Gemman pohjalta.

Suuria kielimalleja voidaan kvantisoida eri tavoin. Kvantisoinnissa mallin painokertoimien arvoja kompressoidaan pienemmiksi. Liukulukuina esitettyjä neuroverkkojen painoja siis pyöristetään siten, että ne vievät vähemmän muistia. Esimerkiksi Q4 tarkoittaa sitä, että 32- tai 16-bittisenä liukulukuna laskettu paino kevennetään 4-bittiseksi. Kompression seurauksena malli on nopeampi ja vie vähemmän levytilaa, mutta samalla sen tarkkuus heikkenee. (Ilvez 2025.)

Gemma3 on Googlen kehittämä multimodaalinen eli sekä tekstiä, että kuvia käsittelevä kielimalli, joka perustuu Geminiin. Opinnäytetyöhön valittiin Gemma3:4b, joka pitää sisällään 4.3 miljardia parametria, ja sen konteksti-ikkuna on 128 tuhatta tokenia. Mallin kvantisointi on Q4_K_M. (Ollama s.a. b.)

OpenEuroLLM-Finnish on erityisesti suomenkielisen tekstin käsittelyyn hienosäädetty kielimalli, jonka pohjamallina on Googlen Gemma3. OpenEuroLLM-Finnish rakentuu 12.2 miljardista parametrista, ja sen konteksti-ikkuna on 128 tuhatta tokenia. Mallin kvantisointi on Q4_K_M. (Ollama s.a. c.)

Llama3.2 on Metan kielimalli, joka on erikoistunut monikielisyyteen. Näistä opinnäytetyöhön valittiin sekä pieni 1.24 miljardin parametrin Llama3.2:1b sekä hieman suurempi 3.21 miljardin parametrin Llama3.2:3b, joiden molempien konteksti-ikkunat ovat 128 tuhatta tokenia. Pienemmän mallin kvantisointi on Q8_0, ja suuremman Q4_K_M. (Ollama s.a. d.)

Phi3 on yksi ladatuimmista ja tällä hetkellä uusin Microsoftin kehittämistä paikallisista kielimalleista Ollaman sivuilla. Opinnäytetyöhön valittiin Phi3:3.8b, jossa on 3.82 miljardia parametria ja 128 tuhannen tokenin konteksti-ikkuna. Mallin kvantisointi on Q4_0. (Ollama s.a. e.)

DeepSeek-r1 on kiinalaisen tekoäly-yhtiö DeepSeekin julkaisema avoimen lähdekoodin malli. Näistä opinnäytetyöhön valittiin sekä pieni 1.78 miljardin parametrin DeepSeek-r1:1.5b ja suurempi 7.62 miljardin parametrin DeepSeek-r1:7b, joiden molempien konteksti-ikkunat ovat 128 tuhatta tokenia. Molempien mallien kvantisoinnit ovat Q4_K_M. (Ollama s.a. f.)

Mistral on suosittu ranskalaisen tekoäly-yrityksen (Mistral AI) julkaisema avoimen lähdekoodin kielimalli. Opinnäytetyöhön valittiin sen 7.25 miljardin parametrin versio Mistral:7b, jonka konteksti-ikkuna on 32 tuhatta tokenia. Mallin kvantisointi on Q4_K_M. (Ollama s.a. g.)

7 Tulokset ja pohdinta

7.1 Suorituskykytestin tulokset

Suorituskykytestien tulokset hämmensivät minua suunnattomasti, ja rikkoivat kaikki odotukseni. Tuloksia on esitelty taulukossa 2, joka on lajiteltu ensisijaisesti nousevassa järjestyksessä vastauksen laadun perusteella, ja toissijaisesti laskevassa järjestyksessä keskimääräisen vastausajan perusteella. Vastausten minimi-, maksimi-, mediaani- ja keskiarvokestot on pyöristetty lähimpään kokonaislukuun ja laskettu vastausten kokonaisaikojen perusteella eli ne sisältävät myös RAG-vaiheen. Vastausten keskiarvoiset laadut 1–4 on kuitenkin esitetty kahden desimaalin tarkkuudella. Sinisellä värikoodatut arvot ovat tuloksia malleista, joita testattiin vain suomenkielisillä kysymyksillä ja muistiinpanoilla; muut mallit testattiin englanniksi.

Taulukko 2. Suorituskykytestin suuntaa antavat tulokset

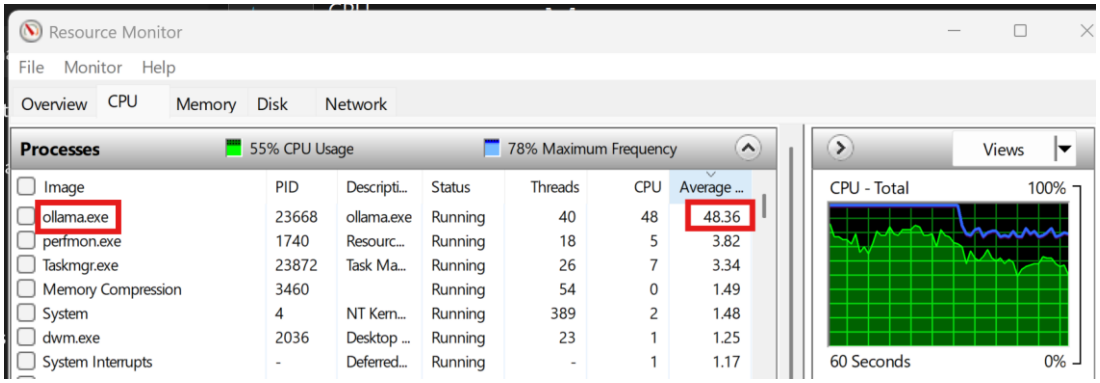
Malli	MIN (ms)	MAX (ms)	AVG (ms)	MEDIAN (ms)	AVG (q 1-4)
Phi3:3.8b	17680	159791	55250	69686	1,67
Deepseek-r1:1.5b	11818	42465	25167	26726	2,00
Deepseek-r1:7b	46413	187215	108436	105721	2,17
Mistral:7b	4679	83120	50557	49042	2,17
Llama3.2:1b	936	20045	16262	13270	2,33
Gemma3:4b	30222	109406	75759	71817	2,67
Llama3.2:3b	4712	35574	16226	19413	2,92
OpenEuroLLM-Finnish	148996	373153	303619	277451	3,50

Nopein yksittäinen vastaus kesti alle sekunnin ja hitain noin 6.2 minuuttia. Kielimallin valinnalla on siis selkeä merkitys. Tuloksissa ilmeni myös joitain kummallisuuksia. Esimerkiksi Deepseek-r1:n 7 miljardin parametrin malli pärjäsi käytännössä huonommin kuin sen 1.5 miljardin parametrin versio. Kyseinen Deepseek-r1:1.5b oli myös huomattavasti hitaampi kuin suurempi kolmen miljardin parametrin Llama3.2. Nämä erikoiset havainnot saivat minut pohtimaan sitä, että miten tulokset muuttuisivat, jos kutakin mallia testattaisiin kahdentoista kierroksen sijaan esimerkiksi sadalla kierroksella. Olisi myös mielenkiintoista selvittää tarkemmin, että miten muistiinpanojen sisältö vaikuttaa mallien toimintaan – olisivatko vastaukset merkittävästi huonompia ja niiden vasteajat entistä hitaampia, jos muistiinpanot kirjoitettaisiin hätäisesti ja kirjoitusvirheiden kanssa? Entä miten upotusmallin valinta vaikuttaa RAG-vaiheen alkupäässä lopputulokseen? Tällaiset kysymykset jäivät askarruttamaan minua, mutta en valitettavasti ehtinyt niitä tämän mittakaavan opinnäytetyössä enää tutkia.

Mielestäni englanninkielisiä muistiinpanoja käsitelleistä malleista Llama3.2:3b oli ylivoimaisesti paras. Sen keskimääräinen vastausaika oli noin 19,5 sekuntia, ja vastauksille antamieni pisteiden keskiarvo oli lähes 3 eli hyvä. Malli oli kaikista malleista toisiksi nopein ja sen vastaukset olivat

englanniksi testatuista malleista keskimäärin parhaat. Oikeastaan kaikista parhaimmat vastaukset kielestä riippumatta antanut OpenEuroLLM-Finnish ei ollut mielestäni radikaalisti parempi vastauksiltaan kuin LLama3.2:3b, ja sen keskimääräinen vastausaika oli yli neljä ja puoli minuuttia. Suomenkielisissä testeissä OpenEuroLLM-Finnish antoi laadukkaampia vastauksia kuin Gemma3, mutta vastaukset kestivät minuutteja eikä laatuero ollut mielestäni mitenkään dramaattinen. OpenEuroLLM-Finnish antoi nopeimman vastauksensa noin kahdessa ja puolessa minuutissa, kun Gemman hitain vastaus kesti alle kaksi minuuttia.

Googlen kehittämän Gemma3:n avulla suomenkielisten muistiinpanojen käsittely toimi siis hyvällä tasolla. Sen tuottamat vastaukset olivat pääosin hyviä, ja vastausten keskimääräinen aika noin sekunnin mittaisen RAG-vaiheen lisäksi oli noin 1 min 10 s. Tulkintani mukaan parhaiten suoriutunut malli antoi kuitenkin lähes yhtä hyviä vastauksia ~13 sekunnissa (Llama3.2:1b) ja parempia vastauksia ~20 sekunnissa (Llama3.2:3b).



Kuva 59. Ollama.exe -prosessin prosessorin käyttöaste ajettaessa raskainta kielimallia

7.2 Yleisiä ajatuksia sovelluksen toimivuudesta sekä kannattavuudesta

Tavoitteena oli luoda paikallisesti toimiva tekoälysovellus, joka pääsee käsiksi vain käyttäjän paikallisesti tallentamiin dokumentteihin. MemoFortin prototyyppi toimii teknisesti suunnitellulla tavalla eli täysin paikallisesti ilman verkkoyhteyttä. RAG-tekniikkaan pohjautuva tiedonhaku ja kontekstipohjaisten vastausten muodostaminen ovat haluamassani lopputuloksessa toimivia ja sovellus tuottaa valitusta kielimallista riippuen enemmän tai vähemmän olennaista sisältöä käyttäjän kysymyksiin. Visiossani MemoFortin ChatApp- ja NoteApp -prosessit voidaan käynnistää näppäimistön pikakomennolla, jotta muistiinpanoja voidaan tallentaa salamannopeasti. Tämä mahdollisuus jäi kuitenkin prototyypin osalta toteuttamatta. Sovellus on mahdollista paketoida nykyisessä muodossaan toimivaksi .exe -tiedostoksi Pyinstallerin avulla, mutta sen toimivuutta ei ole varmistettu toisella laitteella.

Suorituskykytestien perusteella sovellus suoriutuu nykytilassaan englanniksi merkittävästi paremmin kuin suomeksi. Sovelluksen tavoitteena oli hyvä käyttökokemus tavallisella kuluttajalle suunnatulla kannettavalla, mutta tämä ei käsitykseni mukaan ole vielä mahdollista Ollaman tukemilla suomenkielisillä kielimalleilla. Englanniksi tavoite täyttyi testiolosuhteissa mielestäni kaikin puolin.

Tähtäsin siihen, että lopputuloksena syntyvän sovelluksen käyttö ei vaatisi tietoteknistä osaamista, jotta se olisi saavutettavissa tavalliselle käyttäjälle. Tavoitteeni toteutui prototyypin kannalta osittain, koska jotkin toiminnallisuudet kuten järjestelmäkehotteen muokkaaminen edellyttävät nykytilassaan muutoksia lähdekoodiin. Sovelluksessa ei sinänsä ole mitään vallankumouksellisen uutta, mutta en löytänyt kirjallisuutta, jossa toteutettaisiin vastaavaa työkalua suomalaisesta perspektiivistä.

Testit olivat kuitenkin vain suuntaa antavia demonstraatioita sovelluksen toimintakapasiteetista, ja tulokset saattaisivat muuttua merkittävästikin erilaisilla muistiinpanotyypeillä, kysymyksillä ja upotusmalleilla. Ohjelma on myös todennäköisesti hitaampi kannettavan tietokoneen ollessa tavallisessa käytössä. Esitin OpenEuroLLM-Finnish -mallille testikysymyksen 3 tilanteessa, jossa laitteellani oli auki Microsoft Word, Microsoft Excel ja selain. Vastausaika piteni testien noin 150 sekunnista noin 193 sekuntiin.

7.3 Jatkokehitysideat

Toiminnallisuuksien puolesta sovellukseen voisi lisätä mahdollisuuden pysäyttää tekstin generointi kesken, sillä nyt mallille lähetettyä pyyntöä ei voi pysäyttää sulkematta sovellusta. UX:n kannalta myös kielimallin vaihtamista tulisi helpottaa. MemoFort voisi jatkossa hakea listan paikallisesti asennetuista kielimalleista ja antaa käyttäjän valita mallin sen kautta. Sovelluksessa voisi myös olla mahdollisuus muokata järjestelmäkehotetta ilman lähdekoodiin koskemista. Yksi suunnitelluista ominaisuuksista oli mahdollisuus muokata jo olemassa olevien muistiinpanoja. Käytännössä muistiinpanoja voi prototyypissä muokata jälkikäteen vain suoraan Windowsin tiedostohakemiston syövereistä löytyvän CSV-tiedoston kautta, mikä ei ole käyttäjäystävällinen prosessi.

Kehittäessäni sovellusta pidin kaiken tekoälylogiikan aluksi ChatAppin puolella. Jälkikäteen mietttien vektoritietokantaindeksointi olisi ollut järkevämpi toteuttaa NoteAppissa muistiinpanon tallentamisen yhteydessä, sillä ChatAppin avautuminen kestää melko kauan index_again -funktion suorittamisen takia. Kyseisen funktion voisi toki ajaa uudessa säikeessä, ja näyttää käyttäjälle jonkinlaisen splash screenin, mutta toisaalta alkuperäisessä tavoitteessa ChatAppin ja NoteAppin oli tarkoitus olla nopeasti avautuvia kelluvia ikkunoita.

Koodipohjaa voisi refaktoroida entistä modulaarisemmaksi. Esimerkiksi kaikki tyylit ja riippuvuudet olisi voitu paketoida omiin tiedostoihinsa. Sovellusta voitaisiin laajentaa myös esimerkiksi lisäämällä piirtotyökaluja tai käsitekarttatoimintoja eli rakentaa puitteet multimodaalisten mallien hyödyntämiseen esimerkiksi kuvien analysoinnissa ja tuottamisessa.

Sovellukseen voisi lisätä myös keskusteluhistorian tallentamisen siten, että aiemmin käytyjä keskusteluja voitaisiin tarkastella jälkikäteen. Tämän lisäksi keskustelun sisäinen historia olisi hyvä lisätä, jotta malli saisi kontekstina myös aiemmin keskustelussa lähetetyt viestit. Käyttäjäkokemus paranisi myös, jos viestihistorian voisi tyhjentää näytöltä. Visuaalisen ilmeen puolesta sovellukseen voisi lisätä erilaisia teemoja, mutta ainakin vaalean teeman. Thinking_bubble -instanssille voitaisiin antaa animaatio ja mallin vastaukset voitaisiin tulostaa sana kerrallaan jäljitellen suurkorporaatioiden palveluja kuten ChatGPT:n toimintaa. Nyt agentin tuottama viesti putkahtaa kaikki kerralla ulos. Jatkossa MemoFortista voisi tehdä Windows 11 lisäksi myös muissa käyttöjärjestelmissä toimivan version, ja siihen voisi lisätä tietoturvaominaisuuksia kuten esimerkiksi muistiinpanojen suojaamisen salausalgoritmin avulla.

Opinnäytetyöprosessin aikana mieleeni heräsi myös muutamia potentiaalisia tutkimusideoita. Esimerkiksi paikallisten kielimallien suorituskykyä voitaisiin tutkia tarkemmin eri konteksteissa. Etsiessäni suomenkielisiä kielimalleja löysin Ollaman sivuilta myös mallin ”AnhChu2402/gpt3--finnish-small”. Mallin lataussivulta ei löytynyt kuitenkaan muuta dokumentaatiota kuin linkki pohjamalliin TurkuNLP/gpt3-finnish-small HuggingFacen verkkosivustolla. Koska mallista ei löytynyt riittävän helposti varmaa tietoa, ja sillä oli vähän latauskertoja, en tietoturvallisuuden takia halunnut ladata sitä ollenkaan. Tämä sai minut pohtimaan tutkimusta kielimallirepositorioiden tietoturvallisuudesta yleisemmällä tasolla. Seuraava hyvä askel olisi myös perehtyä siihen, että miten muut kuin Ollaman tarjoilemat kielimallit pärjäävät vastaavissa tehtävissä.

7.4 Opinnäytetyön ja henkilökohtaisen kehittymisen reflektointi

Alkuperäinen suunnitelman sovellukselle oli melko yksinkertainen. Kaikki alkoi keväällä 2025, kun huomasin lataamani Ollama-sovelluksen käyttämisen edellyttäneen komentorivillä työskentelyä. Tämä sai minut pohtimaan, että miten paikallisen kielimallin kanssa keskustelemisesta voisi tehdä käyttäjäystävällisempää, ja että miten saisin sen esimerkiksi lukemaan omia tiedostojani. Näin keksin idean kahdesta modernin näköisestä ja helppokäyttöisestä ikkunasta. Toisen ikkunan tehtävä olisi tallentaa käyttäjän muistiinpanoja, ja toisen tehtävä toimia käyttöliittymänä, jossa käyttäjä voi keskustella paikallisen kielimallin kanssa.

Ollama julkaisi sovellukselleen graafisen keskusteluikkunan heinäkuussa 2025 (Ollama 30.07.2025). MemoFortin keskusteluikkuna ei ole yhtä tasokas kuin Ollaman sovellus eli sen hyödyllisyys ei perustu pelkkään mahdollisuuteen keskustella paikallisten kielimallien kanssa graafisen käyttöliittymän kautta. Ollaman keskusteluikkunaan on myös mahdollista pudottaa tiedostoja, mutta kyseessä ei ole vektoritietokantaan perustuva kontekstihaku, eikä tällainen ole ainakaan toistaiseksi Ollaman graafisessa sovelluksessa mahdollista.

Valitsin aiheen myös omasta kiinnostuksestani ripeästi kehittyviin, moderneihin tekoälyteknologioihin, ja siihen, kuinka niitä voidaan soveltaa käytännönläheisesti yksityisessä tiedonhallinnassa tietosuojaa ja yksityisyyttä kunnioittaen. Työssäni yhdistyvät sellaiset tietojenkäsittelytieteiden osa-alueet, joihin haluan syventyä ammatillisesti, kuten koneoppiminen, ohjelmistokehitys ja data-analytiikka.

Aloittaessani projektia minulla ei ollut lainkaan kokemusta paikallisen tekoälysovelluksen kehittämisestä. Ymmärrykseni neuroverkoista ja tekoälyn historiasta oli myös melko vaatimattomalla tasolla. Ymmärtääkseni syvien neuroverkkojen toimintaa luin opinnäytetyöprosessin aikana valtavasti alan kirjallisuutta, ja koen matemaattisen osaamiseni, ohjelmointitaitojeni ja käsitykseni tietojenkäsittelyn historiasta kehittyneen valtavasti projektin aikana.

Suurimmaksi hankaluudeksi osoittautui opinnäytetyön aikatauluttaminen. Projekti osoittautui hieman liian laajaksi suhteessa arvioimaani aikatauluun. Arvioin suurimmaksi ongelmaksi aikataulutuksen kannalta sen, että lähdin kirjoittamaan raportin tietopohjaa liian syväluotaavasti. Halusin toden teolla oppia kirjoittaessani ymmärtämään, että miten LLM-teknologiat toimivat aina ruohonjuuritasolta abstraktimpiin ulottuvuuksiin. En kuitenkaan lähtötilanteessa täysin hahmottanut, että mitä tämä käytännössä työmäärän osalta tulisi tarkoittamaan. Päätin jatkaa aloittamani urakan loppuun, sillä pidän loppupeleissä tietopääomani kehittymistä arvosanaa tärkeämpänä. MemoFortin vaatimusmäärittelyä, ja toteutukseen valittuja teknologioita kuten LangChainia ja ChromaDB:tä olisi voitu kuvailla hieman laajemmin.

Opinnäytetyön tutkimuksellisen ulottuvuuden oli alun perin tarkoitus olla laajempi. Suunnittelin suorittavani suuremmalla volyymilla testejä kielimalleille. Olisin halunnut tutkia myös eroja upotusmallien käytössä, ja sitä, että miten mallit suoriutuvat eri tyyppisistä muistiinpanoista. Opinnäytetyöprojekti oli kuitenkin jo valmiiksi hieman liian laaja.

MemoFortin kehittämisprosessin aikana kohtasin useita erilaisia haasteita. Olin aikaisemmin ohjelmoinut vain melko yksinkertaisia komentorivipohjaisia ohjelmia ja verkkosivuja. Opintojen aikana kertyneen osaamisen tukena käytin niin dokumentaatiota, runsaasti erilaisia opastusvideoita YouTubessa kuin satunnaisesti tekoälyäkin (GPT-5 ja GPT-4o). Koodia ei kopioitu ja liitetty tekoälyn

vastauksista ajattelemattomasti vaan tekoälyä käytettiin pääasiassa apurina lähestymistapojen ideoinnissa ja muutamissa haastavimmissa tilanteissa virheiden analysoinnissa. Ymmärrän koodin rakenteen ja toiminnan läpikotaisin. Haasteet syntyivät lähinnä siitä, etten kokemattomuuteni puolesta osannut ennakoida tiettyjen toiminnallisuuksien välttämättömyyttä. Esimerkiksi se, että keskustelunäkymän tulisi vieriä pohjalle uuden viestin ilmestyessä ei heti juolahtanut mieleeni.

Pienistä aikatauluhaasteista huolimatta sain kuitenkin sovelluksen toimimaan fundamentaalisella tasolla kuten olin suunnitellut. Käyttöliittymästä tuli miellyttävän moderni ja helppokäyttöinen, ja sovelluksen ydinlogiikka on kunnossa. Koen projektissa kertyneen ammatillisen kehitykseni valtavaksi.

Lähteet

Bassett, C. 2019. The computational therapeutic: exploring Weizenbaum's ELIZA as a history of the present, AI and Society, 34(4), s. 803–812. Luettavissa: https://doi.org/10.1007/S00146-018-0825-9. Luettu 18.8.2025.

Berry, D. M. 2023. The Limits of Computation: Joseph Weizenbaum and the ELIZA Chatbot. Weizenbaum Journal of the Digital Society, 3(3), s. 11–17. Luettavissa: https://doi.org/10.34669/WI.WJDS/3.3.2. Luettu 18.8.2025.

Brousseau, C., Sharp, M. 2025. LLMs in production: from language models to successful products. Manning. New York.

Carbonell, J., Goldstein, J. 1998. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. Luettavissa: https://doi.org/10.1145/290941.291025.

Carlini, N., Liu, C., Erlingsson, Ú., Kos, J., Song, D. 2019. The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks. Luettavissa: https://doi.org/10.48550/arXiv.1802.08232.

Cox, A.M., Mazumdar, S. 2024. Defining artificial intelligence for librarians. Journal of Librarianship and Information Science, 56(2), s. 330–340. Luettavissa: https://doi.org/10.1177/09610006221142029.

Dhamani, N., Engler, M. 2024. Introduction to Generative AI. Manning. New York.

Drori, I. 2023. The Science of Deep Learning. Cambridge University Press. Cambridge.

Du, K.-L., Leung, C.-S., Mow, W.H., Swamy, M.N.S. 2022. Perceptron: Learning, Generalization, Model Selection, Fault Tolerance, and Role in the Deep Learning Era. Mathematics. 10(24), s. 4730. Luettavissa: https://doi.org/10.3390/math10244730.

Elon University News Bureau. 12.3.2025. Survey: 52 % of U.S. adults now use AI large language models like ChatGPT. Luettavissa: https://www.elon.edu/u/news/2025/03/12/survey-52-of-u-s-adults-now-use-ai-large-language-models-like-chatgpt/. Luettu 7.11.2025.

GeeksforGeeks. 23.07.2025. Ollama Explained: Transforming AI Accessibility and Language Processing. Luettavissa: https://www.geeksforgeeks.org/artificial-intelligence/ollama-explained-transforming-ai-accessibility-and-language-processing/. Luettu 11.10.2025.

Gemini Apps Privacy Hub. 02.09.2025. Luettavissa: https://support.google.com/gemini/answer/13594961#human_review. Luettu 3.10.2025.

Goel, S., Webb, E. 06.08.2025. Business Insider. Contractors training Meta's AI say they read intimate talks with its chatbot — and see data that identifies users. Luettavissa: https://www.businessinsider.com/meta-ai-chatbot-privacy-user-names-data-contractors-scale-alignerr-2025-8 . Luettu 3.10.2025.

Gollapudi., S. 2016. Practical machine learning: tackle the real-world complexities of modern machine learning with innovative and cutting-edge techniques. Packt Publishing Ltd. Birmingham.

Goodfellow, I., Bengio, Y., Courville, A. 2016. Deep Learning. The MIT Press. Cambridge.

Griewank, A. 2012. Who Invented the Reverse Mode of Differentiation? Documenta Mathematica. Extra, ISMP, s. 389–400. Luettavissa: https://doi.org/10.4171/dms/6/38.

Guran, N., Knauf, F., Ngo, M., Petrescu, S., Rellermeyer, J. S. 2024. Towards a Middleware for Large Language Models. Luettavissa: https://doi.org/10.48550/arXiv.2411.14513.

Ilvez, P. 08.06.2025. Medium. Demystifying LLM Quantization Suffixes: What Q4_K_M, Q8_0, and Q6_K Really Mean. Luettavissa: https://medium.com/@paul.ilvez/demystifying-llm-quantization-suffixes-what-q4-k-m-q8-0-and-q6-k-really-mean-0ec2770f17d3. Luettu 07.11.2025.

Jiang, Y., Li, X., Luo, H., Yin, S., Kaynak, O. 2022. Discover Artificial Intelligence Quo vadis artificial intelligence? Discover Artificial Intelligence, 2, s. 4. Luettavissa: https://doi.org/10.1007/s44163-022-00022-8..

Kelleher, J. D. 2019. Syväoppiminen. 2. painos. Terra Cognita. Helsinki.

Kimothi, A. 2025. A Simple Guide to Retrieval Augmented Generation. Manning. New York.

Kämäräinen, J. 2023. Koneoppimisen Perusteet. Otatieto. Helsinki.

Lappin, S. 2025. Understanding the Artificial Intelligence Revolution - Between Catastrophe and Utopia. 1. painos. CRC Press. Florida & Abingdon (Oxfordshire).

Lefkowitz, M. 25.9.2019. Professor’s perceptron paved the way for AI – 60 years too soon | Cornell Chronicle. Luettavissa: https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon. Luettu 2.9.2025.

Magesh V., Surani, F., Dahl, M., Suzgun, M., Manning, C. D., Ho, D. E. 2024. Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools. Luettavissa: https://doi.org/10.48550/arXiv.2405.20362.

McCulloch, W. S., Pitts, W. 1943. A logical calculus of the ideas immanent in nervous activity, The Bulletin of Mathematical Biophysics, 5(4), s. 115–133. Luettavissa: https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943 Luettu 24.8.2025.

Ollama s.a. a. Api Reference. Luettavissa: https://docs.ollama.com/api Luettu 30.10.2025.

Ollama s.a. b. Gemma3. Luettavissa: https://ollama.com/library/gemma3:4b Luettu 30.10.2025.

Ollama s.a. c. OpenEuroLLM-Finnish. Luettavissa: https://ollama.com/jobautomation/OpenEuroLLM-Finnish:latest Luettu 30.10.2025.

Ollama s.a. d. Llama3.2 Luettavissa: https://ollama.com/library/llama3.2 Luettu 30.10.2025.

Ollama s.a. e. DeepSeek-r1. Luettavissa: https://ollama.com/library/deepseek-r1 Luettu 30.10.2025.

Ollama s.a. f. Phi3. Luettavissa: https://ollama.com/library/phi3:3.8b Luettu 30.10.2025.

Ollama s.a. g. Mistral. Luettavissa: https://ollama.com/library/mistral Luettu 30.10.2025.

Ollama. 30.07.2025. Ollama's new app. Luettavissa: https://ollama.com/blog/new-app. 30.Luettu 9.2025.

OpenAI. 18.09.2025. Läpinäkyvyys ja sisällön moderointi. https://openai.com/fi-FI/transparency-and-content-moderation/. Luettu 3.10.2025.

Qt Company Ltd. 2023. Qt for Python – QMainWindow. Luettavissa: https://doc.qt.io/qtforpython-6.5/PySide6/QtWidgets/QMainWindow.html#PySide6.QtWidgets.PySide6.QtWidgets.QMainWindow.menuBar. Luettu 9.10.2025.

Roberts, J. 2024. How Powerful are Decoder-Only Transformer Neural Models? International Joint Conference on Neural Networks (IJCNN). Yokohama. Luettavissa: https://www.researchgate.net/publication/383894147_How_Powerful_are_Decoder-Only_Transformer_Neural_Models. Luettu 1.11.2025.

Rumelhart, D.E., Hinton, G.E. and Williams, R.J. 1986. Learning representations by back-propagating errors, *Nature 1986 323:6088*, 323(6088), s. 533–536. Available at: https://doi.org/10.1038/323533a0.

Sanderson, G., Pullen, J. 2017. But what is a Neural Network? Luettavissa: https://www.3blue1brown.com/lessons/neural-networks. Luettu 14.9.2025.

Sanderson, G. Sun, J. 2024. Transformers, the tech behind LLMs | Deep Learning Chapter 5. Luettavissa: https://www.3blue1brown.com/lessons/gpt. Luettu 1.11.2025.

Straßer, P. 2025. Diversifying search results with Maximum Marginal Relevance. Luettavissa: https://www.elastic.co/search-labs/blog/maximum-marginal-relevance-diversify-results. Luettu 25.10.2025.

Valiant, L. G. 1984. A Theory of the Learnable. Communications of the ACM. Luettavissa: https://doi.org/10.1145/1968.197 Luettu 5.9.2025.

Vaswani, A. Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. 2017. Attention Is All You Need. Luettavissa: https://doi.org/10.48550/arXiv.1706.03762.

Webster, J., Kit, C. 1992. Tokenization as the initial phase in NLP. Luettavissa: https://dl.acm.org/doi/pdf/10.3115/992424.992434.

Weizenbaum, J. 1967. Contextual Understanding by Computers. Luettavissa: https://dl.acm.org/doi/pdf/10.1145/363534.363545. Luettu 23.8.2025.

Liitteet

Liite 1. Suorituskykytestien tulokset, raakadata

Taulukko 3. Testitulokset, Gemma 3:4b

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	109	52926	53035	2
Q1	2	104	57860	57964	1
Q1	3	108	60920	61028	3
Q2	1	104	90387	90490	2
Q2	2	107	99394	99502	4
Q2	3	108	93686	93794	2
Q3	1	83	30215	30299	3
Q3	2	85	32058	32143	3
Q3	3	80	30143	30222	3
Q4	1	98	109308	109406	3
Q4	2	102	95552	95654	2
Q4	3	103	108167	108270	4

Taulukko 4. Testitulokset, OpenEuroLLM-Finnish:latest

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	110	270520	270630	3
Q1	2	109	291353	291462	3
Q1	3	109	298271	298379	3
Q2	1	104	328969	329073	3
Q2	2	780	335744	336524	3
Q2	3	768	308091	308859	3
Q3	1	87	148909	148996	4
Q3	2	80	151575	151655	4
Q3	3	80	150621	150701	4
Q4	1	99	320297	320396	4
Q4	2	98	373055	373153	4
Q4	3	764	348820	349583	4

Taulukko 5. Testitulokset, Llama3.2:1b

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	71	17474	17545	3
Q1	2	73	11753	11826	2
Q1	3	72	16016	16088	1
Q2	1	69	17238	17307	2
Q2	2	69	19976	20045	1
Q2	3	68	15525	15592	2
Q3	1	63	4166	4229	4
Q3	2	58	878	936	4
Q3	3	60	2428	2488	4

Q4	1	63	16373	16436	1
Q4	2	63	17304	17367	3
Q4	3	64	19312	19377	1

Taulukko 6. Testitulokset, Llama3.2:3b

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	72	20940	21012	2
Q1	2	66	11791	11857	2
Q1	3	70	12930	13000	2
Q2	1	69	30814	30882	2
Q2	2	68	15207	15275	3
Q2	3	67	16391	16458	3
Q3	1	62	15931	15993	4
Q3	2	58	4654	4712	4
Q3	3	58	4987	5045	4
Q4	1	65	35509	35574	3
Q4	2	66	33774	33840	3
Q4	3	63	29240	29303	3

Taulukko 7. Testitulokset, Deepseek-r1:1.5b

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	67	42398	42465	2
Q1	2	69	24413	24481	3
Q1	3	72	21346	21419	2
Q2	1	67	39211	39278	1
Q2	2	68	29689	29757	1
Q2	3	69	34056	34125	1
Q3	1	62	18513	18575	2
Q3	2	61	18604	18665	4
Q3	3	58	11760	11818	1
Q4	1	66	25786	25852	1
Q4	2	64	31306	31369	3
Q4	3	63	22848	22911	3

Taulukko 8. Testitulokset, Deepseek-r1:7b

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	76	75540	75616	2
Q1	2	69	79715	79783	1
Q1	3	72	108074	108146	1
Q2	1	71	141701	141771	1
Q2	2	69	108657	108726	1
Q2	3	70	112966	113036	1
Q3	1	63	65849	65912	4
Q3	2	61	48728	48789	4
Q3	3	60	46353	46413	4

Q4	1	62	187153	187215	1
Q4	2	69	124435	124504	3
Q4	3	65	168681	168746	3

Taulukko 9. Testitulokset, Phi3:3.8b

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	72	49973	50045	1
Q1	2	72	32355	32427	2
Q1	3	73	53513	53585	1
Q2	1	72	113056	113128	2
Q2	2	73	17994	18067	1
Q2	3	72	144770	144842	1
Q3	1	61	17619	17680	4
Q3	2	61	84397	84458	2
Q3	3	61	159730	159791	1
Q4	1	729	45129	45857	1
Q4	2	65	59368	59433	2
Q4	3	65	56849	56914	2

Taulukko 10. Testitulokset, Mistral:7b

Kysymys	Ajo	RAG (ms)	LLM (ms)	Total (ms)	Vastauksen laatu
Q1	1	75	64055	64130	2
Q1	2	75	41563	41638	3
Q1	3	74	40324	40397	3
Q2	1	68	73104	73172	2
Q2	2	69	44768	44837	1
Q2	3	72	57068	57140	1
Q3	1	65	33466	33531	4
Q3	2	65	4614	4679	2
Q3	3	61	13510	13571	2
Q4	1	70	83050	83120	1
Q4	2	67	75948	76015	2
Q4	3	66	56210	56276	3